



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - ВАРНА

Факултет по изчислителна техника и автоматизация

ДИПЛОМНА РАБОТА

Проектиране и реализация на библиотека за предоставяне на облачни услуги

Дипломант: Явор Йорданов Чамов, фак. № 21621577

Специалност: Софтуерни и интернет технологии

Научен ръководител: преп. инж. В. Колесниченко

Варна

2025

Съдържание

Съдържание.....	3
Глава 1. Увод	5
Глава 2. Анализ на проблема и избор на технологии	7
2.1 Анализ на проблема	7
2.2 Сравнение със съществуващи решения.....	7
2.3 Цел и задачи	8
2.4 Използвани технологии и аргументация	9
2.5 Изводи.....	11
Глава 3. Проектиране на системата.....	12
3.1 Изисквания към програмната система.....	12
3.2 Архитектура на системата	13
3.2.1 Пакетна структура на системата	16
3.2.2 Пакетна структура на ресурсен модул.....	19
3.2.3 Архитектура на автентикационния слой	21
3.2.4 Базова конфигурационна архитектура на ресурси	22
3.2.5 Конфигурационни списъци и паралелност при създаване на ресурси.....	24
3.2.6 Интерфейсна архитектура на стъпките за контролиране на ресурс	26
3.2.7 Интерфейсна архитектура на операциите по управление на ресурсите.....	30
3.2.8 Изпълнители на стъпки и стратегии за изпълнение	31
3.2.9 Управление на състоянието на инфраструктурата.....	34
3.2.10 Обединяване на ресурсната логика в единен компонент	37
3.2.11 Архитектура за обработка на изключения.....	41
Глава 4. Реализация на системата	44
4.1 Автентикация към AWS чрез IAM потребител.....	44
4.2 Автентикация чрез Vault и дистрибутиране на ключ чрез алгоритъм на Шамир.....	46
4.3 Репрезентация на ресурси чрез конфигурационни класове	49
4.4 Зареждане на ресурсна конфигурация от файл.....	53
4.5 Обединяване на архитектурните зависимости и компоненти	56
4.6 Прилагане на инфраструктурни промени	58
4.6.1 Начално зареждане на състоянието	58
4.6.2 Запазване на неизменна информация чрез стъпка с метайнформация.....	59
4.6.3 Динамично внедряване на зависимости.....	60
4.6.4 Зареждане на актуално живо състояние	63
4.6.5 Сравнение между последно записано и актуално живо състояние	65
4.6.6 Прилагане на промени върху реалната инфраструктура	66
4.6.8 Гарантиране на атомарност чрез транзакционно възстановяване	68

4.6.9 Генериране и прилагане на желаното състояние	69
4.7 Стъпки за създаване и модифициране на S3 ресурси	70
4.7.1 Основен ресурс	71
4.7.2 Версионизиране на обекти	73
4.7.3 Криптиране на съхранени данни.....	75
4.7.4 Прилагане на маркери	77
4.7.5 Политика за достъп	79
4.7.6 Контрол на собствеността.....	81
4.7.7 Списъци за контрол на достъп	83
4.8 Стъпки за създаване и модифициране на EKS клъстер	86
4.8.1 Основа на клъстера	91
4.8.2 Група от работни виртуални машини	93
4.8.3 Системни приставки.....	95
Глава 5. Тестове и резултати.....	99
5.1 Автентикация	99
5.1.1 Vault.....	100
5.1.2 Статични AWS ключове с променливи на средата	101
5.1.3 Запазен профил от AWS CLI	102
5.2 Услугата Amazon S3	103
5.2.1 Създаване на нов ресурс	103
5.2.2 Модификация на вече съществуващ ресурс.....	105
5.2.3 Валидиране на прилагането на ръчно дефинирана ACL политика.....	108
5.2.4 Прилагане на ръчно зададен KMS ключ за криптиране чрез LocalStack.....	110
5.2.5 Проверка на функционалността за възстановяване	112
5.2.6 Предотвратяване на изтриване на главен ресурс	114
5.2.7 Реална операция по качване на ресурс	119
5.2.8 Окончателно изтриване на ресурсите.....	119
5.3 Услуга EKS.....	121
5.3.1 Цялостен процес по създаване на клъстер.....	121
Глава 6. Заключение	124
Източници	125
Приложение	126

Глава 1. Увод

В настоящия етап и развитие на софтуерна разработка и бърза еволюция на технологиите, изграждането на ефективна и сигурна облачна инфраструктура е от съществено значение за всяка организация. Все по-често се наблюдава необходимост от решения, които позволяват едновременно бързо разработване на нови услуги и централизирано управление на инфраструктурата. Настоящата дипломна работа цели разработването на библиотека за декларативно изграждане на облачни ресурси, която предоставя унифициран подход към управлението на инфраструктурата и улеснява сътрудничеството между различни инженерни екипи и отдели.

Мотивацията за реализирането на тази библиотека произтича от нуждата да се преодолее съществуващият проблем с липсата на координация между екипите по разработка и инфраструктура.

В рамките на тази дипломна работа се разглеждат два основни типа отдели: отдел „Разработка“ и отдел „Инфраструктура“. Отдел „Разработка“ е фокусиран върху изграждането на продуктовата линия и бизнес логика, осигуряваща основната добавена стойност за организацията. Отдел „Инфраструктура“ носи отговорността по създаването, поддръжката и развитието на облачната платформа, която служи като фундамент за работата на екипа по разработка.

Разработчиците имат нужда от свобода и бързина при създаването на нужната им инфраструктура за конкретна функционалност, като е важно този процес да се осъществява на разбираем за тях програмен език, за да бъде интуитивен и лесно усвоим. От друга страна, инфраструктурните инженери следва да осигурят устойчивост, сигурност и съответствие на инфраструктурата с организационните политики. Едновременно с това трябва да предоставят на разработчиците интуитивен и структуриран начин за дефиниране на нужната им инфраструктура. Съществуващите решения към момента често изискват допълнително обучение и усвояване на нови декларативни езици със специфичен синтаксис, възможности и ограничения. Това затруднява използването им от екипите по разработка и води до забавяне и усложняване на процеса.

Разработваната библиотека използва декларативен подход чрез конфигурационни файлове за създаване на ресурси в облачна среда. Поддържа проследяване на състоянието на инфраструктурата и прилагане на промени чрез валидиран и контролиран процес. Библиотеката използва структуриран файлов формат за декларативно описание на ресурсите, но зад всяка конфигурация стои предвидима и разбираема логика, реализирана на използвания в организацията програмен език. Това позволява на разработчиците лесно да разберат поведението на всяко свойство, без да се налага да усвояват изцяло нови инструменти или езици.

Същевременно, създаването и поддръжката на конфигурационните файлове е изцяло отговорност на екипа от разработчици, който ги дефинира, което допринася за самостоятелност и ясното разпределение на отговорности.

Сценарият, който поражда нуждата от тази автоматизирана система, включва типична разработка на нова функционалност в даден продукт, при която възниква необходимост от създаване или промяна на инфраструктурни облачни ресурси. Без централизирано решение, всеки екип би могъл да прилага собствени подходи, което води до рискове от нарушаване на сигурността, увеличаване на разходите и затруднения в поддръжката. С внедряването на библиотеката се изгражда балансирана и мащабируема архитектура, в която инфраструктурните промени се извършват автоматизирано и в съответствие с предварително дефинирани правила и процеси за одобрение.

Библиотеката е изградена с идеята за отвореност и колективно развитие. Тъй като е предвидена да бъде написана на утвърдения в организацията програмен език, инженерите от различни екипи могат лесно да се ориентират в кода, да предложат подобрения и да добавят поддръжка за нови облачни услуги. Това се осъществява чрез процес на споделено развитие, в който допълненията се реализират под формата на заявки за сливане и преминават през преглед и одобрение от страна на инфраструктурния екип. По този начин се насърчава сътрудничеството между екипите, като същевременно се запазва високо ниво на контрол, сигурност и съответствие с архитектурните стандарти на организацията.

Глава 2. Анализ на проблема и избор на технологии

2.1 Анализ на проблема

В контекста на съвременното софтуерно инженерство все по-често възниква необходимостта от автоматизирано и контролирано създаване на инфраструктура в облачна среда. Особено в големи организации, където множество екипи работят паралелно, липсата на централизирано решение за управление на инфраструктурата води до сериозни предизвикателства — от нарушаване на сигурността и непредвидими разходи, до затруднена поддръжка и липса на проследимост.

В разработката на нови функционалности често се налага създаването на нови облачни ресурси. Когато това се извършва ръчно или с помощта на различни инструменти и практики, дефинирани индивидуално от всеки екип, се губи съгласуваност и се увеличават рисковете. Проблемът, който дипломната работа разглежда, е именно стремеж към унифициран, сигурен и лесен за използване подход за изграждане на облачна инфраструктура, който да е съвместим както с нуждите на разработчиците, така и с изискванията на инфраструктурния екип.

2.2 Сравнение със съществуващи решения

Сред най-широко използваните решения за управление на инфраструктура чрез код е Terraform — инструмент, базиран на свой собствен декларативен език *HashiCorp Configuration Language*. Terraform създава и управлява ресурси на облачни платформи и други услуги чрез техните интерфейси за приложно програмиране (API). Доставчиците позволяват на Terraform да работи с почти всяка платформа или услуга с достъпен приложен интерфейс.[1] Въпреки широката му популярност, Terraform изисква от инженерите да изучат нов синтаксис и да се съобразяват с неговите ограничения. Това създава допълнителна сложност и обучителна крива с тенденция за растеж, особено за разработчици, фокусирани върху бизнес логиката.

В допълнение, Terraform предоставя единствено конфигурационен модел и не дава на инфраструктурния екип контрол върху начина, по който програмата комуникира с приложния програмен интерфейс на облачния доставчик. Всички операции по създаване, актуализация и изтриване на ресурси се управляват от изходния код на доставчика. Това ограничава възможностите за прилагане на допълнителна логика, правила или динамични проверки по време на изпълнение. Разработената библиотека осигурява пълна гъвкавост при обработката на

конфигурациите — въпреки че ресурсите се описват декларативно, тяхното създаване и управление се извършва чрез собствена императивна логика. Това позволява внедряване на специфични правила, адаптиране на поведението спрямо контекста, валидиране на стойности, динамично изчисляване на параметри и пълна проследимост при взаимодействие с облачния доставчик.

Макар да се различава по архитектура и контрол, разработената библиотека заимства и адаптира някои утвърдени практики от Terraform. [2] Като положителен пример е възприето разделянето на прилагане на промени по инфраструктурата на отделни стъпки, което улеснява проследимостта, отстраняването на грешки и прилагането на частични промени. Всяка стъпка отговаря за конкретен ресурс или свойство към този ресурс, което подобрява прозрачността и модулността на процеса. Друга добра практика, заимствана от Terraform, е поддържането на системни файлове със състоянието на инфраструктурата. Чрез сравнение между желаното състояние (дефинирано в конфигурационните файлове) и текущото реално състояние на инфраструктурата, библиотеката може да изчисли разликите и да предприеме действия за изравняване на двете състояния чрез добавяне, редактиране или изтриване на ресурси или техни свойства.

2.3 Цел и задачи

Целта на настоящата дипломна работа е създаването на библиотека, която позволява декларативно описание на желаната инфраструктура чрез конфигурационни файлове, като същевременно контролът върху реализацията остава в ръцете на инфраструктурните инженери. Основните задачи включват:

- дефиниране на подходящ архитектурен модел
- осигуряване на сигурна автентикация чрез външна система
- предоставяне на възможност за валидиране и преглед на промени
- изграждане на механизъм за следене на промени
- поддръжка на разширяемост и модулност

Разработеното решение въвежда структуриран и устойчив модел, който предлага ефективен баланс между контрол, автоматизация и сътрудничество, като:

- използва познат на организацията език за програмиране
- осигурява декларативен слой за описание на желаното състояние на ресурсите
- прилага императивна логика в основата на системата, която се управлява от инфраструктурния екип

- гарантира сигурност и валидност чрез централизирани политики и автоматична валидация
- осигурява възможност за лесно разширение и колективен принос чрез заявки за сливане в хранилището на проекта

Допълнително, дипломната работа демонстрира пълния път на софтуера – от дефинирането на инфраструктурата на локалната машина на програмиста до реалното ѝ прилагане в облачен доставчик. Като краен резултат, се създават виртуални ресурси, които могат да бъдат достъпвани и използвани от други услуги или инженери в рамките на системата.

2.4 Използвани технологии и аргументация

Следните технологии са избрани в реализацията на проекта, с оглед на нуждата от стабилност, сигурност, лесна интеграция и възможност за тестване:

- **Java 21** (Amazon Corretto 21): Използван като основен програмен език поради широкото му разпространение в индустрията и стабилната поддръжка. Java все по-често се използва в облачни среди, особено в разработката на архитектури с микро услуги посредством Spring Cloud библиотеки. [3] При реализацията на библиотеката Java предоставя редица предимства като управление на изключения, поддръжка на многонишковост, обектно-ориентиран подход, строга типизация и компилационна проверка.
- **AWS SDK for Java**: Осигурява директен достъп до AWS ресурси и позволява пълна контролируемост чрез Java. Използването на структура на изделие (Bill of Materials) улеснява версионизирането и съвместимостта между модулите. Предоставя императивен интерфейс, който дава възможност за прецизен контрол върху създаването, актуализацията и изтриването на ресурси – всяка стъпка може да бъде управлявана програмно, с възможност за динамична логика, валидации и условно поведение. [4]
- **Google Guice**: Използван за внедряване на зависимости (Dependency Injection) в библиотеката. Чрез него се постига модулност, по-добра разделяемост на компонентите и улесняване на тестването. [5] Позволява лесно конфигуриране на поведението в зависимост от средата или конкретната реализация, което е особено полезно при добавяне на нови модули за различни облачни доставчици. За дефиниране на зависимостите се използват анотации от стандарта JSR-330, които осигуряват по-добра

четимост и съвместимост с други работни рамки за внедряване на зависимости. [6]

- **Jackson:** Използван за десериализация и сериализация на конфигурационните файлове. Jackson предоставя надежден и разширяем механизъм за десериализация, който позволява декларативно описаните ресурси да бъдат трансформирани в обектни модели, използвани от библиотеката в работно състояние. Чрез поддръжката на персонализирани правила за десериализация и анотации се осигурява гъвкавост при разширяване на конфигурационния формат. [7]
- **Vault:** Предоставя централизиран, добре одитиран привилегирован достъп и управление на критично важни тайни данни, независимо дали става въпрос за локални системи, в облака или в хибридна среда. Тайните са чувствителни, отделни части от информацията, като идентификационни детайли, ключове за криптиране, сертификати за удостоверяване и други критични части от информацията. [8] В библиотеката е използван за осигуряване на сигурна автентикация и достъп до ключове за автентикация с доставчик на облачни услуги.
- **LocalStack:** Позволява локално и изолирано тестване на AWS инфраструктура, без реален достъп до програмния интерфейс на доставчика на облачните услуги. Това значително улеснява интеграционното тестване и ускорява разработката. [9]
- **Docker:** Използва се за контейнеризация при стартиране на свързаните с библиотеката компоненти като Vault и LocalStack.
- **SLF4J:** Simple Logging Facade for Java служи като фасада или абстракция за различни работни рамки за регистрация на логове, позволявайки на крайния потребител да включи желаната рамка за регистриране по време на внедряването. [10] Позволява проследяване на събития, диагностика и мониторинг по време на изпълнение на библиотеката.
- **JUnit 5:** Избран като тестова рамка за модулно тестване на компонентите. Позволява лесно създаване на надеждни автоматизирани тестове и поддръжка на добри практики в разработката.
- **Gradle:** Използван като система за изграждане и управление на зависимостите. Библиотеката е предоставена като стандартна Gradle зависимост, както и под формата на Gradle плъгин, което улеснява интеграцията ѝ в различни проекти. Чрез добавяне на съответния плъгин, потребителите получават достъп до предварително дефинирани Gradle задачи, които стартират библиотеката с конкретни аргументи и променливи на средата. Това позволява автоматизирано прилагане на промени по

инфраструктура в различни среди чрез лесно и конфигурируемо изпълнение.

2.5 Изводи

Реализира се решение, като съчетава предимствата на декларативното описание с контролирано и императивно изпълнение, без да изисква от разработчиците усвояване на нови езици или инструменти. Използваните технологии са внимателно подбрани, така че да отговарят на изискванията за сигурност, мащабируемост, модулност и възможност за локално тестване.

Глава 3. Проектиране на системата

3.1 Изисквания към програмната система

Разработваната програмна система – библиотека за декларативно изграждане на облачни ресурси – има за цел да обслужва нуждите на два основни типа потребители: екипите по разработка и инфраструктурните инженери. За да бъде ефективна, системата трябва да отговаря на набор от функционални и нефункционални изисквания, които гарантират нейната практичност, сигурност и разширяемост в реална производствена среда.

Функционални изисквания:

- Декларативна дефиниция на инфраструктурата: системата трябва да приема конфигурационни файлове, описващи желаното състояние на инфраструктурата.
- Анализ на текущото състояние: системата трябва да извършва сравнение между желаното и реалното състояние в облака.
- Планиране и прилагане на промени: трябва да се извършва валидиране, визуализиране на планираните промени и контролирано прилагане върху реалната инфраструктура.
- Разделение на отговорностите: разработчиците дефинират нуждите си чрез конфигурации, докато инфраструктурният екип дефинира правилата и логиката на реализация.
- Поддръжка на множество типове ресурси: системата трябва да позволява разширяване с нови типове облачни ресурси чрез модулна архитектура.
- Автоматизирано стартиране чрез Gradle: трябва да се осигурява възможност за изпълнение чрез интерфейс с команден ред и CI/CD с параметри и променливи на средата.

Нефункционални изисквания:

- Сигурност: достъпът до облачни доставчици трябва да бъде защитен чрез външна система като Vault.
- Проследимост и логване: действията в системата трябва да бъдат записвани чрез централизирана логика за нуждите на диагностика и одит.
- Масштабируемост: архитектурата трябва да бъде проектирана така, че да позволява паралелно изпълнение на стъпки, без загуба на консистентност.

- Разширяемост: системата трябва да позволява добавяне на нови доставчици, ресурси и правила, без необходимост от промени в основното кодово пространство.
- Удобство за потребителя: конфигурационният формат и структурата на проектите трябва да бъдат интуитивни и добре документирани, така че разработчиците да могат да работят без дълбоко познаване на облачната платформа.
- Съвместимост: библиотеката трябва да бъде съвместима с Java и Gradle, както и да може да се използва както локално, така и в реални среди от по-високо ниво.

3.2 Архитектура на системата

Архитектурата на разработваната библиотека отразява ясно разделение на отговорностите между отделните компоненти, като същевременно поддържа гъвкавост и разширяемост. Системата е изградена по модулен принцип, което позволява лесно добавяне на нови функционалности, както и поддръжка на различни източници на автентикация и типове ресурси.

От архитектурна гледна точка, взаимодействието между програмните модули може да бъде разделено на няколко ключови фази:

1. Интеграция и стартиране

Библиотеката може да бъде използвана по два основни начина:

- чрез добавяне като Gradle зависимост в проектите на екипа по разработка
- чрез директно стартиране в CI/CD процес от екипа по инфраструктура при създаване на глобални ресурси

2. Автентикация

Управлява се чрез централен мениджър по автентикация, който координира различни начини за автентикация (напр. Vault, AWS профил, статични ключове). Мениджърът пробва последователно наличните механизми до успешна автентикация или провал на процеса.

3. Зареждане на инфраструктурното състояние

След автентикацията се зарежда текущото запазено състояние на инфраструктурата от системен файл. Ако такъв липсва, се създава нов,

който да служи като отправна точка. Това състояние представлява последната известна „истина“ за вече създадени ресурси.

4. Процес по изравняване (Reconciliation)

Системата извършва сравнение между съхраненото състояние и реалното (живо) състояние на инфраструктурата в облака. Ако бъдат открити несъответствия и е активирана опцията за автоматично изравняване, се прилагат коригиращи действия, така че реалната инфраструктура да съвпадне с описаното състояние.

5. Съпоставка между желано и текущо състояние

След като инфраструктурата е приведена в съответствие със запазеното състояние, се извършва нова проверка – този път между текущото състояние и ново зададеното (декларативно описано). Всички разлики се извеждат на потребителя и подлежат на прилагане.

6. Прилагане на промени

Прилагането на промените се извършва поетапно и при спазване на зададените от инфраструктурния екип ограничения. Всеки ресурс се създава, обновява или изтрива в зависимост от състоянието и конфигурацията.

7. Процес по връщане назад при неуспех (Rollback)

Ако при прилагането на промените възникне грешка, се активира механизъм за автоматично възстановяване. Целта му е да върне системата в изходното ѝ състояние от началото на изпълнението, базирайки се на предварително заредения файл на състоянията. Това гарантира консистентност и минимизира риска от частично прилагане на промени.

8. Изход и отчет

В края на процеса библиотеката връща обобщен резултат относно успеха или провала на операциите, който може да бъде използван както от потребителя, така и от автоматизирани системи в CI/CD процеса.



3.2.1 Пакетна структура на системата

Разработената библиотека е организирана в ясно разграничени пакети, съобразени с принципите на модулност, отговорности и повторна употреба. Всеки пакет съдържа конкретна функционалност, като взаимовръзките между тях следват логиката на архитектурния поток.

core

Ядрото на системата – съдържа основни абстракции и базови компоненти, които управляват цялостния процес по създаване на ресурси. Пакетът служи като централен координатор между конфигурации, изпълнение, валидации и състояния.

config

Пакетът config има ключова роля в инициализацията на системата, като дефинира внедряването на зависимости (Dependency Injection) чрез Google Guice. Той служи като централно място за конфигуриране на общо споделени обекти, които се използват в различни модули на библиотеката.

provisioner

Главният модул за координиране на създаването на ресурси. Той изгражда пълния жизнен цикъл от зареждане на конфигурации, съпоставка на състояние, до извършване на реални промени в облака.

executor

Съдържа реалните изпълними стъпки и механизми, чрез които се извършва промяна на инфраструктурата – създаване, обновяване, изтриване.

model

Пакетът съдържа модели, които описват резултатите от изпълнението на стъпките по прилагане на промени по инфраструктурата. Основната цел на този пакет е да предостави стандартизиран начин за предаване на резултати от стъпките – независимо дали са успешни или неуспешни – към централната логика на библиотеката и към потребителите ѝ.

exception

Съдържа специално създадени изключения за грешки по време на създаването на ресурси – напр. липсващи ресурси, невалидни конфигурации, неуспешна автентикация и др. Това позволява централизирана обработка и проследимост на проблемите.

utils

Помощен пакет с помощни класове и методи, използвани в цялата система – напр. логика за сериализация, файлови операции, проверка на променливи на средата и др.

aws

Модулът, отговорен за взаимодействие с конкретния доставчик AWS. Разделен е на подпакети:

- **eks** - съдържа логика за управление на EKS клъстър и техните възлови работни групи (worker node groups)
- **s3** - управлява създаването, настройката и политиките за сигурност на S3 ресурси

common

Съдържа споделени компоненти между различни услуги – напр. интерфейси, базови модели, общи трансформации или константи. Използва се споделено от различни услуги.

provider

Управлява различните типове доставчици на автентикация и достъп до облака. Това включва както реални доставчици (напр. AWS), така и симулирани (при тестване с LocalStack).

vault

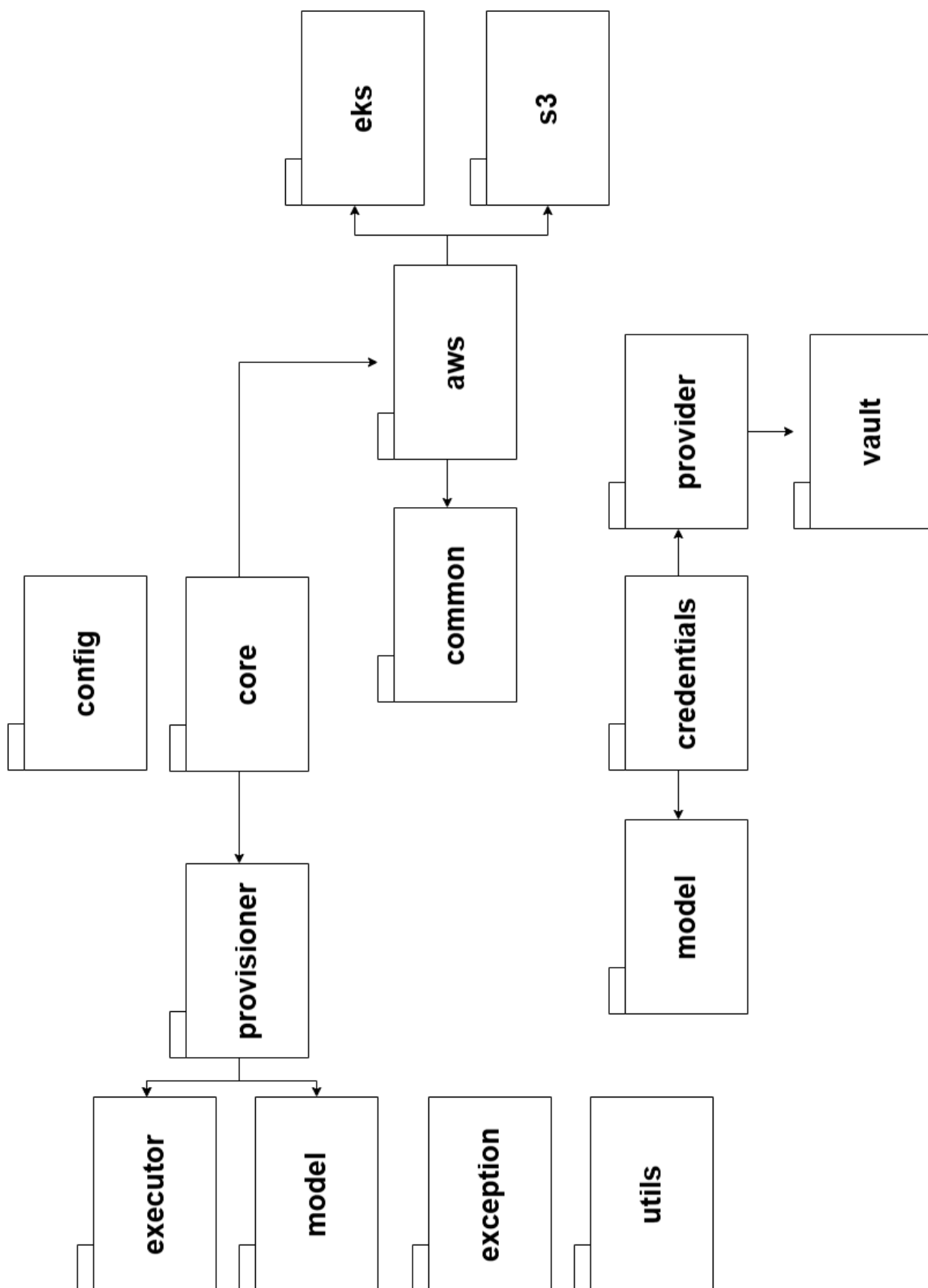
Подпакет реализиращ конкретна интеграция с HashiCorp Vault. Използва се за сигурна автентикация и получаване на защитени стойности за удостоверяване и доказване на самоличност.

credentials

Осигурява модели и логика за представяне и обработка на автентикационни данни.

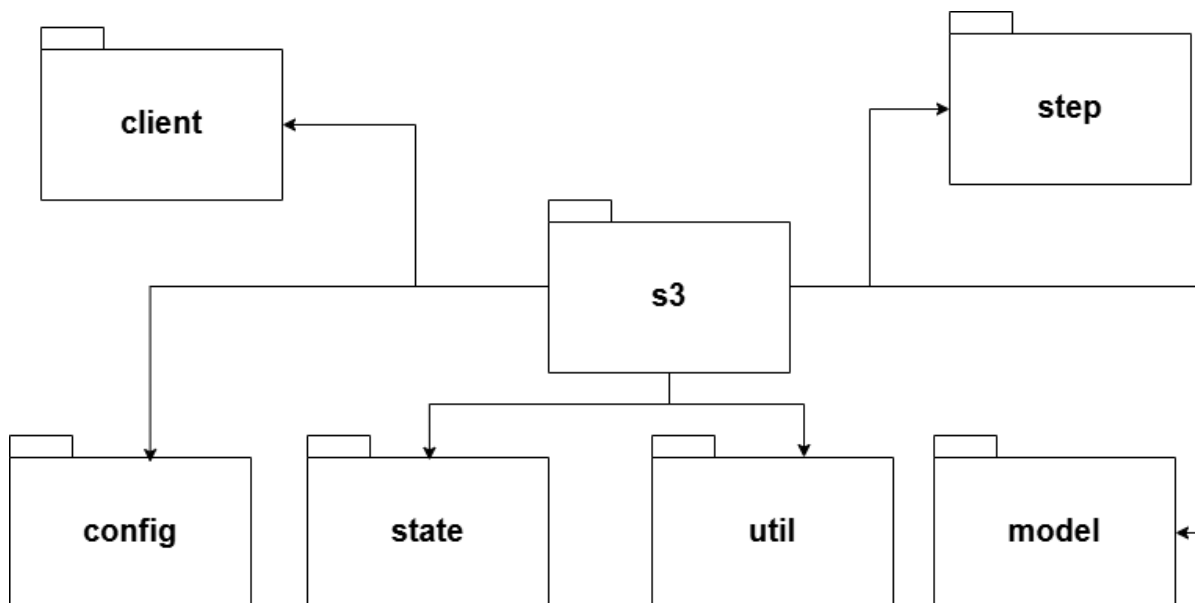
Тази модулна структура позволява:

- лесно добавяне на нови доставчици на облачни услуги (напр. Azure, GCP)
- поддръжка на различни видове ресурси без промяна на централната логика
- сигурно и ясно разграничение между конфигурация, изпълнение и инфраструктурен достъп
- добра тестируемост и преизползваемост на код



Фиг. 2 Архитектурна зависимост между пакетите на библиотеката

3.2.2 Пакетна структура на ресурсен модул



Фиг. 3 Вътрешна модулна архитектура на S3 компонента

Ресурсните модули са организирани в няколко ясно разграничени под-пакета, които инкапсулират различни аспекти от управлението и създаването на определени ресурси. Архитектурната структура следва принципи на разделение на отговорностите, което улеснява тестването, поддръжката и повторната употреба.

client

Съдържа клиентски класове, които служат като фасада между библиотеката и приложния програмен интерфейс на облачния доставчик. Основната му роля е да абстрахира директната комуникация с външния доставчик и да централизира всички извиквания към неговия API. Това осигурява по-добра капсулация и улеснява обработката на изключения или прилагане на механизми за повторно изпълнение.

config

Отговаря за десериализацията и структурирането на конфигурационните файлове, които описват желаното състояние на ресурсите. Съдържа класове, използвани при обработка на потребителските входни данни, които след това се предават на останалите модули за изпълнение.

model

Съдържа POJO класове, които представят структурите в конфигурацията. Те се използват в комбинация с конфигурационния пакет, за да се осигури чисто

представяне на входните данни и да се позволи тяхната валидирана трансформация във вътрешни обекти за създаване.

state

Този пакет инкапсулира логиката, свързана с управление на състоянието на инфраструктурата:

- Желано (desired) - състоянието, което потребителят дефинира
- Реално, живо (live) - реалното състояние в облака
- Съхранено (stored) - предишно запазеното състояние
- Сравнител (comparator) - логика за откриване на разлики и определяне на нужни промени

Това е ядрото на процесите по изравняване и последващо прилагане на промени в библиотеката.

util

Съдържа помощни класове и методи, специфични за работата на ресурсния модул – напр. трансформации, валидации, филтриране или извличане на метаданни. Пакетът допринася за поддържане на чистота на логиката в останалите слоеве, като премахва дублиране на код.

step

Съдържа конкретни стъпки за прилагане на промени, които представляват атомарни действия като създаване на ресурс, задаване на политика за сигурност, добавяне на маркери и др. Всяка стъпка изпълнява точно определена задача и може да бъде използвана самостоятелно или като част от цялостен процес.

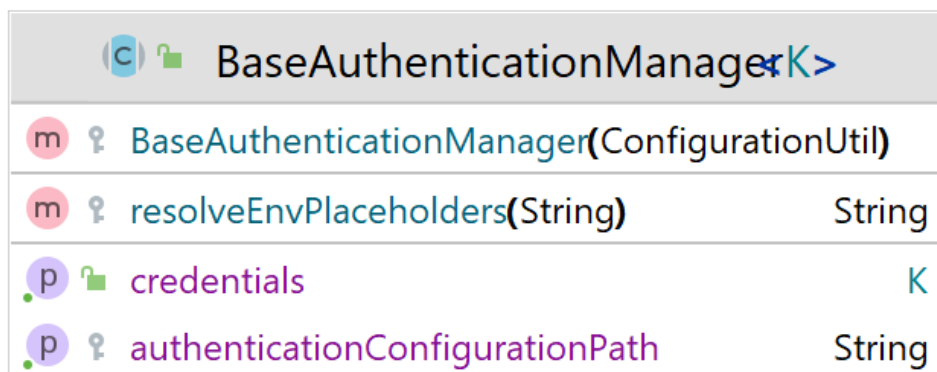
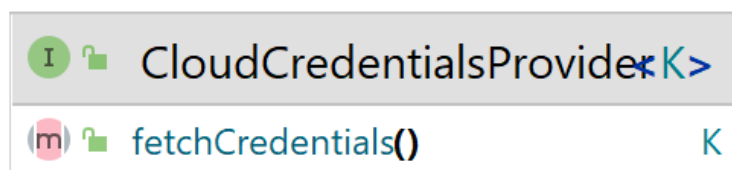
На най-горно ниво в ресурсния модул се намират компоненти, които свързват подмодулите и организират работата им:

- Cloud Bundle Runner - основният разпределителен компонент, който обединява всички части и управлява целия жизнен цикъл по създаването
- Provisioner, Destroyer, Revert изпълнители - координиращи класове, които стартират съответните фази от процеса (създаване, изтриване, връщане назад)
- Context - обект, съдържащ всички свойства за построяване на клиент за комуникация с приложния интерфейс на облачния доставчик
- Injection - Guice модул, който дефинира как се създават и инжектират компонентите в рамките на ресурса

Тази структура позволява:

- независима разработка на нови ресурси по същия модел
- лесно разширяване с нови стъпки или валидиращи правила
- изолиране на комуникацията с облачния доставчик
- поддържане на чиста архитектура чрез добре разграничени слоеве:
конфигурация → модел → състояние → изпълнение

3.2.3 Архитектура на автентикационния слой



Фиг. 4 Архитектура на абстракцията за автентикация и извличане на данни за вход

Интерфейсът `CloudCredentialsProvider` дефинира унифициран договор за извличане на автентикационни данни, използвани при комуникация с облачни доставчици. Той съдържа един метод – `fetchCredentials`, който връща обект от параметризирания тип `K` и може да генерира изключение `IOException` при неуспех. Чрез този подход всяка конкретна имплементация (напр. `Vault`, `AWS` профил, статични данни за вход) може да осигури различен механизъм за извличане на информация за потвърждение на самоличността, без да променя логиката на останалата част от системата. Този интерфейс позволява гъвкаво добавяне на нови източници за автентикация, без да се нарушава отворено-затвореният принцип.

Абстрактният клас `BaseAuthenticationManager` играе ролята на базова реализация и координатор, използван от всички конкретни мениджъри за автентикация. Той съдържа логика за зареждане на конфигурации, както и метод `getCredentials`, който трябва да бъде имплементиран от наследниците, за да доставят конкретните данни за достъп. Освен това, класът дефинира и `getAuthenticationConfigurationPath`, абстрактен метод, чрез който всяка реализация може да определи от къде да зареди конфигурационния файл със своите настройки.

Особено полезна функционалност в този базов клас е методът `resolveEnvPlaceholders`. Той обработва низове, съдържащи шаблони от вида `${ENV_VAR}`, като ги заменя с реалните стойности от системните променливи на средата. Това позволява динамична конфигурация, без нужда от статично задаване на чувствителна информация във файлове или код. Ако дадена променлива не е дефинирана, се регистрира предупреждение и тя се заменя с празен низ – така се избягва прекъсване на процеса при непълна конфигурация.

Комбинацията от интерфейс `CloudCredentialsProvider` и абстрактен базов клас `BaseAuthenticationManager` осигурява стабилна и разширяема архитектура за автентикация, която може лесно да бъде адаптирана към различни изисквания на инфраструктурния екип или на конкретни среди (напр. разработка, продукция).

3.2.4 Базова конфигурационна архитектура на ресурси

За да бъде реализирана гъвкава и разширяема система за декларативно дефиниране на облачни ресурси, е създадена йерархия от базови конфигурационни класове. Те предоставят обща структура, интерфейси и поведение, които са валидни за всички типове ресурси. Основната цел на тази архитектура е да гарантира, че всеки ресурс съдържа минимално изискваните атрибути и може да бъде управляван през унифицирани механизми за сравнение, сериализация, валидиране и изпълнение.

В основата стои абстрактният клас `BaseCloudResourceConfiguration`, който имплементира два интерфейса – `IdentifiableConfig` и `DestroyProtection`. Чрез тях се гарантира, че всеки ресурс има уникален идентификатор и поддържа механизъм за защита от неволно изтриване чрез метода `preventDestroy`.

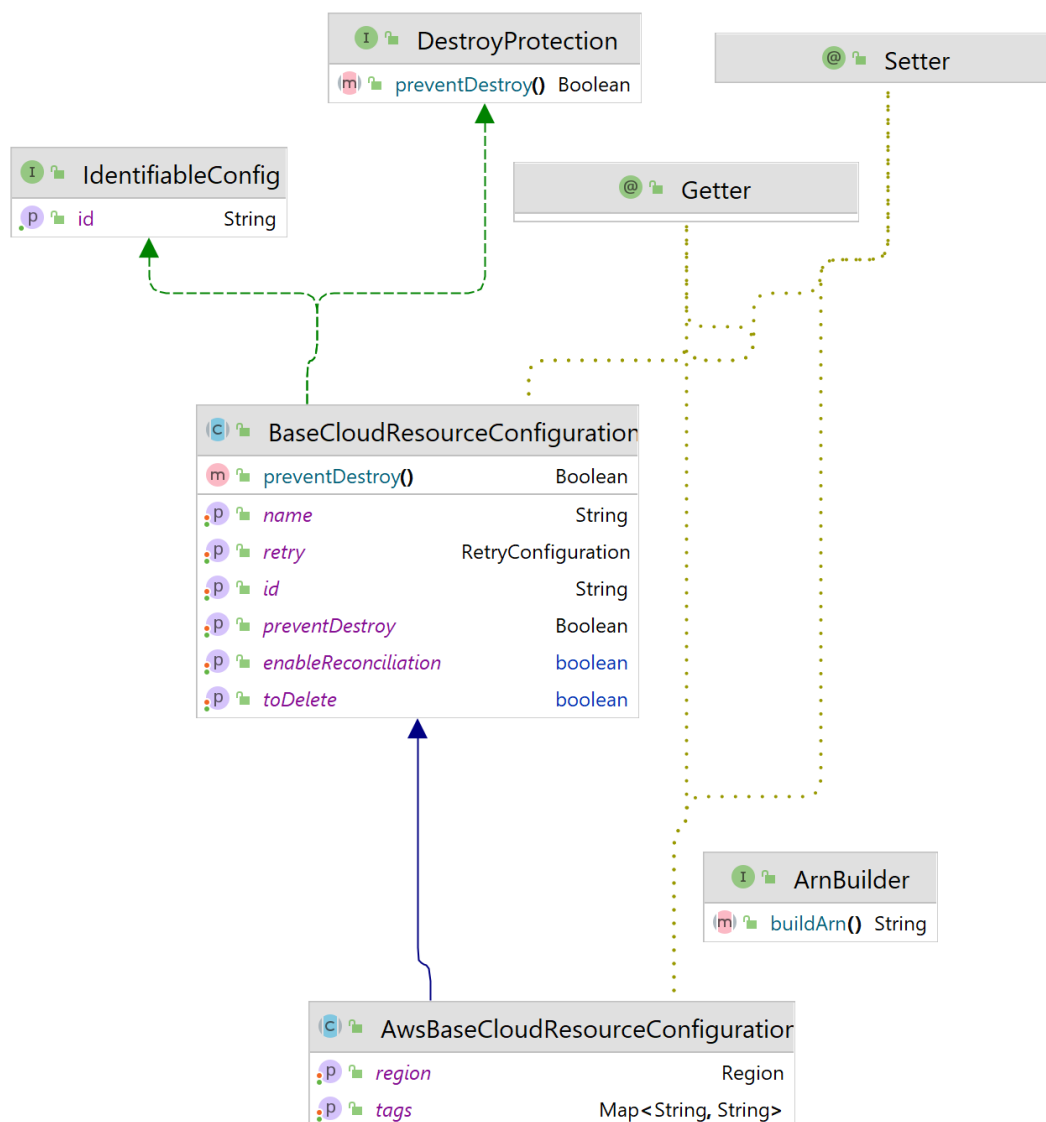
Сред основните атрибути на класа са:

Таблица 1. Основни свойства на ресурсите в конфигурационния модел

Наименование	Значение
id	Стойност, чрез която ресурсът се идентифицира в конфигурацията и файловете със състояние
name	Потребителски четимо име на ресурса
enableReconciliation	Флаг, указващ дали ресурсът да бъде подложен на автоматично сравнение и синхронизация с текущото му състояние
preventDestroy	Булева стойност, която при истина предотвратява изтриване на ресурса
toDelete	Флаг, който изрично маркира ресурса за изтриване
retry	Обект от тип RetryConfiguration, който дефинира максимален брой повторни опити и времеви интервал между тях, използван при временни грешки при прилагане на промените

```
@Getter
public static class RetryConfiguration {
    int max = 3;
    long backoffMs = 1000;
}
```

Фрагмент 1. Пример за конфигурация на повторни опити



Фиг. 5 Структура на наследяване и интерфейси при конфигурация на ресурси от конкретен облачен доставчик

3.2.5 Конфигурационни списъци и паралелност при създаване на ресурси

В системата за декларативно управление на инфраструктура, всеки вид ресурс може да бъде дефиниран индивидуално, но в реална конфигурация обикновено се обработват множество инстанции от един и същи тип ресурс. За тази цел е създадена абстракция за представяне на списъци с ресурси – т.нар. листови конфигурации, базирани на интерфейса `CloudResourceConfigList`.

```
public interface CloudResourceConfigList<C extends
BaseCloudResourceConfiguration> {
    List<C> getItems();
    void setItems(List<C> items);
}
```

Фрагмент 2. Интерфейс за обобщено представяне на списък с конфигурации

Този интерфейс дефинира методи за достъп и модификация на списък от обекти от тип C, който трябва да бъде конкретен наследник на BaseCloudResourceConfiguration. Така се гарантира, че всички елементи в листа отговарят на изискванията за стандартен ресурс и поддържат ключовите операции като getId, preventDestroy, и др.

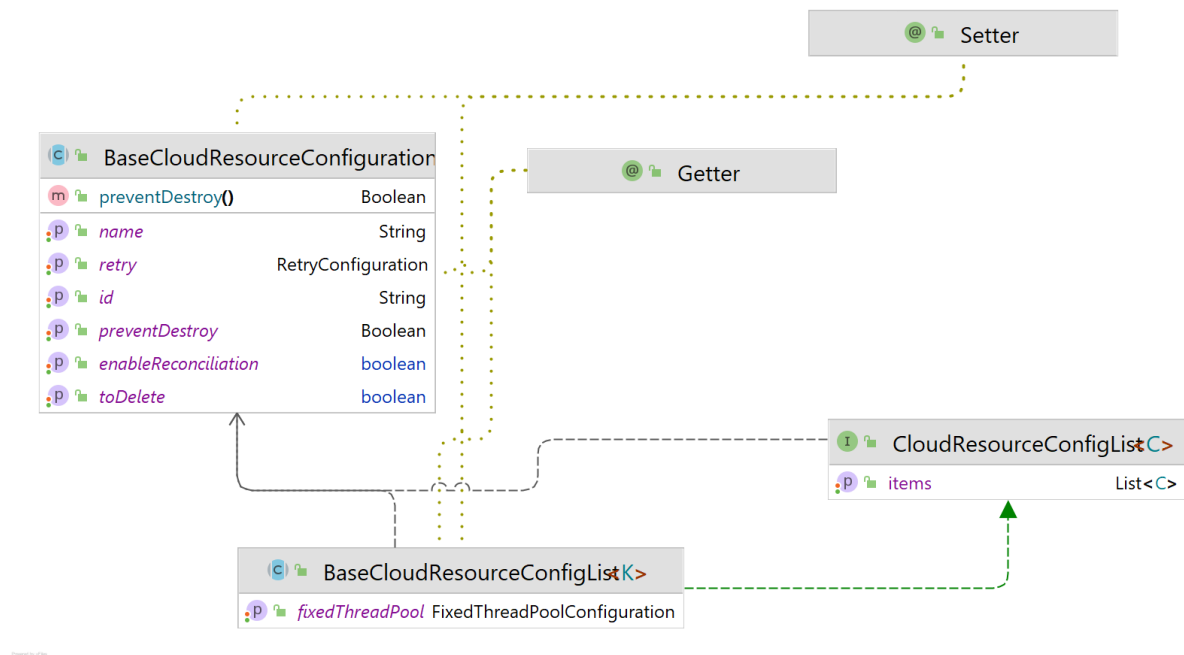
За да бъде улеснено управлението на такива списъци, е въведен абстрактният клас BaseCloudResourceConfigList, който имплементира интерфейса и добавя допълнителна функционалност, свързана с управлението на паралелността при обработка на елементите:

```
public abstract class BaseCloudResourceConfigList<K
extends BaseCloudResourceConfiguration> implements
CloudResourceConfigList<K> {
    private FixedThreadPoolConfiguration fixedThreadPool =
new FixedThreadPoolConfiguration();
    @Getter
    public static class FixedThreadPoolConfiguration {
        int maxSize = 5;
    }
}
```

Фрагмент 3. Абстрактен клас за списък с облачни конфигурации и настройка на банки от нишки

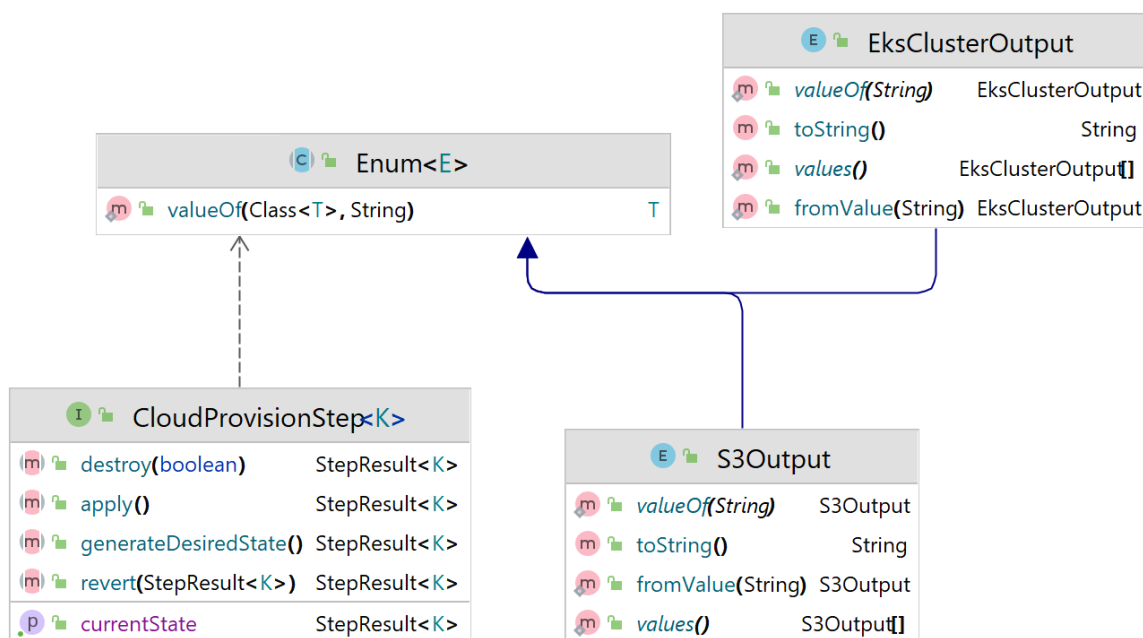
Свойството fixedThreadPool съдържа конфигурация за максималния брой нишки, които могат да бъдат използвани при паралелно създаване, валидиране или сравнение на ресурсите от този списък. Това е от ключово значение, когато инфраструктурата съдържа десетки или стотици обекти, които трябва да бъдат обработени по ефективен начин, без да се блокират останалите операции.

Тази архитектура позволява ясна модулност, управление на паралелността и централизирана обработка на множество ресурси, като същевременно поддържа лесно разширяване за нови типове конфигурации.



Фиг. 6 Класова диаграма за списъчна конфигурация на облачни ресурси

3.2.6 Интерфейсна архитектура на стъпките за контролиране на ресурс



Фиг. 7 UML диаграма на стъпки за прилагане и тяхната типизирана изходна информация

CloudProvisionStep е унифициран интерфейс, който дефинира поведението на всяка отделна стъпка от модифицирането на облачен ресурс. Чрез обобщаващия параметър K той позволява различните реализации да връщат конкретен тип резултат от тип изброима стойност, който описва какво е било създадено или конфигурирано.

Методът apply е основният механизъм за създаване или прилагане на ресурс. Той използва съответния фасаден клиент на услугата, за да извърши реални действия в облака. След изпълнение връща резултат StepResult, съдържащ стойности, които се записват в състоянието на системата и представляват реалното положение след изпълнение.

Методът generateDesiredState отговаря за генериране на желаното състояние въз основа на предоставената конфигурация, обикновено идваща от YAML файл. Този метод не модифицира ресурсите, а изчислява стойностите, които впоследствие ще се сравнят с реалното състояние при етапите на изравняване и прилагане на нови промени. Той е от ключово значение за идемпотентността и надеждната автоматизация.

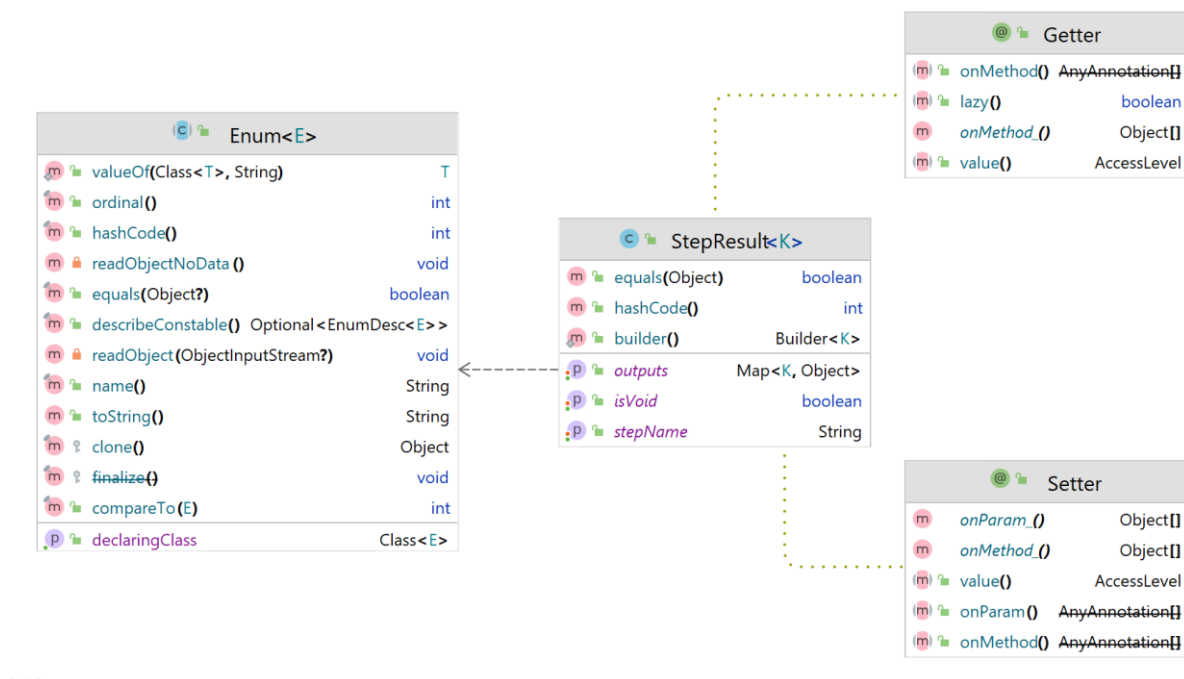
Методът getCurrentState се използва за извличане на текущото реално състояние на ресурса директно от облачния доставчик. Извиквайки съответния програмен интерфейс, той позволява съпоставка между конфигурираното и фактическото състояние на ресурса. Това е от решаващо значение за прецизното управление на промени и засичане на разлики.

Методът revert осигурява механизъм за възстановяване на ресурса до предходно състояние, например при възникване на грешка по време на прилагане или когато се открие несъответствие с изискванията. Използвайки записаното предишно състояние, системата може да извърши връщане назад и да върне ресурсите в познато стабилно състояние.

Накрая, методът destroy се използва за унищожаване на съществуващия ресурс. Преди изпълнение се проверява дали е разрешено премахването, чрез флаг preventDestroy, и ако е разрешено, се предприема реално изтриване чрез клиента.

Тази група от методи оформя ядрото на всяка стъпка и гарантира повторимост, възстановимост и контрол върху ресурсите в облачна среда.

Чрез използване на генерализиран параметър K, интерфейсът е едновременно гъвкав и типово сигурен. Всеки нов ресурс дефинира собствен изброим тип със свои изходни стойности, без да нарушава основния интерфейс. Всички резултати могат лесно да се сериализират и да се използват за сравнение, валидация или повторно използване. Тъй като всяка стъпка знае какво е променила, тя може безопасно да върне тези промени назад до предишното си състояние в случай на възникнала грешка в процеса на модификация на ресурса.



Фиг 8. Класова диаграма на резултат от стъпка за прилагане

`StepResult` е обобщен контейнер за резултати от изпълнение на стъпка в процеса по прилагане на промени. Използва се във всички методи на интерфейса `CloudProvisionStep`, като начин за съобщаване на:

- какво е било извършено
- какво е било създадено или предвидено
- дали стъпката е завършила успешно или с пропусната операция
- какво трябва да се съхрани като част от състоянието

Основното съдържание на класа `StepResult` е полето `outputs`, което представлява асоциативна структура от тип `Map<K, Object>`. Ключовете в тази структура са от тип `K`, който винаги е конкретен изброим тип. Тези изброими типове дефинират допустимите изходни стойности за всяка стъпка и осигуряват типова безопасност. Стойностите в речника са обекти (най-често от тип `String`, `List` или `Map`), които подлежат на сериализация във файлов формат и впоследствие се използват за изграждане на текущото състояние от файла на състоянията. Например, след създаване на `S3`, резултатът може да се сериализира в следния формат:

```
{
  "stepName" :
  "bg.tuvarna.sit.cloud.core.aws.s3.step.S3BucketStep",
  "outputs" : {
    "name" : "tuvarna-eu-west-1-000000000000",
    "region" : "eu-west-1"
  }
}
```

Фрагмент 4. Примерен JSON изход от стъпка за създаване на S3

Флагът `isVoid` указва дали дадена стъпка е "празна", т.е. не е извършила реално действие. Това се използва в случаи, когато стъпката е била пропусната поради липса на разлики между желано и текущо състояние. По този начин `isVoid` позволява точно отчитане на случилото се по време на изпълнението, без да се въвеждат допълнителни специални типове резултати.

Полето `stepName` представлява символичното име на стъпката и служи за лог, проследяване и организиране на информацията във файловете със състояния. Благодарение на него, резултатите от различни стъпки могат лесно да бъдат идентифицирани, групирани и анализирани, особено при комплексни операции, включващи множество ресурси и междинни състояния.

Класът `StepResult` предоставя удобен верижан метод `builder`, който връща обект от тип `Builder`. Това позволява лесно и интуитивно създаване на инстанции на `StepResult`, като чрез верижно извикване на методи могат да се зададат името на стъпката и нейните изходни стойности. Например, създаването на резултат за S3 ресурс може да изглежда така:

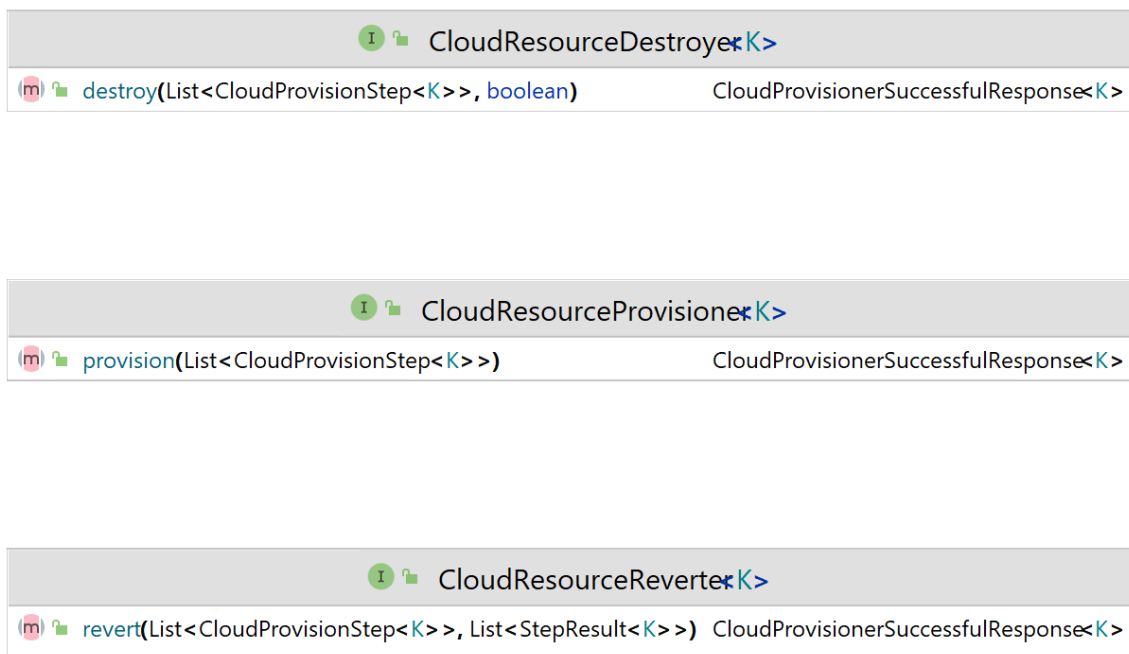
```
return StepResult.<S3Output>builder()
    .stepName(step)
    .put(S3Output.NAME, bucketName)
    .put(S3Output.REGION, config.getRegion().getValue())
    .build();
```

Фрагмент 5. Изграждане на резултат от стъпка за създаване на S3 ресурс

`StepResult` изпълнява ключова архитектурна роля в системата, като служи като връзка между интерфейсната логика на всяка стъпка и съхраняемото състояние на инфраструктурата. Всички данни, които трябва да бъдат запомнени след изпълнението на дадена стъпка – независимо дали е създаване, обновяване или изтриване на ресурс – преминават през `StepResult`. Благодарение на генерализирания параметър `K`, се осигурява типова сигурност, тъй като само предварително дефинирани и позволени ключове могат да участват в съхранявания резултат. Това значително улеснява работата с файловете на

състоянията и предотвратява грешки при сериализация. Освен това, `StepResult` облекчава реализацията на логиката за връщане назад, като предоставя запазените стойности от изпълнението, които могат директно да бъдат подадени към метода `revert` за възстановяване на инфраструктурата до предходно състояние. Финално, този обект унифицира комуникацията между отделните архитектурни слоеве – от изпълнителя на стъпката, през координиращия компонент, до компонента, който записва състоянието, което прави системата модулна, проследима и устойчива на промени.

3.2.7 Интерфейсна архитектура на операциите по управление на ресурсите



Фиг. 9 Основни изпълнители на операции по управление на облачни ресурси

Интерфейсите `CloudResourceProvisioner`, `CloudResourceDestroyer` и `CloudResourceReverter` представляват унифициран архитектурен шаблон, който всяка имплементация на ресурс следва, за да изпълнява стандартен набор от операции върху всички дефинирани стъпки. Всеки от тези интерфейси приема списък със стъпки за обработка и връща унифициран отговор под формата на `CloudProvisionerSuccessfulResponse`, който обобщава резултатите от изпълнението.

Интерфейсът `CloudResourceProvisioner` дефинира основната операция `provision`, която изпълнява последователно всички подадени стъпки по създаване или

промяна на ресурсите. Стъпките са подредени в логическа последователност и се извикват една след друга чрез техните `apply` методи. След завършване на процеса, всички резултати се агрегират и връщат като част от отговора.

`CloudResourceDestroyer` дефинира операцията `destroy`, която служи за изтриване на ресурси. Методът приема списък със стъпки, както и булев флаг, указващ дали изтриването да бъде категорично, дори и да е спряно от конфигурацията. Всяка стъпка извършва своето унищожение чрез метода `destroy`, като резултатите отново се агрегират и се връща пълен отчет за изпълнението.

Третият интерфейс, `CloudResourceReverter`, дефинира операцията `revert`, която изпълнява връщане назад на състоянието на ресурсите до предходно такова. Методът приема списък със стъпки, както и съответстващ списък с предишни `StepResult`, които описват състоянието, към което трябва да се върне системата. Всяка стъпка използва тези данни, за да извика своя метод `revert` и да върне ресурса в стабилно и предефинирано състояние.

Тази интерфейсна архитектура позволява всеки отделен ресурс да реализира собствената си логика за изпълнение, изтриване и възстановяване, като същевременно спазва унифициран подход към обработката на стъпки. Така се постига гъвкавост, повторна употреба на код и ясно разделение на отговорностите между абстракция и имплементация.

3.2.8 Изпълнители на стъпки и стратегии за изпълнение

  <code>StepExecutionStrategy<K></code>		
	 <code>execute(CloudProvisionStep<K>)</code>	<code>StepResult<K></code>
  <code>CloudStepRevertExecutor<K></code>		
	 <code>execute(List<CloudProvisionStep<K>>, List<StepResult<K>>)</code>	<code>List<StepResult<K>></code>
	 <code>executeWithRetry(CloudProvisionStep<K>, List<StepResult<K>>)</code>	<code>StepResult<K></code>
  <code>CloudStepStrategyExecutor<K></code>		
	 <code>execute(List<CloudProvisionStep<K>>, StepExecutionStrategy<K>) t<StepResult<K>></code>	
	 <code>executeWithRetry(CloudProvisionStep<K>, StepExecutionStrategy<K>)</code>	<code>StepResult<K></code>
  <code>CloudStepDeleteExecutor<K></code>		
	 <code>execute(List<CloudProvisionStep<K>>, boolean)</code>	<code>List<StepResult<K>></code>

Powered by JFrog

Фиг. 10 Изпълнители и стратегии за обработка на стъпки

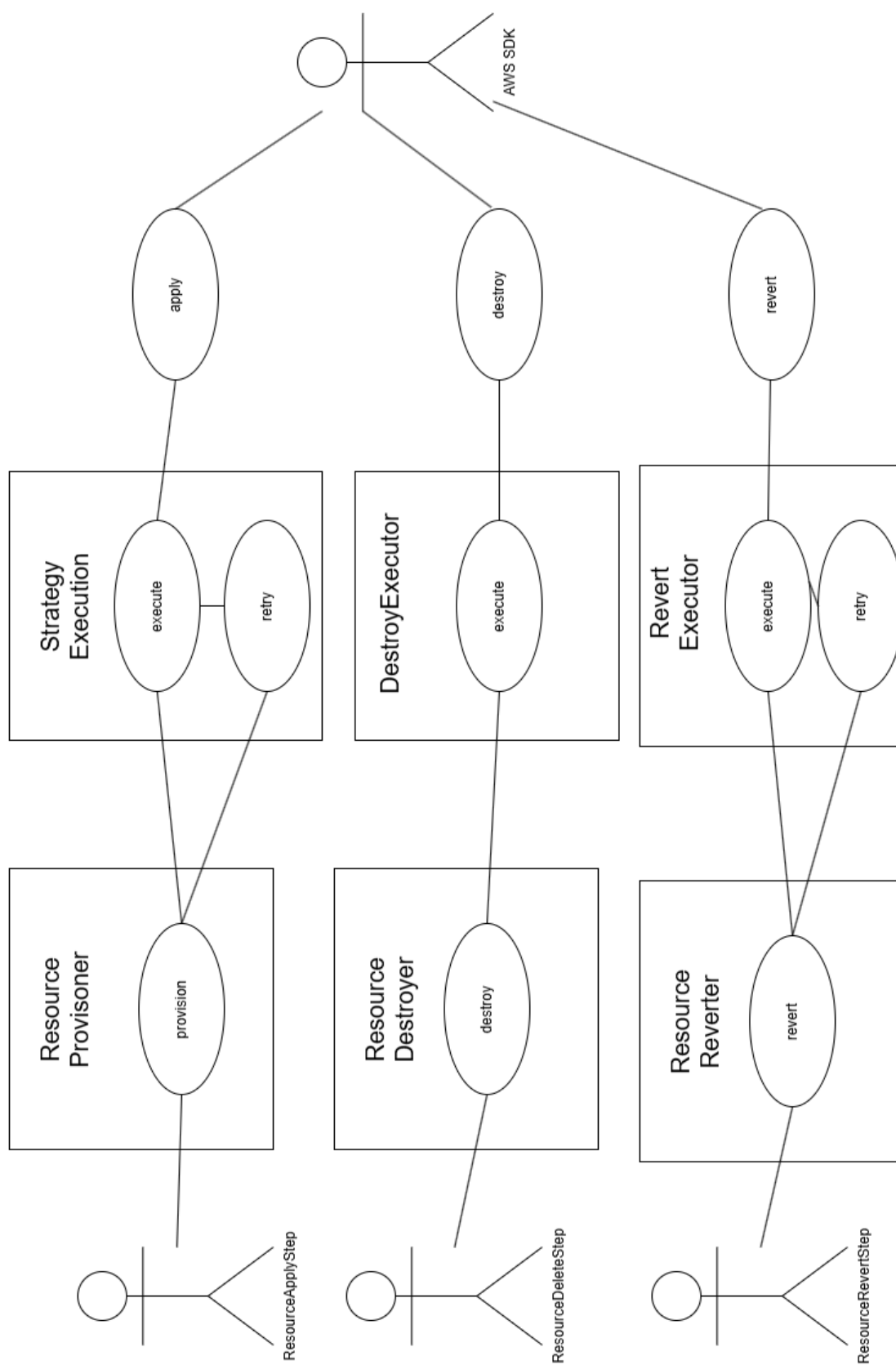
Изпълнителите в архитектурата на библиотеката представляват специализирани компоненти, които управляват изпълнението на стъпките, използвайки предварително дефинирани стратегии и поведение при грешки. Всеки от основните контролни компоненти – `CloudResourceProvisioner`, `CloudResourceDestroyer` и `CloudResourceReverter` – делегира реалното изпълнение на стъпките към съответен `Executor`. По този начин се постига ясен раздел между координиращата логика и механизма за изпълнение, което улеснява повторната употреба и разширяване.

`CloudStepStrategyExecutor` е най-гъвкавият от тези изпълнители. Той приема списък със стъпки и стратегия за изпълнение `StepExecutionStrategy`, която определя коя операция трябва да бъде извикана – например `apply`, `generateDesiredState` или `getCurrentState`. Изпълнението е разделено на синхронни и асинхронни стъпки, като се използва анотация, за да се определи как дадена стъпка трябва да бъде обработена. Синхронните стъпки се изпълняват последователно, докато асинхронните – паралелно, с помощта на набор от нишки. Това дава възможност за по-висока производителност, без да се нарушава детерминираното поведение на стъпките, които изискват подредено изпълнение.

Всеки `Executor` използва `retry` механизъм, дефиниран чрез `RetryConfiguration`, който позволява автоматично повторно изпълнение на стъпки при възникване на `RetryableCloudResourceStepException` изключение. Методът `executeWithRetry` в `CloudStepStrategyExecutor` капсулира тази логика, като брои опитите, спазва зададен интервал между тях, и обработва различните типове изключения. Това повишава надеждността при прилагането на промени, особено при временни грешки от облачния доставчик или мрежови неточности.

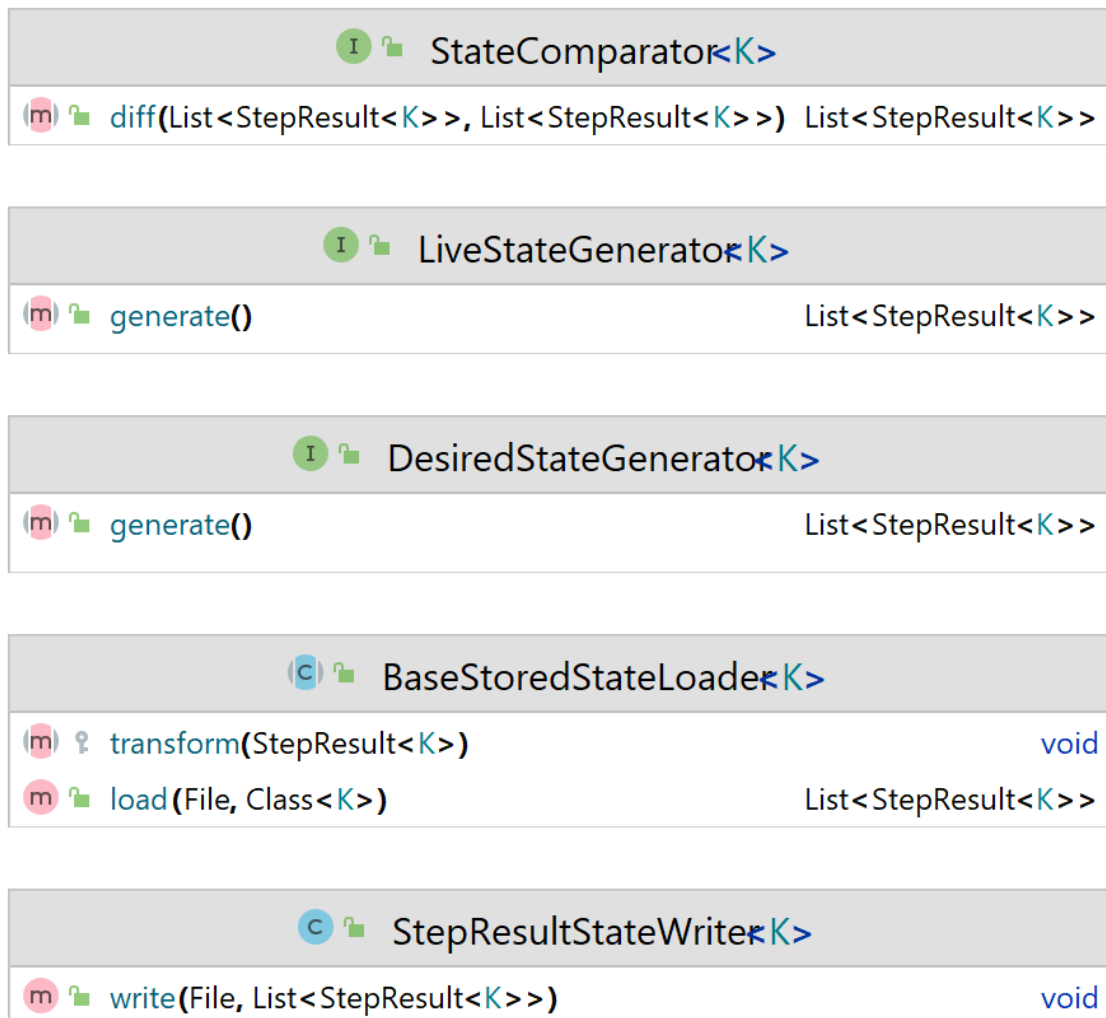
Други изпълнители като `CloudStepDeleteExecutor` и `CloudStepRevertExecutor` са специализирани за изтриване и възстановяване на ресурси. `DeleteExecutor` приема булев флаг, указващ дали изтриването да бъде категорично, докато `RevertExecutor` използва списък със `StepResult` обекти, за да възстанови предходно състояние на инфраструктурата чрез извикване на `revert` метода на всяка стъпка. И в двата случая логиката по паралелизация и повторно изпълнение може да бъде приложена аналогично.

Чрез такава архитектура с `Executor`-и, системата постига гъвкавост, разширяемост и висока степен на контрол върху изпълнението на инфраструктурни операции. Това позволява различни поведения – като паралелно пускане, или връщане назад – да се реализират без да се променя основната логика на самите стъпки.



Фиг. 11 Общ преглед на изпълнението на операции по създаване, изтриване и връщане назад на ресурси

3.2.9 Управление на състоянието на инфраструктурата



Powered by yUe

Фиг 12. Компоненти за управление и сравнение на състояния на ресурси

Системата разчита на специализирани базови компоненти, които управляват зареждането, генерирането, съпоставянето и съхраняването на състоянията на инфраструктурните ресурси. Тези състояния са представени чрез обекти от тип `StepResult` и се записват във файлове, за да бъдат проследими, сериализирани и лесно валидируеми. Всеки запис в такъв файл съдържа резултати от изпълнени стъпки – какво е било създадено, обновено или върнато назад по време на изпълнението на процеса. Компонентът `BaseStoredStateLoader` отговаря за зареждането на вече записано състояние от файл. Методът `load` прочита файла и го преобразува в списък от `StepResultK`. Това състояние служи като отправна точка за следващи операции като сравнение, възстановяване или повторно изпълнение.

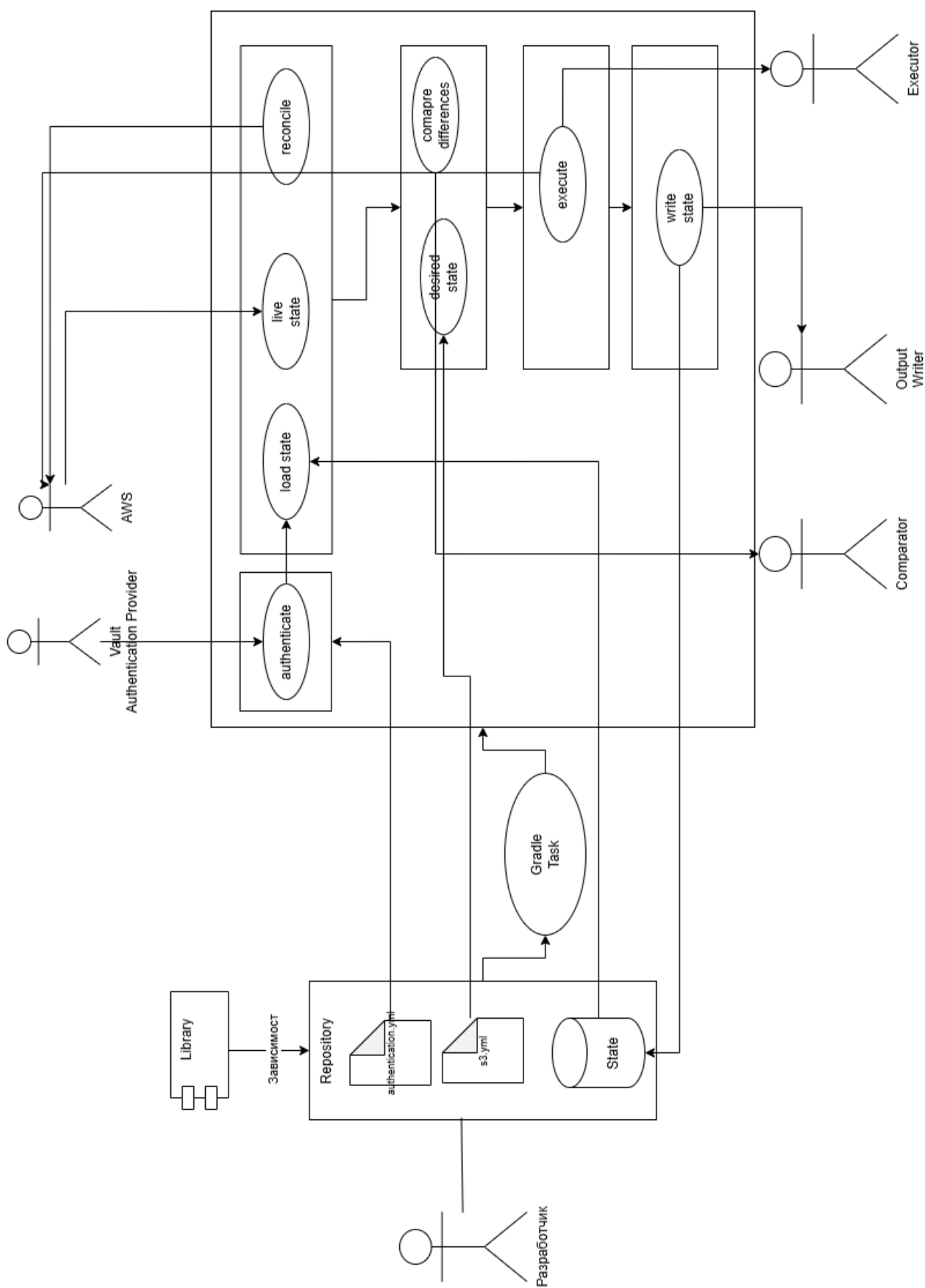
Методът `transform` позволява модификации върху отделни резултати, например за преформатиране или валидиране на стойности преди употреба.

Генерирането на текущо и желано състояние се управлява от интерфейсите `LiveStateGenerator` и `DesiredStateGenerator`. Първият използва клиента към облака, за да извлече реалното състояние на вече съществуващи ресурси. Вторият изгражда желаното състояние, базирано на входни декларативни конфигурации. И двете операции връщат списък от `StepResult`, който след това се използва за сравнение и валидиране на нуждата от промени.

Компонентът `StateComparator` изпълнява логиката за съпоставка между състоянията. Методът `diff` сравнява желаното и реалното състояние, като извежда списък от стъпки, при които има несъответствия. Това е в основата на функционалността за автоматично откриване и прилагане на нужните промени.

След като ново състояние е генерирано или изчислено, то се записва обратно във файл чрез класа `StepResultStateWriter`. Методът `write` сериализира данните във формат JSON, като по този начин се гарантира пълна проследимост на извършените действия. Това състояние може да бъде използвано както за следващо сравнение, така и за възстановяване при неуспех.

Целият този механизъм гарантира идемпотентност, възстановимост и контролируемо изпълнение на инфраструктурни промени – като всяко действие може да бъде повторено, отменено или анализирано постфактум чрез файловото състояние.



Фиг. 13 Диаграма на процеса по декларативно прилагане на облачни ресурси

3.2.10 Обединяване на ресурсната логика в единен компонент

В основата на архитектурния модел за реализация на инфраструктурни ресурси стои абстрактният клас `CloudBundleRunner`. Този компонент обединява в едно място всички основни участници, свързани с обработката на даден ресурсен тип – конфигурация, състояние, сравнение, логика за прилагане на промени, изтриване и възстановяване. За всеки отделен ресурс се създава конкретна имплементация на `CloudBundleRunner`, която дефинира поведението за този тип.

Класът е параметризиран с тип `K`, който представя валидните ключове за изходи от стъпките във файла на състоянията. Сред основните вътрешни компоненти са:

- `BaseStoredStateLoader` – за зареждане на предходно съхранено състояние
- `StateComparator` – за откриване на отклонения между конфигурации
- `StepResultStateWriter` – за сериализация и запис на състояние
- `Slf4jLoggingUtil` – за лог в различни формати
- `ConfigurationUtil` – за зареждане на YAML конфигурации

Всяка реализация трябва да дефинира методите:

```
protected abstract CloudResourceType getType();  
protected abstract String getServiceConfigurationPath();  
protected abstract void run(AwsBasicCredentials  
credentials);
```

Фрагмент 5. Абстрактни методи за описание и изпълнение на облачен ресурс

Методът `provisionGeneric` в `CloudBundleRunner` е централен за изпълнението на всички действия, свързани с конкретна конфигурация. За целта той приема обект от тип `ProvisionRequest`, който съдържа цялата необходима информация и функционални зависимости за прилагане на промени по единичен ресурс:

```

@Builder
public static class ProvisionRequest<K extends Enum<K>, C extends
BaseCloudResourceConfiguration> {
    private final C config;
    private final String profile;
    private final File stateDir;
    private final String resourceKey;
    private final String metadataStepName;
    private final Function<Injector, List<CloudProvisionStep<K>>>
stepProvider;
    private final Function<Injector, CloudResourceDestroyer<K>>
destroyerProvider;
    private final Function<Injector, CloudResourceProvisioner<K>>
provisionerProvider;
    private final Function<Injector, CloudResourceReverter<K>>
reverterProvider;
    private final Function<Injector, List<StepResult<K>>> liveStateGenerator;
    private final Function<Injector, List<StepResult<K>>>
desiredStateGenerator;
    private final BiFunction<String, StepResult<K>, Injector>
injectorFactory;
    private final Function<C, StepResult<K>> metadataSupplier;
    private final Function<StepResult<K>, String> nameExtractor;
    private final Function<StepResult<K>, String> regionExtractor;
    private final Class<K> outputClass;
}

```

Фрагмент 6. Структура на заявка за прилагане на промени върху облачни ресурси

Класът `ProvisionRequest` действа като инжекционен контейнер, който капсулира всички зависимости, необходими за извършване на промени по даден ресурс. Той предоставя методи-фабрики и функционални интерфейси за създаване на стъпки, достъп до текущото състояние в облака, изпълнение на операции по създаване или изтриване на ресурсите, както и механизъм за възстановяване при грешки. Освен това съдържа правила за екстракция на ключови параметри като име и регион на ресурса, извлечени от стъпката за метаданни.

Целият процес на създаване в метода `provisionGeneric` преминава през няколко добре дефинирани фази. На първо място, системата опитва да зареди вече съществуващо състояние на ресурса от файловата система. Ако съответният файл не съществува, се приема, че ресурсът ще бъде създаден от нулата и се инициализира празно състояние. Тази логика позволява едновременно поддръжка на нови ресурси и обновяване на съществуващи.

Следва фазата по извличане на метаданни, в която се създава специална стъпка с метаданни, целяща да определи основните атрибути на ресурса – най-често неговото име и регион. Това е необходимо, за да бъде създаден инжекционен контейнер, който съдържа ресурсно-специфични зависимости, нужни за изграждане на останалите стъпки.

След като инжекторът е инициализиран, се пристъпва към зареждане на всички стъпки за ресурса, като се използва подаденият `stepProvider`. Всяка стъпка отговаря

за конкретна част от прилагането на промени. Ако липсва състояние от предишно изпълнение, текущото състояние се инициализира с „текущото“ (или празно) състояние на всяка стъпка.

След това се извършва сравнение между съхраненото и текущото живо състояние, с цел да се открият промени, направени извън системата – т.нар. конфигурационно отклонение. За тези промени се изготвя списък със стъпки, които трябва да бъдат изпълнени отново. Ако в конфигурацията е указано, че изравняването е разрешено, се преминава към прилагане на тези промени чрез `applyChanges`. При неуспешно изпълнение на която и да е стъпка се задейства връщащ механизъм, който възстановява инфраструктурата в предходното ѝ състояние с помощта на `reverter`.

Веднага след това, за да се гарантира, че живото състояние отговаря на новото желано състояние, се извършва вторично сравнение между желаната конфигурация и актуализираното живо състояние. Ако бъдат открити несъответствия, те отново се прилагат с помощта на същия механизъм за стъпково изпълнение.

В заключение, финалният резултат от всички успешно изпълнени стъпки се инкапсулира в `CloudProvisionerSuccessfulResponse`, който се записва като JSON файл на състоянието. Това гарантира проследимост, повторемост и съвместимост при следващи изпълнения на системата.

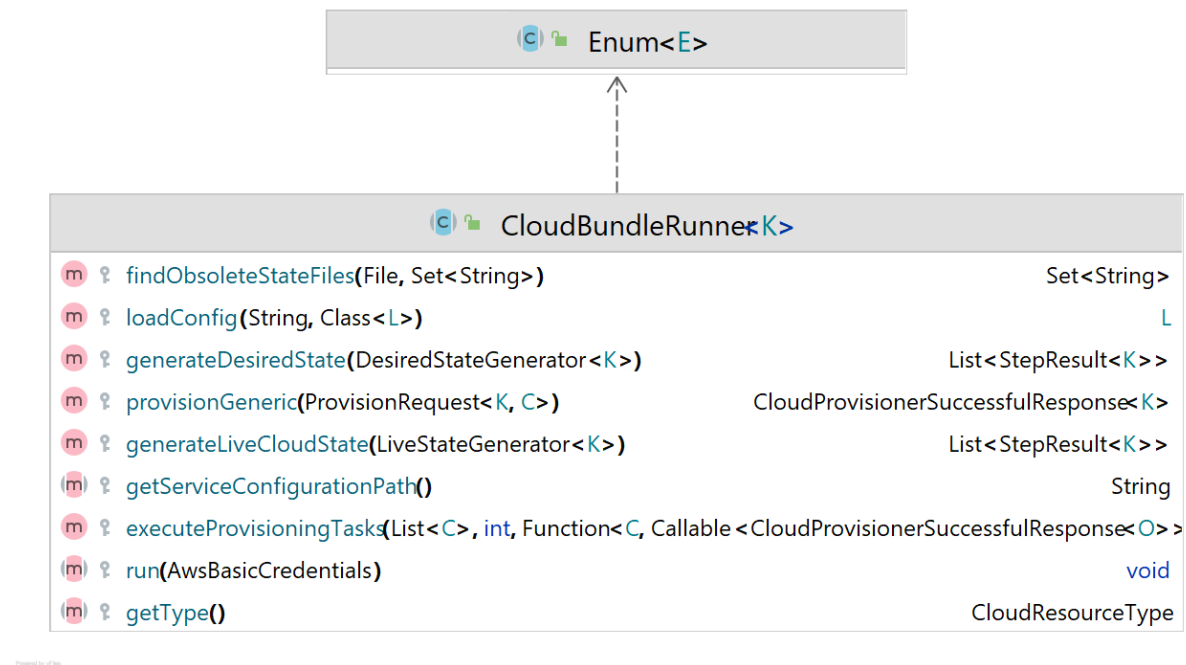
`CloudBundleRunner` трябва да обработи множество конфигурации и използва методът `executeProvisioningTasks`, който приема списък с конфигурации, размер на банка от нишки (`thread pool`) и разпределителна (`mapping`) функция, превръщаща всяка конфигурация в `Callable` задача.

```
protected <C extends BaseCloudResourceConfiguration, O  
extends Enum<O>> void executeProvisioningTasks (  
    List<C> configs,  
    int fixedThreadPoolMaxSize,  
    Function<C,  
Callable<CloudProvisionerSuccessfulResponse<O>>>  
taskMapper,  
    ErrorCode errorCode  
)
```

Фрагмент 7. Метод за паралелно създаване на ресурси с конфигурируем пул от нишки

Изпълнението е напълно паралелно и използва `ExecutorService`, а резултатите се логват индивидуално. При грешка в даден ресурс, системата улавя изключението и преминава към останалите без да спира целия процес.

CloudBundleRunner е пример за добре капсулиран архитектурен шаблон, който изолира всички логики около конкретен ресурс на едно място. Това позволява лесно добавяне на нови типове ресурси чрез имплементиране само на изпълнителя и неговите зависимости, без да се променя основната работна рамка.



Фиг.14 CloudBundleRunner: Управление на прилагането на промени по облачни ресурси

3.2.11 Архитектура за обработка на изключения

В тази глава се разглежда архитектурата, използвана за улавяне, категоризиране и обработка на изключения, възникващи по време на прилагането на инфраструктурни промени на облачни ресурси. Управлението на изключения е от съществено значение за гарантиране на устойчивост, възможност за възстановяване и проследимост на грешки в системата. Изградена е йерархия от изключения, както и централен механизъм за обработката им чрез класа `CloudExceptionHandler`.



Фиг 15. Йерархия и обработка на изключения при прилагане на промени

Класът `CloudExceptionHandler` изпълнява ролята на фасада за обработка на грешки, която предоставя различни методи за трансформиране и класифициране на изключения:

Таблица 2. Методи за обработка на изключения

<code>handleUnexpectedException</code>	Прихваща неочаквани изключения, възникнали при изпълнение на дадена стъпка (<code>CloudProvisionStep</code>) и ги обвива в <code>CloudResourceStepException</code> .
<code>handleRetryableException</code>	Обработва грешки, които са временно възстановими и могат да бъдат изпълнени отново, като ги маркира като <code>RetryableCloudResourceStepException</code> .
<code>wrapToCloudResourceStepException</code>	Позволява обвиване на произволно изключение в стандартното ни изключение от тип <code>CloudResourceStepException</code> , като запазва контекст
<code>handleSdkClientException</code>	Специализиран метод за обработка на грешки от AWS SDK клиента (<code>SdkClientException</code>), който често се използва при мрежови или конфигурационни грешки.

`CloudResourceStepException` клас представлява основният тип грешка, която може да възникне по време на изпълнение на ресурсна стъпка. Той съдържа поле `messageDetails`, което може да включва допълнителни обяснения за причината за грешката. Всички конкретни изключения, свързани с изпълнение на стъпки, се унаследяват от него.

`RetryableCloudResourceStepException` наследява `CloudResourceStepException` и маркира грешки, които могат да бъдат временно преодолими, като например лимити на ресурси, конфликти, или временна недостъпност. Това позволява на системата да приложи логика за опитване наново и да продължи изпълнението без нужда от ръчна намеса.

`CloudProvisioningTerminationException` изключение е създадено за прекъсване на целия provisioning процес, когато се достигне до ситуация, която не позволява продължаване (напр. критични конфигурационни липси или несъвместимости). То също наследява `CloudResourceStepException`.

В архитектурата присъстват и специализирани изключения за ключови етапи от инициализацията на системата. `AuthenticationException` – за грешки при автентикация (напр. при извличане на AWS credentials).

`ConfigurationLoadException` – за грешки при зареждане на YAML конфигурации.
`InitializationException` – при неуспешно стартиране на основни компоненти.
`StepResultStateWriteException` – при невъзможност да се запише състояние на стъпка (важно за поддържане на поведение за следене на състоянията).

Тази архитектура за обработка на изключения предоставя гъвкав, централизирано управляван и устойчив механизъм, който позволява класификация на грешки, откриване на възстановими състояния, както и навременно прекъсване на процеса при необходимост. Тя е от ключово значение за гарантиране на надеждност при автоматизираното изграждане на облачна инфраструктура.

Глава 4. Реализация на системата

В настоящата глава ще бъде представена конкретната реализация на основните компоненти, които изграждат системата за декларативно прилагане на промени по облачни ресурси. Докато в предходните глави беше разгледана архитектурата, принципите на проектиране и взаимодействието между модулите, тук фокусът ще бъде поставен върху детайлите на изпълнението – от механизма за автентикация до реалното създаване и управление на ресурси в AWS.

Първата част на главата е посветена на автентикационния слой, който осигурява сигурен достъп до облачните доставчици. Ще бъдат разгледани различните механизми, които библиотеката поддържа, включително автентикация чрез IAM потребители и достъпни ключове (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`), локални AWS профили (`AWS_PROFILE`), както и интеграция с Vault като външна система за управление на тайни. Ще бъде обърнато внимание на това как се изгражда абстракция върху тези механизми, така че да бъде осигурена модулност и лесна разширяемост.

Втората част на главата е насочена към реализацията на ресурсни компоненти, като ще се фокусираме върху два конкретни типа ресурси – Amazon S3 (обектно хранилище) и Amazon EKS (услуга за управлявани Kubernetes клъстър). Ще бъдат описани реалните класове, отговорни за прилагане на конфигурации, създаване, изтриване и възстановяване на ресурси, както и начина, по който се използва AWS SDK за взаимодействие с инфраструктурата. В допълнение, ще бъде разгледан начинът на генериране и сравнение на състояние, съхранението му във файлове, както и обработката на грешки и повторни опити при неизпълнени операции.

Целта на тази глава е да осигури пълна техническа яснота относно вътрешната реализация на системата, като свърже абстрактните концепции от архитектурния модел с реалния програмен код, използван при изпълнението.

4.1 Автентикация към AWS чрез IAM потребител

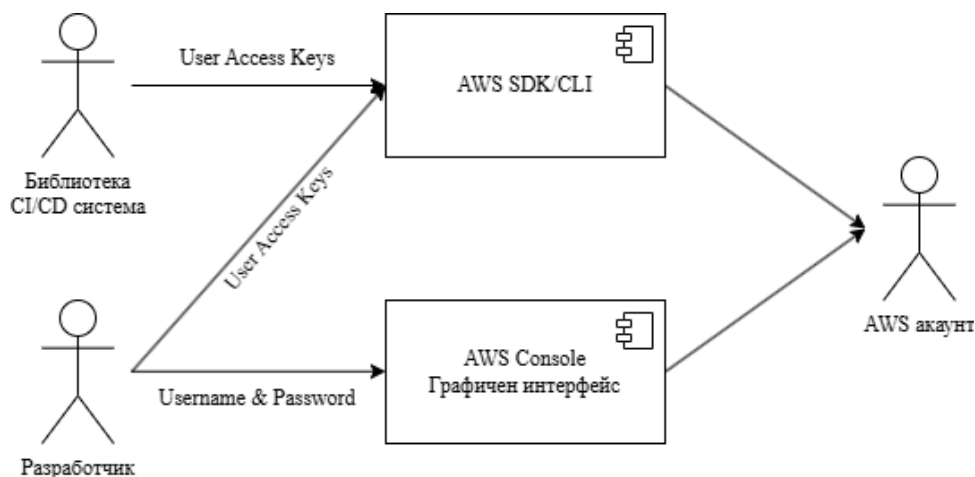
В системата за декларативно прилагане на промени по инфраструктура, автентикацията към AWS се извършва чрез два основни подхода: използване на статични идентификационни данни (`staticCredentials`) и използване на локален профил (`profileCredentials`). И в двата случая целта е да се осигури валидна идентичност, чрез която системата може да извършва операции с облачните ресурси, предоставени от Amazon Web Services (AWS).

При първия подход – статични идентификационни данни, се използват две променливи на средата: `AWS_ACCESS_KEY_ID` и `AWS_SECRET_ACCESS_KEY`. Това са уникални ключове, генерирани за конкретен IAM потребител в рамките на AWS акаунта. `AWS_ACCESS_KEY_ID` представлява публичната част на идентификатора, а `AWS_SECRET_ACCESS_KEY` е неговият таен ключ, използван за криптографски подпис на заявките. Този механизъм позволява на системата да се представи пред AWS като конкретен потребител с дефинирани права и ограничения.

Алтернативният подход използва AWS профил, конфигуриран в локалния файл `~/.aws/credentials`. В този случай променливата `AWS_PROFILE` сочи към име на профил, който съдържа същите ключове (`aws_access_key_id` и `aws_secret_access_key`), но управлението им е делегирано на AWS CLI или други инструменти. Това е удобен метод за разработчици, които използват множество профили или искат да поддържат разделение между среди (`development`, `staging`, `production`).

Централно място в този механизъм за достъп заема IAM User (Identity and Access Management User). Това е индивидуален потребител в AWS акаунт, на когото се присвояват права чрез политики (IAM policies). IAM потребителят представлява конкретен разработчик, услуга или система и има контрол върху това какви действия могат да се извършват (напр. създаване на S3 директории, четене от DynamoDB, управление на EC2 инстанции и др.). Всеки IAM потребител може да има един или повече достъпни ключове (access keys), които се използват от приложения за програмна комуникация с AWS API

.Обединявайки тези два подхода, библиотеката осигурява гъвкава и сигурна автентикация към AWS, като предоставя възможност за работа както в локални среди, така и в автоматизирани CI/CD процеси. Системата избира и прилага съответния метод чрез абстракция на ниво конфигурация и имплементация, което позволява безопасна и централизирана обработка на чувствителни идентификационни данни.



Фиг. 16 Достъп до AWS акаунт чрез AWS SDK/CLI и графичен интерфейс

```
providers:
- staticCredentials:
  accessKeyId: ${AWS_ACCESS_KEY_ID}
  secretAccessKey: ${AWS_SECRET_ACCESS_KEY}
- profileCredentials:
  profileName: ${AWS_PROFILE}
```

Фрагмент 8. Примерна конфигурация на статични идентификационни данни

4.2 Автентикация чрез Vault и дистрибутиране на ключ чрез алгоритъм на Шамир

В допълнение към автентикацията чрез статични ключове и AWS профили, системата поддържа и интеграция с HashiCorp Vault – популярен доставчик на решения за управление на тайна и чувствителна информация.

HashiCorp Vault е мощен инструмент за управление на тайна и чувствителна конфигурация, използван широко в системи с високи изисквания за сигурност. Vault предоставя централизирана услуга за съхранение на тайни стойности като пароли, API ключове, токени и сертификати, и осигурява контролиран и криптографски защитен достъп до тях. За разлика от статично записаните конфигурации или environment променливи, Vault позволява гъвкаво управление, автоматично ротация на ключове, достъп с времево ограничение и одит на всички действия.

За да бъде използван, Vault преминава през процес на инициализация и отключване (unsealing). При първоначално стартиране се генерират няколко ключа за отключване (unseal keys), които трябва да бъдат въведени от оторизирани лица, за да бъде системата функционална.

Дистрибутиране на ключ чрез алгоритъм на Шамир е криптографски алгоритъм, използван от HashiCorp Vault при инициализацията на системата, за да гарантира висока сигурност и разпределено доверие между администраторите. Идеята зад този подход е, че тайната (в случая ключът за отключване на Vault, наричан master key) не се съхранява в едно цяло, а се разделя на няколко части, наречени shares.

Когато Vault се инициализира, master ключът не се показва директно. Вместо това:

- Той се разделя на n части (например 5),
- Системата изисква k от тези n части (например 3 от 5), за да бъде Vault отключен (unsealed).

Това означава, че не е нужно всички администратори да бъдат налични едновременно – достатъчно е минимумът k да предоставят своите ключове. Това носи няколко предимства:

- Устойчивост на загуба – ако един или двама администратори загубят своите shares, Vault може да бъде отключен с останалите.
- Сигурност – нито един администратор не разполага с пълния ключ самостоятелно, което елиминира риска от вътрешни злоупотреби.

След отключване, достъпът до API се осъществява чрез специален root token, който предоставя пълни административни права. В реална продукционна среда използването на root token трябва да бъде ограничено, но за целите на разработка и локално тестване той често се използва чрез променлива на средата, например VAULT_TOKEN.

В системата за декларативно прилагане на промени, Vault се използва като външен доставчик на AWS идентификационни данни. Това става чрез конфигурация от вида:

```
providers:
- vault:
  scheme: http
  host: localhost
  port: 8200
  secretPath: /v1/application/data/cloudprovisioner/867102643694
  token: ${VAULT_TOKEN}
```

Фрагмент 9. Конфигурация за достъп до Vault доставчик на удостоверителни данни

Тази конфигурация указва адреса на Vault сървър, пътя до тайната и токена за достъп, който често се подава като променлива на средата (VAULT_TOKEN) и се извлича чрез механизма в BaseAuthenticationManager.

Основната логика за извличане на AWS кредитеншъли се съдържа в класа VaultAwsCredentialsProvider, който имплементира интерфейса CloudCredentialsProvider. Методът fetchCredentials изпраща заявка към Vault чрез клиента VaultClient и обработва отговора:

```
@Override
public AwsBasicCredentials fetchCredentials() {
    VaultResponse response = vaultClient.getSecrets(config);
    VaultAwsCredentialsData credentials = response.getData().getData();

    AwsBasicCredentials result =
        AwsBasicCredentials.create(credentials.getAccessKeyId(),
        credentials.getSecretAccessKey());
}
```

Фрагмент 10. Извличане на AWS удостоверителни данни от Vault

След успешно извличане, чувствителните стойности се замъгляват (****) преди логване, а в зависимост от режима на логване (JSON или YAML) се използва Slf4jLoggingUtil:

```
log.info("AWS credentials successfully fetched from Vault",
keyValue("response", response));
```

Фрагмент 11. Логване на замаскиран отговор от Vault

Самата HTTP комуникация с Vault се управлява от класа VaultClient, който използва Apache HttpClient и конструира URI на базата на подадената конфигурация. Заявката се изпраща чрез X-Vault-Token заглавка:

```
return new URI(
    config.getScheme(),
    null,
    config.getHost(),
    config.getPort(),
    config.getSecretPath(),
    null,
    null
);
```

Фрагмент 12. Генериране на URI адрес за достъп до Vault

```
String json = executor.execute(
    Request.get(vaultUri)
        .addHeader(VAULT_TOKEN_HEADER, extractToken(config))
).returnContent().asString();
```

Фрагмент 13. Изпращане на HTTP GET заявка към Vault с токен за автентикация

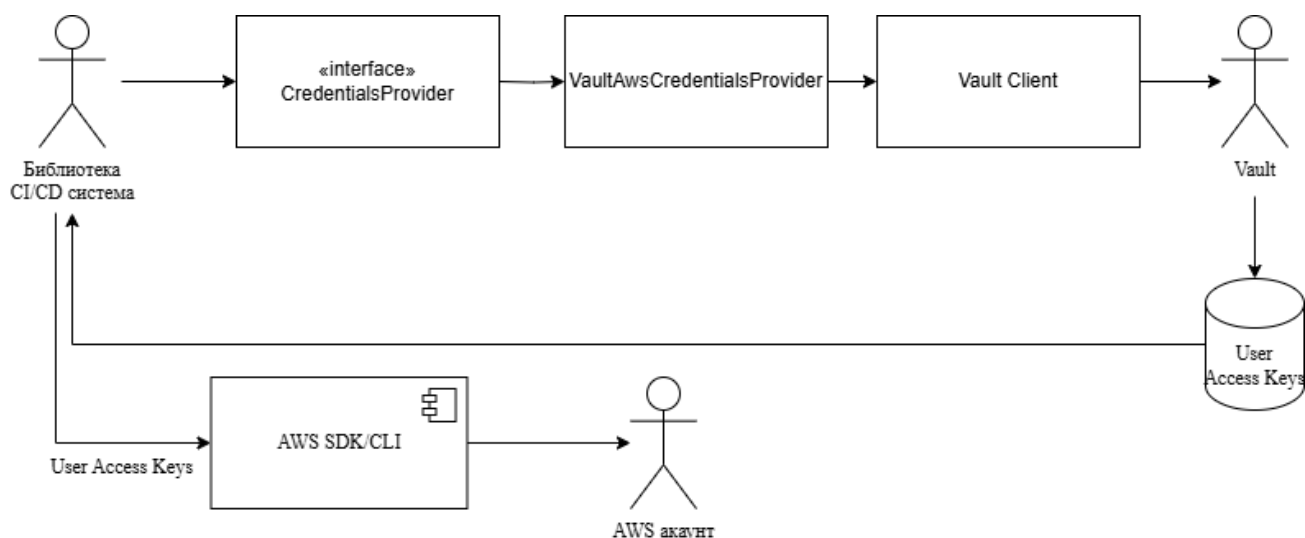
Полученият JSON отговор се парсва с помощта на Jackson в обект от тип VaultResponse, който съдържа вътрешно поле data.data, където се намират стойностите на accessKeyId и secretAccessKey.

В случай на неуспех (напр. липса на връзка или грешен токен), клиентът хвърля изключение VaultAuthenticationException с ясно съобщение и лог на нивото DEBUG, което подпомага диагностициране:

```
log.debug(Slf4jLoggingUtil.DEBUG_PREFIX + "{}", message, e);
throw new VaultAuthenticationException(message, e);
```

Фрагмент 14. Логване на изключение при опит аз автентикация с Vault

Чрез тази имплементация, библиотеката постига високо ниво на сигурност, централизация и конфигурируемост на автентикационните механизми, подходящо за среди с високи изисквания за контрол и поверителност на достъпа.



Фиг. 17 Удостоверяване към AWS чрез Vault

4.3 Репрезентация на ресурси чрез конфигурационни класове

В системата за декларативно изграждане на облачна инфраструктура, всеки вид ресурс е представен чрез йерархия от Java POJO (Plain Old Java Object) класове. Това осигурява структурирано и типово безопасно описание на конфигурациите, което улеснява валидацията, сериализацията и по-нататъшната обработка на данните. За всеки тип ресурс (напр. S3, EKS клъстер) е създаден основен конфигурационен клас, който наследява абстрактни базови класове с общи свойства и поведение.

Йерархията започва с `BaseCloudResourceConfiguration`, който дефинира универсални полета като `id`, `name`, флагове за разрешение на реконсилиация (`enableReconciliation`), забрана за изтриване (`preventDestroy`), както и конфигурация за повторни опити (`RetryConfiguration`). Този клас осигурява основна функционалност, валидна за всички видове ресурси.

```

public abstract class BaseCloudResourceConfiguration implements
IdentifiableConfig, DestroyProtection {
    protected static final String DEFAULT_EMPTY_STRING = "";
    private String id;
    private String name;
    private boolean enableReconciliation = true;
    private Boolean preventDestroy = true;
    private boolean toDelete = false;
    private RetryConfiguration retry = new RetryConfiguration();
    @Override
    public Boolean preventDestroy() {
        return preventDestroy;
    }
}
  
```

```

    }
    @Getter
    public static class RetryConfiguration {
        int max = 3;
        long backoffMs = 1000;
    }
}

```

Фрагмент 15. Абстрактна конфигурация за облачен ресурс с механизъм за защита и повторен опит

Следващото ниво в йерархията е `AwsBaseCloudResourceConfiguration`, който разширява базовия клас с AWS-специфични характеристики като регион (`region`) и тагове (`tags`). Това позволява всички AWS ресурси да споделят обща структура и поведение, без дублиране на код.

```

public abstract class AwsBaseCloudResourceConfiguration extends
BaseCloudResourceConfiguration {
    private Region region = Region.GLOBAL;
    private Map<String, String> tags = new HashMap<>();
}

```

Фрагмент 16. Абстрактна конфигурация за AWS ресурс с регион и етикети

Специфичните конфигурации се реализират в класове като `S3BucketConfig`, който описва детайлите за създаване на S3 пространство. Той включва свойства като `versioning`, `encryption`, `ownershipControls`, `acl` и `policy`. Всеки от тези атрибути е моделиран чрез вложени подкласове, което допълнително структурира конфигурацията. Например, `EncryptionConfig` описва типа криптиране и дали е активиран `bucketKey`, докато `AclConfig` управлява правата за достъп чрез политики и списъци с достъп.

Интерфейсът `ArnBuilder`, който `S3BucketConfig` имплементира, дефинира метод за изграждане на ARNs (Amazon Resource Names), като по този начин се осигурява стандартен механизъм за идентифициране на ресурси в системата.

```

public class S3BucketConfig extends AwsBaseCloudResourceConfiguration
implements ArnBuilder {
    private String versioning = BucketVersioningStatus.ENABLED.toString();
    private EncryptionConfig encryption = new
EncryptionConfig(S3EncryptionType.AES_256);
    private S3OwnershipType ownershipControls =
S3OwnershipType.BUCKET_OWNER_ENFORCED;
    private AclConfig acl = new AclConfig();
    private String policy = DEFAULT_EMPTY_STRING;
    @Override
    public String buildArn() {
        return "arn:aws:s3:::%s".formatted(getName());
    }
}

```

Фрагмент 17. Конфигурационен клас за S3 с версии, криптиране и ACL и др.

Част от конфигурацията включва вложени класове за криптиране, ACL настройки и собственик:

```
public static class EncryptionConfig {  
    private S3EncryptionType type;  
    private String kmsKeyId;  
    private boolean bucketKeyEnabled = false;  
}
```

Фрагмент 18. Конфигурационен клас за криптиране на S3 ресурс

Колекцията от ресурси е представена чрез класове като S3BucketConfigList, който наследява BaseCloudResourceConfigList и капсулира списък с конкретни конфигурации от даден тип. Това позволява зареждането на YAML конфигурации, съдържащи множество ресурси, в единна обектна структура.

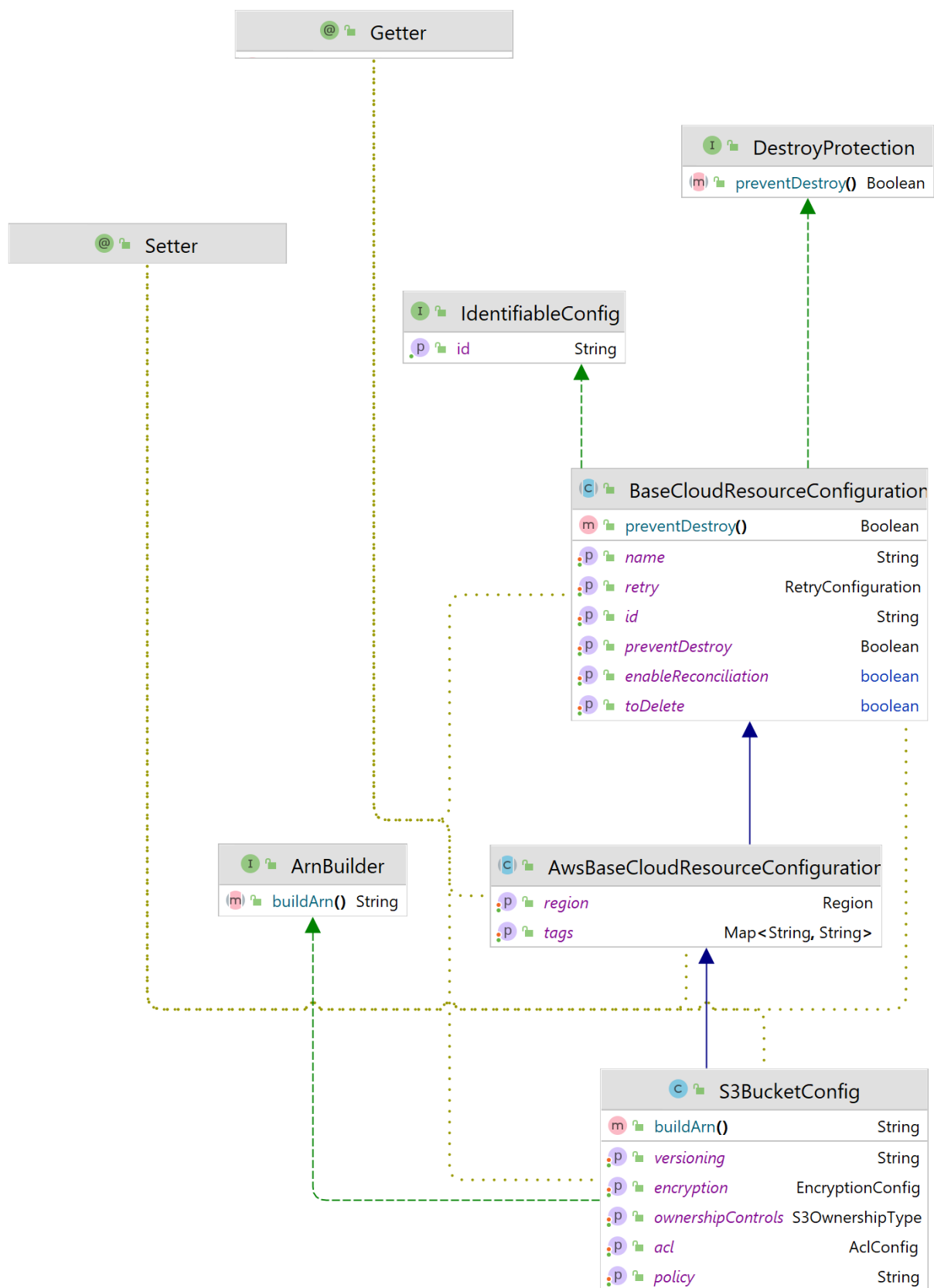
```
public abstract class BaseCloudResourceConfigList<K extends  
BaseCloudResourceConfiguration> implements CloudResourceConfigList<K> {  
    private FixedThreadPoolConfiguration fixedThreadPool = new  
FixedThreadPoolConfiguration();  
    @Getter  
    public static class FixedThreadPoolConfiguration {  
        int maxSize = 5;  
    }  
}
```

Фрагмент 19. Абстрактен клас за списъци с фиксиран брой нишки

```
public class S3BucketConfigList extends  
BaseCloudResourceConfigList<S3BucketConfig> {  
    private List<S3BucketConfig> buckets = new ArrayList<>();  
    @Override  
    public List<S3BucketConfig> getItems() {  
        return buckets;  
    }  
    @Override  
    public void setItems(List<S3BucketConfig> items) {  
        this.buckets = items;  
    }  
}
```

Фрагмент 20. Имплементация на конфигурационен списък за S3 ресурси

Тази йерархична архитектура с добре дефинирани POJO класове предоставя гъвкавост, разширяемост и лесна интеграция с YAML/JSON сериализация чрез библиотеки като Jackson. Освен това, използването на Java като описателен език за ресурсите позволява директна валидация и tooling поддръжка в среда, с която разработчиците вече са запознати.



Фиг. 18 Йерархия на конфигурационните класове за AWS ресурси (S3 пример)

4.4 Зареждане на ресурсна конфигурация от файл

След като потребителят е подготвил декларативна конфигурация за желаните ресурси – например чрез YAML файл – системата зарежда тази конфигурация в подходяща вътрешна структура от обекти. Това е първата стъпка от реалния процес по време на изпълнение (runtime), в който конфигурацията започва да бъде прилагана от библиотеката. Зареждането се извършва в метода `run()` на всеки конкретен `CloudBundleRunner` (например `S3BundleRunner`), който имплементира логиката за съответния ресурс.

```
@Override
public void run(AwsBasicCredentials credentials) {

    String path = getServiceConfigurationPath();
    log.info("Loading S3 config from: {}", path);
    S3BucketConfigList configList;
    try {
        configList = loadConfig(path, S3BucketConfigList.class);
    } catch (ConfigurationLoadException e) {
        loggingUtil.logError(log, ErrorCode.S3_CONFIG_LOAD_ERROR, e);
        return;
    }
    ...
}
```

Фрагмент 21. Изпълнение на S3 стъпка с зареждане на конфигурация и обработка на грешки

Методът `loadConfig()`, извикан вътре в `run()`, е универсален механизъм за зареждане на конфигурация от YAML файлове. Той използва помощния клас `ConfigurationUtil`, който е отговорен за десериализацията с помощта на Jackson и YAML маркер. Освен това се грижи и за безопасност при празни списъци:

В конкретния случай се зарежда файл от следния вид:

```
buckets:
- id: 11cf0cf4-76d3-4615-aecc-21497b01ecc9
  name: tuvarna-dev-eu-central-1-867102643694
  region: eu-central-1
  preventDestroy: false
  tags:
    environment: dev
    team: storage
  versioning: Enabled
  encryption:
    type: aes256
    bucketKeyEnabled: true
  ownershipControls: BucketOwnerEnforced
  acl:
```

```

canned: private
policy: |
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowFullAccessToSpecificUser",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::867102643694:root"
      },
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694",
        "arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694/*"
      ]
    }
  ]
}

```

Фрагмент 22. Декларативна конфигурация на цялостен S3 ресурс

Този YAML файл дефинира списък с обекти от тип `S3BucketConfig`, групирани под ключа `buckets`. При зареждане с помощта на Jackson, библиотеката автоматично десериализира всяка конфигурация в Java обект, и структурата от вложени класове, които описват криптиране, достъп, ACL, и т.н.

След успешното зареждане, резултатът е обект от тип `S3BucketConfigList`, който съдържа метод `getItems()` връщащ списък от `S3BucketConfig`. Тези обекти се използват в следващите стъпки за създаване или актуализиране на ресурсите в AWS.

Този подход позволява ясна граница между конфигурационно описание и изпълнение, като дава възможност потребителите на библиотеката да поддържат своите YAML файлове извън кода и да ги версионират самостоятелно. В същото време, валидирането и сериализацията са силно типове защитени благодарение на Java структурата.

След успешното зареждане на YAML конфигурацията и създаването на обекта `S3BucketConfigList`, следващата важна стъпка е идентифициране на ресурси, които преди са били част от инфраструктурата, но вече липсват в новата конфигурация. Това се отнася до така наречените „осиротели“ ресурси – такива, които съществуват в локалното състояние (state файловете), но вече не присъстват в текущия YAML файл.

За тази цел се използва методът `findObsoleteStateFiles()`, който обхожда `.cloudprovisioner/` директорията и събира имената на всички state файлове, които не отговарят на текущите id стойности в конфигурацията:

```
Set<String> ids =
configList.getBuckets().stream().map(S3BucketConfig::getId).collect(Collectors.toSet());
String profile = EnvVar.AWS_PROFILE.getValue();
File stateDir = new File(".cloudprovisioner/" + profile + "/" +
getType().toString().toLowerCase());
```

Фрагмент 23. Маркиране на остарели S3 конфигурации за изтриване (1)

Резултатът е списък от „остарели“ конфигурационни файлове – такива, които трябва да бъдат изтрети от реалната инфраструктура. Вместо да се изтрият директно, библиотеката добавя временна конфигурация за всеки от тях, маркирана с флаг `toDelete = true`. Това позволява системата да тренира унифициран подход за изпълнение на всички ресурси – съществуващи и маркирани за премахване – по еднакъв начин.

```
for (String stateKey : findObsoleteStateFiles(stateDir, ids)) {
    String[] parts = stateKey.split("#");
    S3BucketConfig delete = new S3BucketConfig();
    delete.setName(parts[0]);
    delete.setId(parts[1].replace(".json", ""));
    delete.setToDelete(true);
    delete.setPreventDestroy(null);
    configList.getBuckets().add(delete);
    log.info("Bucket with configKey '{}' is removed from config and is marked
for deletion", stateKey);
}
```

Фрагмент 24. Маркиране на остарели S3 конфигурации за изтриване (2)

В този блок от код се създава нов обект от тип `S3BucketConfig`, който съдържа само основна информация – името и ID-то на ресурса, извлечени от името на съществуващия state файл. Целта на тази конфигурация е да отбележи, че ресурсът вече не присъства в YAML конфигурацията и следва да бъде премахнат. За тази цел свойството `toDelete` се задава на `true`, което ясно указва, че това е ресурс за изтриване („deletion-only“). Освен това, ако конфигурацията е била защитена от изтриване чрез `preventDestroy`, на тази защита се присвоява празна стойност, за да не се възприема като „вярна“ всеки път. За проверка дали изтриването на ресурса е позволено ще се използва флагът, който по-рано е зададен в `metadata` стъпката. Така създаденият обект се добавя обратно в списъка с ресурси (`configList.getBuckets()`), където ще бъде обработен по същия начин като останалите конфигурации в следващите етапи от изпълнението – но със специфично поведение, насочено към изтриване вместо създаване или актуализиране.

4.5 Обединяване на архитектурните зависимости и компоненти

След като желаната конфигурация е заредена и са идентифицирани ресурсите, които трябва да бъдат създадени, актуализирани или изтрети, библиотеката преминава към етапа на изпълнение. За целта всяка отделна конфигурация от списъка се обработва чрез извикване на метода `executeProvisioningTasks`. Той стартира процеса по изпълнение на инфраструктурните стъпки за всеки ресурс, като използва фиксиран брой нишки, чиито максимум е дефиниран в YAML конфигурацията.

```
try (ExecutorService executor =  
    Executors.newFixedThreadPool(Math.min(configs.size(),  
        fixedThreadPoolMaxSize)))
```

Фрагмент 25. Изпълнение на фиксиран брой нишки

Всяка задача предава съответната конфигурация към метода `provisionS3`, който отговаря за конкретната имплементация за даден ресурс – в случая `S3 bucket`.

Методът `provisionS3` създава и подава обект от тип `ProvisionRequest`, който съдържа всички архитектурни зависимости и компоненти, нужни за изпълнението на инфраструктурната логика. Този обект действа като инжекционен контейнер и предоставя методи-фабрики, които библиотеката използва, за да създаде:

- списък със стъпки, нужни за изпълнение (`stepProvider`)
- компоненти за унищожаване на съществуващи ресурси (`destroyerProvider`);
- компоненти за създаване на нови или актуализиране на съществуващи ресурси (`provisionerProvider`);
- компоненти за възстановяване при грешка (`reverterProvider`);
- функции за генериране на текущото състояние в облака (`liveStateGenerator`);
- функции за създаване на желаното състояние (`desiredStateGenerator`);
- функция за създаване на `Injector`, който ще се използва за внедряване на зависимости;
- `metadataSupplier`, който създава началното състояние с основни параметри като име, регион и ARN;
- правила за извличане на конкретни стойности от `StepResult`, като име и регион;
- класът на изходите с изброими стойности, използван от стъпките (`outputClass`).

```

• ...
• return provisionGeneric(
    ProvisionRequest.<S3Output, S3BucketConfig>builder()
        .config(config)
        .profile(profile)
        .stateDir(stateDir)
        .resourceKey(getType().getValue())
        .metadataStepName(S3PersistentMetadataStep.class.getName())
        .stepProvider(inj -> inj.getInstance(Key.get(new
TypeLiteral<>() {
    }, Names.named(NamedInjections.S3_STEPS))))
        .destroyerProvider(inj ->
inj.getInstance(S3BucketDestroyer.class))
        .provisionerProvider(inj ->
inj.getInstance(S3BucketProvisioner.class))
        .reverterProvider(inj ->
inj.getInstance(S3BucketReverter.class))
        .liveStateGenerator(inj ->
generateLiveCloudState(inj.getInstance(S3LiveStateGenerator.class)))
        .desiredStateGenerator(inj ->
generateDesiredState(inj.getInstance(S3DesiredStateGenerator.class)))
        .injectorFactory((region, metadata) ->
Guice.createInjector(new S3InjectionModule(
    new S3ProvisioningContext(credentials,
EnvVar.ENDPOINT_URL.getValue(), Region.of(region)), config,
metadata)))
        .metadataSupplier(cfg -> StepResult.<S3Output>builder()
            .stepName(S3PersistentMetadataStep.class.getName())
            .put(S3Output.NAME, cfg.getName())
            .put(S3Output.REGION, cfg.getRegion().getValue())
            .put(S3Output.ARN, cfg.buildArn())
            .put(S3Output.PREVENT_DESTROY, cfg.preventDestroy() !=
null ? cfg.preventDestroy() : Boolean.TRUE)
            .build())
        .nameExtractor(step -> (String)
step.getOutputs().get(S3Output.NAME))
        .regionExtractor(step -> (String)
step.getOutputs().get(S3Output.REGION))
        .outputClass(S3Output.class)
        .build()
    );
• ...

```

Фрагмент 26. Конструирание на заявка за създаване на S3 ресурси

Всички тези компоненти се обединяват в един унифициран `ProvisionRequest` чрез билдър шаблон, който след това се предава към абстрактния метод `provisionGeneric`. По този начин се постига висококохезивна и слабо-свързана архитектура, при която всеки ресурс може да използва своя собствена имплементация на стъпки, генератори и обработчици, без да пречи на останалите.

Това структуриране позволява архитектурата да бъде изключително разширяема – нов тип ресурс изисква само добавяне на конкретна имплементация на нужните компоненти и дефиниране на съответния `ProvisionRequest`, без да се променя основната библиотека.

4.6 Прилагане на инфраструктурни промени

4.6.1 Начално зареждане на състоянието

Процесът по прилагане на инфраструктурни промени в библиотеката започва с критичната фаза на зареждане на съществуващото състояние на ресурса. Това е необходимо, за да се извърши т.нар. изравняване (reconciliation) – сравнение между последно известното състояние и реалното състояние в облака, с цел откриване на разлики (configuration drift) и планиране на съответните действия, както и за сравнение с бъдещото желано състояние.

```
List<StepResult<K>> loadedState = new ArrayList<>();
try {
    File[] matchingFiles =
Optional.ofNullable(request.stateDir.listFiles((dir, name) ->
    name.startsWith("state-") && name.endsWith("#" +
request.config.getId() + ".json"))).orElse(new File[0]);
    if (matchingFiles.length > 1) {
        throw new ConfigurationLoadException("Found multiple state files for
id: " + request.config.getId());
    } else if (matchingFiles.length == 0) {
        throw new FileNotFoundException("No state file found for id: " +
request.config.getId());
    }
    File stateFile = matchingFiles[0];
    loadedState = storedState.load(stateFile, request.outputClass);
}
```

Фрагмент 27. Зареждане на съществуващото състояние на ресурса

Първата стъпка е откриването на файл, който съдържа записаното състояние на ресурса. Всеки state файл следва конвенцията: state-
<resourceName>#<resourceId>.json, което позволява лесно филтриране и съвпадение по идентификатор на конфигурацията. Ако са намерени повече от един файл с еднакъв ID, се счита, че конфигурацията е повредена и се прекратява изпълнението с изключение ConfigurationLoadException.

Ако такъв файл изобщо не съществува, това означава, че ресурсът никога не е бил създаван чрез библиотеката и следователно трябва да бъде създаден от нулата. В този случай се логира информационно съобщение и процесът продължава с празно състояние.

```
catch (FileNotFoundException e) {
    log.info("{}, {}' is not yet provisioned. Will be created from scratch",
request.resourceKey, request.config.getName());
}
```

Фрагмент 28. Логиране на информационно съобщение за нов ресурс

В случай че съществува файл, но не може да бъде прочетен – поради входно-изходен проблем, загуба на права или проблеми при сериализация – се обработва

като `IOException`. В този случай се логира грешка със съответен код и се прекратява изпълнението с изключение `CloudProvisioningTerminationException`:

```
catch (IOException e) {
    loggingUtil.logError(log, ErrorCode.S3_STORED_STATE_ERROR, e);
    String message = "Failed to load persisted state for %s '%s'. File might
be unreadable or corrupted."
        .formatted(request.resourceKey.toLowerCase(),
request.config.getName());
    throw new CloudProvisioningTerminationException(message, e);
}
```

Фрагмент 29. Терминиране на процеса при грешка в прочитането

Тази фаза е от особено значение за поддържането на идемпотентност в процеса – всяко изпълнение започва от точно определено и последователно състояние, което се сравнява и при нужда се модифицира, а не се изпълняват слепи операции върху реалната инфраструктура. Това е и основата, върху която се надгражда сравняването със състоянието в облака и прилагането на промените.

4.6.2 Запазване на неизменна информация чрез стъпка с метайнформация

След като началното състояние на ресурса бъде успешно заредено, следва ключова фаза от процеса по прилагане на инфраструктурни промени – създаването или извличането на така наречената `metadata` стъпка. Тя служи като „нулева“ стъпка, която не се променя по време на жизнения цикъл на ресурса и съдържа перманентни характеристики, нужни за точно проследяване и защита на ресурса.

```
StepResult<K> metadata = loadedState.stream()
    .filter(result ->
result.getStepName().equals(request.metadataStepName))
    .findFirst()
    .orElseGet(() -> request.metadataSupplier.apply(request.config));
```

Фрагмент 30. Опит за извличане на стъпка с метайнформация

Ако ресурсът не е бил създаван преди (няма заредена `metadata` стъпка от `loadedState`), тя се създава в момента чрез `metadataSupplier`. Това се случва при първоначално създаване на ресурс, като от конфигурацията се извличат нужните стойности и се запазват в неизменен вид.

```
.metadataSupplier(cfg -> StepResult.<S3Output>builder()
    .stepName(S3PersistentMetadataStep.class.getName())
    .put(S3Output.NAME, cfg.getName()))
```

```
.put(S3Output.REGION, cfg.getRegion().getValue())
.put(S3Output.ARN, cfg.buildArn())
.put(S3Output.PREVENT_DESTROY, cfg.preventDestroy() != null ?
cfg.preventDestroy() : Boolean.TRUE)
.build()
```

Фрагмент 31. Създаване на стъпка с метатаинформация

Идентифициране на ресурса

Всяка услуга в AWS има полета, които са непроменими след създаване. Например при S3 това е името на пространството, което е глобално уникално и не може да бъде променено. Ако потребителят промени името в конфигурационния YAML файл, системата трябва да може да сравни „декларираното“ с реално съществуващото име, за да засече неправилна промяна и да хвърли изключение. Това сравнение е възможно само ако началното име е било запазено в стъпката с метатаинформация.

Защита от грешно изтриване

Атрибутът `preventDestroy` също се съхранява като метатаинформация, защото в бъдещи изпълнения той може да е изтрит или променен в конфигурационния файл. Ако в текущата конфигурация `preventDestroy` е празна стойност, няма как да знаем дали някога е бил зададен или ресурсът винаги е бил „изтриваем“. Затова се ползва стойността от `metadata` стъпката като източник на истина.

Изграждане на инжекционен контейнер:

Стойностите от `metadata` стъпката (напр. име, регион, ARN и др.) се използват за изграждане на `Injector`, който събира всички зависимости, нужни за прилагането на стъпките.

```
String name = request.nameExtractor.apply(metadata);
String region = request.regionExtractor.apply(metadata);
Injector injector = request.injectorFactory.apply(region, metadata);
```

Фрагмент 32. Изграждане на ресурсен инжекционен контейнер

Така стъпката с метатаинформация играе централна роля в цялостния жизнен цикъл на ресурса – тя служи като стабилен показател за състоянието му, позволява защита от потребителски грешки и спомага за точното и безопасно управление на ресурсите в облака.

4.6.3 Динамично внедряване на зависимости

След като бъдат извлечени ключовите стойности от стъпката с метатаинформация (например име и регион на ресурса), следващият важен етап от прилагането на инфраструктурните промени е създаването на инжекционен контейнер. В

системата тази роля се изпълнява от Google Guice, който предоставя лесен и гъвкав механизъм за внедряване на зависимости - dependency injection.

Guice инжекторът има за цел да създаде и управлява всички необходими обекти (стъпки, клиенти, изпълнители и контексти), нужни за прилагането на конкретна конфигурация. Вместо тези обекти да бъдат създавани ръчно във всяка стъпка от изпълнението, те се регистрират и подават автоматично от Guice въз основа на дефинирани модули.

```
.injectorFactory((region, metadata) -> Guice.createInjector(new  
S3InjectionModule(  
    new S3ProvisioningContext(credentials, EnvVar.ENDPOINT_URL.getValue(),  
    Region.of(region)), config,  
    metadata)))
```

Фрагмент 33. Регистриране на S3 конфигурация и метаданни в Guice контейнер

S3 конфигурация и метаданни

Тези два обекта са уникални за всеки ресурс:

```
@Override  
protected void configure() {  
  
    bind(S3BucketConfig.class).toInstance(s3BucketConfig);  
    bind(new TypeLiteral<StepResult<S3Output>>() {}).toInstance(s3metadata);  
}
```

Фрагмент 34. Регистриране на S3 конфигурация и метаданни в Guice контейнер

Предоставя обект S3SafeClient за работа с AWS S3 API, като използва предварително подадени AWS данни за автентикация, регион и по желание персонализирана крайна API точка:

```
@Provides  
S3SafeClient s3SafeClient() {  
    S3ClientBuilder clientBuilder = S3Client.builder()  
  
    .credentialsProvider(StaticCredentialsProvider.create(s3Context.getCredentials()))  
    .region(s3Context.getRegion())  
    .forcePathStyle(true);  
    if (s3Context.getEndpoint() != null) {  
        clientBuilder.endpointOverride(URI.create(s3Context.getEndpoint()));  
    }  
    return new S3SafeClient(clientBuilder.build());  
}
```

Фрагмент 35. Създаване на AWS S3 клиент с параметризирана конфигурация

Списък от стъпки за изпълнение

Guice предоставя списък от стъпки, който се използва за изграждането на целия процес по създаване и настройка на S3 ресурса:

```
@Provides
@Named(NamedInjections.S3_STEPS)
List<CloudProvisionStep<S3Output>> s3ProvisionSteps(...) {
    return List.of(
        metadataStep,
        createStep,
        policyStep,
        versioningStep,
        taggingStep,
        encryptionStep,
        aclStep,
        ownershipControlsStep
    );
}
```

Фрагмент 36. Всички S3-специфични стъпки

Изпълнители за стратегии, изтриване и възстановяване

Тези обекти управляват изпълнението на стъпките по конкретна стратегия, при неуспех или при изтриване.

```
@Provides
CloudStepStrategyExecutor<S3Output> s3StepExecutor() {
    return new CloudStepStrategyExecutor<>(s3BucketConfig.getRetry());
}

@Provides
CloudStepDeleteExecutor<S3Output> s3DeleteStepExecutor() {
    return new CloudStepDeleteExecutor<>();
}

@Provides
CloudStepRevertExecutor<S3Output> s3StepRevertExecutor() {
    return new CloudStepRevertExecutor<>(s3BucketConfig.getRetry());
}
```

Фрагмент 37. Изпълнители за стратегии, изтриване и възстановяване

Guice инжекторът действа като временен контейнер за зависимости, създаден по време на изпълнение за всяка S3 конфигурация поотделно. Той позволява гъвкаво инжектиране на всички нужни компоненти, без да е необходимо те да бъдат създавани ръчно. По този начин архитектурата на системата остава модулна, мащабируема и лесна за разширение с нови ресурси или стъпки.

4.6.4 Зареждане на актуално живо състояние

След инициализирането на инжектора и компонентите, които го съпровождат, следващата важна стъпка в процеса на прилагане на инфраструктурни промени е зареждането на текущото (реално) състояние на ресурсите в облака. Това се осъществява чрез специален компонент — Live State Generator.

```
List<StepResult<K>> currentState =  
request.liveStateGenerator.apply(injector);
```

Фрагмент 38. Зареждане на актуално състояние

Всяка заявка ProvisionRequest съдържа функция liveStateGenerator, която получава Injector и извлича конкретен генератор, отговарящ за дадения ресурс. За S3 това изглежда така:

```
.liveStateGenerator(inj ->  
generateLiveCloudState(inj.getInstance(S3LiveStateGenerator.class)))
```

Фрагмент 39. Извличане на генератор на живо състояние

```
@Slf4j  
@Singleton  
public class S3LiveStateGenerator implements LiveStateGenerator<S3Output> {  
    private final CloudStepStrategyExecutor<S3Output> stepExecutor;  
    private final List<CloudProvisionStep<S3Output>> provisionSteps;  
    @Inject  
    public S3LiveStateGenerator(CloudStepStrategyExecutor<S3Output>  
stepExecutor,  
                                @Named(NamedInjections.S3_STEPS)  
List<CloudProvisionStep<S3Output>> provisionSteps) {  
        this.stepExecutor = stepExecutor;  
        this.provisionSteps = provisionSteps;  
    }  
    @Override  
    public List<StepResult<S3Output>> generate() throws  
CloudProvisioningTerminationException {  
        try {  
            return stepExecutor.execute(provisionSteps,  
CloudProvisionStep::getCurrentState);  
        } catch (CloudResourceStepException e) {  
            ...  
        }  
    }  
}
```

Фрагмент 40.

Този генератор използва вече конфигурираните стъпки и специален CloudStepStrategyExecutor, който стартира всяка от тях с цел да извлече моментното състояние на съществуващите ресурси. Това става чрез извикване на метода getCurrentState във всяка стъпка.

Примерно, ако имаме S3 пространство, което вече съществува в AWS, тогава `getCurrentState` ще направи заявка към AWS и ще върне реалните атрибути — дали има включено версионирание, какви тагове има, дали е активирана криптографията и др.

Компонентът `CloudStepStrategyExecutor` изпълнява всяка стъпка, като при необходимост използва конфигурацията за `retry` от конфигурационния файл. Така се гарантира устойчивост срещу временни грешки в мрежата или `cloud API`.

В случай че при извличане на състоянието възникне грешка, генераторът връща изключение `CloudProvisioningTerminationException`, което прекъсва понататъшната обработка и записва грешката в логовете.

`LiveStateGenerator` е отговорен за синхронизирането между реалната инфраструктура и описаната в конфигурационните файлове. Той е от критично значение при откриване на разлики, защото без точна представа за настоящото състояние не може да се определи какво и дали трябва да бъде променено. Чрез тази стъпка системата получава пълна картина на „какво имаме“ преди да реши „какво искаме да променим“.

`CloudStepStrategyExecutor` прилага `Strategy Design Pattern`, и то в много ясен и целенасочен вид.

Имаме следната функционална интерфейсна дефиниция:

```
@FunctionalInterface
public interface StepExecutionStrategy<K extends Enum<K>> {
    StepResult<K> execute(CloudProvisionStep<K> step);
}
```

Фрагмент 41.

Това е типичен стратегически интерфейс, който дефинира поведение за изпълнение на стъпка — но без да казва как точно ще се изпълни. Това поведение се предоставя динамично при стартиране.

```
private StepResult<K> executeWithRetry(CloudProvisionStep<K> step,
StepExecutionStrategy<K> strategy) {
    ...
    return strategy.execute(step);
    ...
}
```

Фрагмент 42.

Стратегията `executionStrategy` се подава отвън като поведение — например `CloudProvisionStep::getCurrentState` или `CloudProvisionStep::apply`.

Тя се инжектира по време на изпълнение, което позволява на CloudStepStrategyExecutor да е независим от конкретната логика.

4.6.5 Сравнение между последно записано и актуално живо състояние

В процеса на прилагане на инфраструктурни промени, след като бъде създаден Injector и бъде извлечено живото състояние от облачната среда, следващият ключов етап е сравнението между последно записаното състояние и актуалното живо състояние. Целта на този етап е да се идентифицират конфигурационни разминавания (configuration drift), които изискват реакция от системата – възсанолявяване към състоянието от заредения файл или игнориране на разликите.

Сравнението се извършва чрез компонент, имплементиращ интерфейса StateComparator, например:

```
public class S3StateComparator implements StateComparator<S3Output> {
    @Override
    public List<StepResult<S3Output>> diff(List<StepResult<S3Output>>
desired, List<StepResult<S3Output>> actual) {
        Map<String, StepResult<S3Output>> actualMap = actual.stream()
            .collect(Collectors.toMap(StepResult::getStepName,
Function.identity()));
        List<StepResult<S3Output>> changed = new ArrayList<>();
        for (StepResult<S3Output> desiredStep : desired) {
            StepResult<S3Output> actualStep =
actualMap.get(desiredStep.getStepName());

            if (actualStep != null && !actualStep.equals(desiredStep)) {
                changed.add(desiredStep);
            }
        }
        return changed;
    }
}
```

Фрагмент 43.

Този метод сравнява всяка стъпка по име и съдържание. Ако стойностите не съвпадат, стъпката се маркира като променена и се добавя в списъка changed.

След като разликите бъдат определени, те се извеждат чрез метода logProvisioningChanges:

```
log.info("Comparing live state with the last persisted state to detect
configuration drift...");
List<StepResult<K>> changed = stateComparator.diff(loaderState,
currentState);
logProvisioningChanges(changed, currentState);
```

Фрагмент 44.

Методът `logDifferences` предоставя подробна разбивка за всяка промяна:

```
for (Map.Entry<K, Object> entry : newOutputs.entrySet()) {
    Object oldVal = oldOutputs.get(entry.getKey());
    Object newVal = entry.getValue();

    if (!Objects.equals(oldVal, newVal)) {
        diffs.add("  ↪ " + entry.getKey() + ": '" + oldVal + "' → '" + newVal +
        "'");
    }
}
```

Фрагмент 45.

Този етап осигурява надежен механизъм за засичане на отклонения между текущо запазеното и реално състояние на инфраструктурата. Така системата е нясно за промени, които не са направени през библиотеката. Това е ключов аспект на всеки инструмент за управление на инфраструктура като код.

4.6.6 Прилагане на промени върху реалната инфраструктура

След като системата е установила разлики между текущото състояние в облака и последно записаното състояние в локалната файлова система (configuration drift), следва етапът на прилагане на коригиращи действия, така че инфраструктурата да се върне в съответствие с конфигурационния файл.

Преди да се премине към действие, библиотеката проверява дали за конкретния ресурс е позволено извършването на автоматично изравняване на състоянията (reconciliation). Това е флаг от конфигурацията:

```
if (!request.config.isEnabledReconciliation()) {
    log.info("Reconciliation is disabled for {} '{}'. Skipping provisioning
changes", request.resourceKey.toLowerCase(), name);
    changed = new ArrayList<>();
}
```

Фрагмент 46.

Ако автоматичното изравняване на състоянията не е разрешено, списъкът с разлики `changed` се занулява и не се извършват действия. В противен случай, се преминава към прилагане на промените чрез метода `applyChanges`.

Методът `applyChanges` е отговорен за изпълнението на нужните стъпки, които водят до актуализиране на инфраструктурата:

Той съдържа три основни фази:

1. Определяне кои стъпки да се изтрият и кои да се изпълнят

```
List<CloudProvisionStep<K>> stepsToProvision =  
resolveStepsToProvision(allSteps, changes);  
List<CloudProvisionStep<K>> stepsToDelete = resolveStepsToDelete(allSteps,  
changes);
```

Фрагмент 47.

Използва се `destroyer` – компонент, който унищожава съществуващи ресурси. В примера с S3 това се извършва чрез `S3BucketDestroyer`:

```
CloudProvisionerSuccessfulResponse<K> destroyed = destroy(destroyer,  
stepsToDelete, true);
```

Фрагмент 48.

`destroyer` използва вътрешно `CloudStepDeleteExecutor`, който преминава през всички стъпки и извиква метод `delete()`.

След изтриването се проверява дали между изтритите стъпки присъства главен ресурс (маркиран с `@MainResource`). Ако да — това означава, че целият ресурс е бил унищожен и не се продължава с последваща модификация:

```
if (isMainResourceDeleted(destroyed, profile, bucket, id)) {  
    return destroyed;  
}
```

Фрагмент 49.

2. Създаване и модифициране на стъпки

Ако основният ресурс не е бил изтрит, продължаваме с прилагане на промените чрез `provisioner`. В S3 контекста, това е `S3BucketProvisioner`:

```
CloudProvisionerSuccessfulResponse<K> response = provision(provisioner,  
stepsToProvision);
```

Фрагмент 50.

provisioner използва CloudStepStrategyExecutor, за да приложи всички стъпки (например create, tag, enable versioning и т.н.) чрез извикване на step.apply().

3. Актуализиране на state файла

След успешно изпълнение, резултатите от стъпките се обединяват със съществуващото състояние, а новото състояние се записва обратно във файлова система, с цел бъдещи сравнения:

```
mergeAndPersistState(profile, bucket, id, response, originState);
```

Фрагмент 51.

Етапът на изравняване позволява библиотеката да поправя разлики между описаната конфигурация и реалното състояние. Това включва:

- Изтриване на стари или несъответстващи ресурси чрез destroyer.
- Модифициране или създаване на липсващи компоненти чрез provisioner.
- Записване на резултатите в нов state файл, който ще бъде използван при следващи проверки.

Този процес гарантира, че инфраструктурата се поддържа в синхрон с декларативното описание, минимизирайки нуждата от ръчни интервенции.

4.6.8 Гарантиране на атомарност чрез транзакционно възстановяване

В реална инфраструктурна среда, възможността за възникване на грешки по време на прилагането на ресурси е неизбежна — може да се дължат на временна липса на достъп до облака, непълна конфигурация или конфликт в текущото състояние. Затова библиотеката внедрява транзакционен подход към прилагането на промени: или всички стъпки се изпълняват успешно, или нито една не се счита за приложена, а всички направени действия се възстановяват.

Ако по време на applyChanges() възникне изключение, се задейства логиката за връщане назад чрез метода handleRollback(). Това се случва в рамките на try-catch блок:

```
try {  
    applied = applyChanges(...);  
} catch (CloudProvisioningTerminationException e) {  
    handleRollback(injector, destroyer, request.reverterProvider, name,  
request.profile, request.resourceKey, request.config.getId(), allSteps,  
loadedState, request.liveStateGenerator);  
    throw e;  
}
```

Фрагмент 52.

Веднага след като бъде хваната грешка, системата преминава в режим на възстановяване на началното състояние, заредено преди началото на промените. Това гарантира, че не се оставят ресурси в частично модифицирано състояние.

Връщането назад се състои от следните фази:

1. Зареждане на текущото живо състояние, за да се идентифицират разлики с оригиналното
2. Определяне на засегнатите стъпки – кои трябва да бъдат изтрити, кои да се върнат към предходното си състояние
3. Изпълнение на унищожаване чрез `destroyer`, но без да се проверява `preventDestroy`
4. Ако основният ресурс не е бил изтрит, се изпълняват действията по възстановяване чрез `reverter`
5. Актуализиране на файла на състоянието, така че системата да отразява реалното възстановено състояние

По този начин библиотеката гарантира пълна атомарност – или всички стъпки се изпълняват, или при неуспех всички странични ефекти се елиминират, връщайки системата в първоначалното ѝ състояние.

Механизмът за връщане назад е сърцето на транзакционния подход към облачните операции. Той елиминира риска от частично приложени ресурси, нестабилни или неочаквани конфигурационни състояния, загуба на контрол над инфраструктурата след неуспешно прилагане на промените. Тази вградена устойчивост прави библиотеката подходяща за използване в критични среди, където надеждността и предвидимостта на инфраструктурните промени са от съществено значение.

4.6.9 Генериране и прилагане на желаното състояние

Последната фаза от процеса по прилагане на инфраструктурни промени е свързана с извеждането на желаното състояние, дефинирано в конфигурационните YAML файлове, и неговото прилагане върху реалната облачна инфраструктура. Тази стъпка идва след синхронизирането с текущото живо състояние и цели да гарантира, че инфраструктурата отговаря на последната конфигурация, зададена от потребителя.

Генерирането на желаното състояние се реализира чрез компонента `DesiredStateGenerator`, конкретно неговата имплементация за S3 – `S3DesiredStateGenerator`.

Този компонент използва същите CloudProvisionStep обекти, но извиква generateDesiredState() върху всяка стъпка.

```
return stepExecutor.execute(steps,  
CloudProvisionStep::generateDesiredState);
```

Фрагмент 53.

По този начин се симулира как трябва да изглежда състоянието на ресурса, ако конфигурацията бъде напълно приложена.

След генерирането на желаното състояние, то се сравнява със зареденото преди това текущо състояние (live state). Ако бъдат открити различия – независимо дали това са нови атрибути, промяна на стойности или липсващи ресурси – те се извеждат в логовете и се подготвят за прилагане. Прилагането на разликите следва същия механизъм като при предишната фаза. Това включва - изпълнение на стъпки за изтриване, когато съществуват остарели елементи и изпълнение на стъпки за създаване/модификация, за да се достигне до новото конфигурационно състояние. Ако по време на прилагането на желаното състояние възникне грешка, системата повтаря същата логика за връщане назад, както и при предходната фаза. Това гарантира, че инфраструктурата остава стабилна, дори ако потребителят е подал невалидна или непълна конфигурация.

Компонентът DesiredStateGenerator служи като връзка между потребителската конфигурация и реалния свят. Чрез него: Се валидира дали дефинираното в YAML съвпада с действителното състояние, осъществява се прилагане на най-новите промени, постига се цялостно съответствие между желаната и реалната инфраструктура, без да се допуска частично или непълно състояние.

Тази фаза затваря цикъла по инфраструктурно управление и осигурява завършена, предсказуема и надеждна автоматизация.

4.7 Стъпки за създаване и модифициране на S3 ресурси

След като описахме цялостния процес по прилагане на инфраструктурни промени — от зареждането на конфигурацията до обновяването на файловете със състояние — в тази глава ще преминем към разглеждане на конкретните ресурси, които нашата библиотека създава и управлява. Ще започнем с Amazon S3, един от най-използваните облачни услуги за съхранение на обекти.

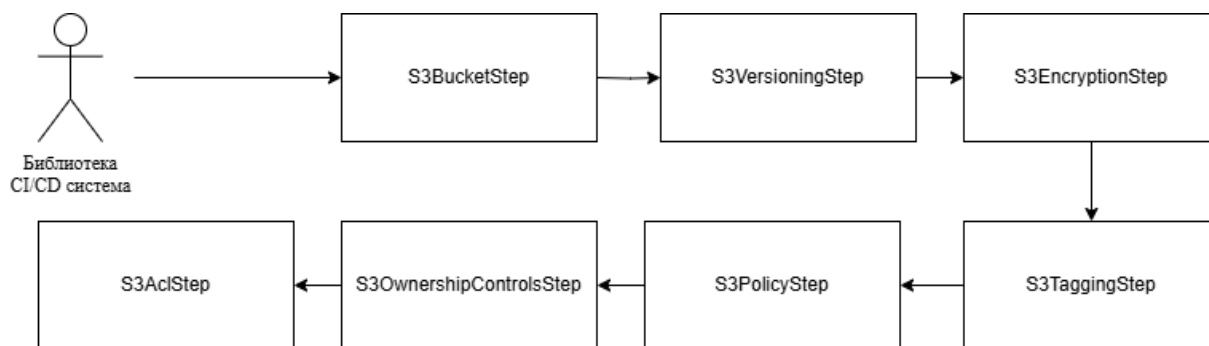
Всяка инфраструктурна операция върху S3 ресурси се състои от поредица от стъпки, които се изпълняват в строго дефиниран ред, контролиран от анотацията

@ProvisionOrder. Това позволява изграждането на последователен и зависим поток от действия, като например първо се създава пространството, след което се активира versioning, добавят се политики и накрая се настройват права за достъп.

Първата стъпка в процеса е S3BucketStep, която е маркирана с анотациите @MainResource и @PreventModification

@MainResource указва, че това е основният и водещ ресурс в групата. Ако той бъде маркиран за изтриване, това означава, че целият S3 ресурс ще бъде премахнат.

@PreventModification сигнализира, че веднъж създаден, ресурсът не подлежи на промяна – всяка разлика в конфигурацията води до съобщение за грешка, но не и до реална промяна. Това е критично важно при ресурси като S3, чието име е уникално и непроменимо след създаване.

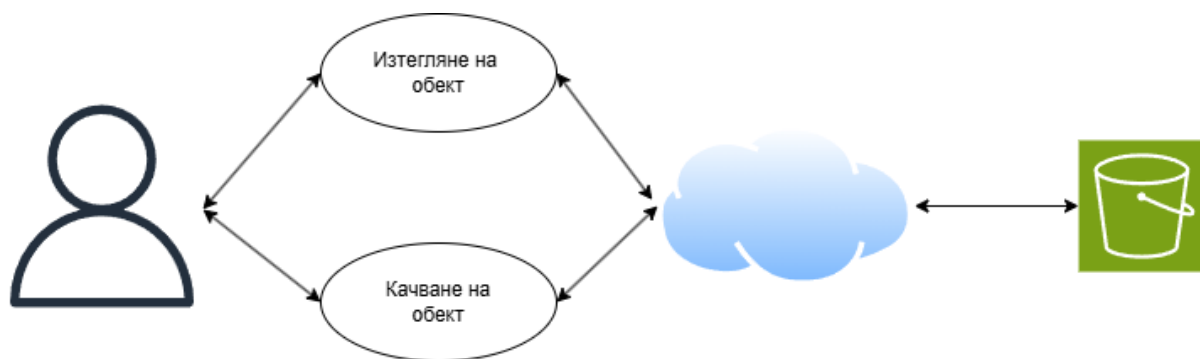


Фиг. 19 Последователност на стъпките при конфигуриране на S3 ресурс

Тази архитектура от стъпки позволява всяка операция да бъде независимо конфигурирана, сравнявана и изпълнявана, като същевременно се гарантира спазване на бизнес ограниченията и последователността на AWS API.

4.7.1 Основен ресурс

Amazon S3 (Simple Storage Service) е мащабируема и надеждна услуга за съхранение на обекти, предлагана от AWS. Чрез нея могат да се съхраняват всякакви типове данни – от изображения и видеа до архиви и машинно-генерирани логове. Данните се организират в „кофи“ (buckets), които представляват логически контейнери за файлове. Всяка кофа има уникално име в глобалния пространствен контекст на AWS и служи като основна единица за администриране, конфигурация и контрол на достъпа. [11]



Фиг. 20 Процес на качване и изтегляне на обекти в AWS S3

В контекста на инфраструктурната библиотека, S3BucketStep представлява основната стъпка за създаване и управление на S3, маркирана с анотациите @MainResource и @PreventModification. Тя гарантира, че S3 се създава веднъж и не може да бъде модифицирана впоследствие – всякакви опити за промяна ще генерират грешка.

```

@Override
public StepResult<S3Output> apply() {
    ...
    s3.create(bucketName);
    log.info("Successfully created bucket '{}'", bucketName);
    s3.waitForBucketExists(bucketName, Duration.ofMinutes(5));
    ...
}
  
```

Фрагмент 54.

Методът apply е отговорен за създаването на S3 в AWS. Първо извлича името от конфигурацията и проверява дали е валидно. След това използва S3SafeClient за създаване на ресурса. Изчаква се потвърждение, че ресурсът е напълно създаден (waitForBucketExists). Резултатът от стъпката съдържа името на S3 и региона – те се използват в следващи фази на сравнение и проверка.

```

@Override
public StepResult<S3Output> destroy(boolean enforcePreventDestroy) {
    if (enforcePreventDestroy && preventDestroy) {
        String msg = "Destruction of bucket '%s' is prevented by configuration.".formatted(bucket);
        log.warn(msg);
        throw new CloudResourceStepException(msg);
    }
    s3.delete(bucket);
}
  
```

Фрагмент 55.

Методът `destroy` се използва при премахване на ресурса. Извлича се `preventDestroy` от мета-данните, за да се провери дали ресурсът е защитен от изтриване. Ако унищожението е забранено (`preventDestroy = true`), се генерира изключение и операцията се прекратява. В противен случай се използва `S3SafeClient` за комуникация с AWS API изтриване на S3.

Методът `revert` в тази стъпка няма функционално действие, тъй като S3 не могат да бъдат модифицирани и съответно не се очакват различия и опити за възстановяване след модификация. В логовете се отбелязва, че `revert` не е поддържан. Връща се структура, която съдържа само идентификаторите, без реални промени.

```
@Override
public StepResult<S3Output> getCurrentState() {
    s3.head(bucket);
}
```

Фрагмент 56.

Този метод проверява дали S3 съществува в AWS в момента на изпълнение. Извършва се HEAD заявка към S3 чрез `S3SafeClient`. Ако ресурсът съществува, се връща състояние с име и регион. Ако не съществува (`NoSuchBucketException`), се връща празен резултат, който сигнализира нужда от създаване.

`S3BucketStep` е най-важната и задължителна стъпка при работа с S3 ресурси. Тя определя дали S3 съществува, дали трябва да бъде създадена или изтрита, и дали всяка следваща стъпка (`versioning`, `tagging`, `policy` и др.) ще има на какво да се приложи. Нейната неизменност е в основата на коректното сравнение и управление на инфраструктурата през времето.

4.7.2 Версионизиране на обекти

Версионизирането в Amazon S3 е функция, която позволява съхранение на няколко версии на един и същ обект в рамките на един S3 ресурс. Това осигурява защита срещу неволно изтриване и презаписване на данни. Всяка промяна върху файл не заменя предишната версия, а създава нова – по този начин може да се възстанови стара версия при нужда. [12]

Версионизирането може да бъде в едно от две състояния:

- **ENABLED** – включено; всички обекти имат версии

- **SUSPENDED** – изключено, но историята на предишни версии се запазва

```
@Override
public StepResult<S3Output> apply() {
    s3.putVersioning(bucket, status);
    ...
    GetBucketVersioningResponse response = s3.getVersioning(bucket);
    ...
    return S3VersioningResultBuilder.fromResponse(response);
}
```

Фрагмент 57.

Този метод прилага настройката, посочена в конфигурационния файл, към съответния S3: Избира между **ENABLED** и **SUSPENDED**, като преобразува текстовата стойност към **BucketVersioningStatus**. Извиква **putVersioning** върху S3 клиента, за да зададе състоянието. След това валидира операцията чрез **getVersioning**. Резултатът от стъпката съдържа текущия статус и се използва за бъдещо сравнение.

```
@Override
public StepResult<S3Output> getCurrentState() {
    ...
    GetBucketVersioningResponse response = s3.getVersioning(bucket);
    return S3VersioningResultBuilder.fromResponse(response);
    ...
}
```

Фрагмент 58.

Методът извлича текущото състояние на версионизиране от AWS, използвайки **getVersioning()**. Ако S3 не съществува (404 Not Found), връща празен **StepResult**. В останалите случаи връща актуалния статус (**ENABLED** или **SUSPENDED**), който ще бъде използван при сравнение със **desiredState**.

```
@Override
public StepResult<S3Output> destroy(boolean enforcePreventDestroy) {
    ...
    s3.putVersioning(bucket, SUSPENDED);
    ...
}
```

Фрагмент 59.

При изтриване на ресурса или подготовка за такова, се връща настройката за версионизиране към стойност по подразбиране – **SUSPENDED**.

```

@Override
public StepResult<S3Output> revert(StepResult<S3Output> previous) throws
CloudResourceStepException {
    ...
    BucketVersioningStatus status =
ENABLED.toString().equalsIgnoreCase(previousStatus) ? ENABLED : SUSPENDED;
    s3.putVersioning(bucket, status);
    ...
}

```

Фрагмент 60.

Ако се налага връщане назад, методът използва предишно съхраненото състояние от `StepResult`, за да възстанови точната стойност на версионизирането. Това осигурява транзакционна консистентност, като връща инфраструктурата в същото състояние, в което е била преди неуспешната промяна.

`S3VersioningStep` предоставя важна функционалност за контрол на версиите на данни в S3. Тя е втората по ред в процеса на прилагане на инфраструктурните промени, след създаването на главния ресурс, и се грижи за прилагането, проверката, изтриването и възстановяването на конфигурацията по версионизиране. Тази стъпка значително допринася за устойчивостта и възстановимостта на данни в облака.

4.7.3 Криптиране на съхранени данни

Криптирането на данните в Amazon S3 е от съществено значение за сигурността на съхраняваната информация. То гарантира, че данните, съхранени в S3, са четими само от оторизирани потребители, дори и ако физическият достъп до тях бъде компрометиран.

S3 поддържа криптиране от страна на сървъра (SSE – Server-Side Encryption), при което AWS автоматично шифрова обектите при запис и ги дешифрова при достъп. Има няколко основни режима на криптиране: [13]

- SSE-S3 (AES-256) – Amazon управлява всички ключове. Най-лесният за настройка метод
- SSE-KMS (AWS_KMS / AWS_KMS_DSSE) – използва AWS Key Management Service (KMS), който предоставя по-добър контрол, възможност за ротация на ключове, логване на достъп чрез CloudTrail и интеграция с политики.
- SSE-C – клиентът предоставя собствени ключове

```

@Override
public StepResult<S3Output> apply() {
    ...
    ServerSideEncryption sseAlgorithm = ServerSideEncryption.AES256;
    switch (encryption.getType()) {
        case AWS_KMS, AWS_KMS_DSSE -> sseAlgorithm =
ServerSideEncryption.AWS_KMS;
    }
    ...
    s3.putEncryption(bucket, sseAlgorithm, encryption.getKmsKeyId(),
encryption.isBucketKeyEnabled());
    ...
}

```

Фрагмент 61.

Методът извлича конфигурацията от YAML файла и определя вида криптиране, което ще бъде приложено: По подразбиране се използва SSE-S3 (AES256). Ако е посочено AWS_KMS или AWS_KMS_DSSE, се използва KMS-базираното криптиране. Извиква се putEncryption върху клиента, като се подава и KMS ключът, ако е наличен. След това се извиква getEncryption за валидация и резултатът се пакетира в StepResult.

```

@Override
public StepResult<S3Output> getCurrentState() {
    ...
    GetBucketEncryptionResponse response = s3.getEncryption(bucket);
    ...
}

```

Фрагмент 62.

Методът извлича текущото криптиране от S3, като използва getEncryption. Ако S3 няма зададено криптиране, методът връща празен StepResult. В останалите случаи резултатът съдържа информация за алгоритъма, KMS ключа и активирана ли е bucketKey функцията.

```

@Override
public StepResult<S3Output> destroy(boolean enforcePreventDestroy) {
    ...
    s3.putEncryption(bucket, ServerSideEncryption.AES256, null, false);
    ...
}

```

Фрагмент 63.

При унищожаване на ресурс криптирането се нулира до стандартната стойност SSE-S3 (AES256): Това е най-неутралната и безрискова стойност, тъй като не изисква KMS ключове и не оставя криптирани данни с неуправляем ключ.

```
@Override
public StepResult<S3Output> revert(StepResult<S3Output> previous) {
    ...
    ServerSideEncryption sseAlgorithm = ...
    String kmsKeyId = ...
    Boolean bucketKeyEnabled = ...
    s3.putEncryption(bucket, sseAlgorithm, kmsKeyId, bucketKeyEnabled);
    ...
}
```

Фрагмент 64.

Ако се наложи връщане към предходно състояние, методът използва информацията, запазена в предишния StepResult: Възстановява типа криптиране – SSE-S3 или SSE-KMS. Възстановява KMS ключа и стойността на bucketKeyEnabled. След това валидира конфигурацията чрез повторно извличане на настройките от AWS.

S3EncryptionStep добавя важен слой сигурност към инфраструктурата чрез контрол върху криптирането на данни. Поддържането на точна и сигурна конфигурация тук е критично, особено за чувствителни приложения. Чрез възможност за прилагане, проверка, изчистване и възстановяване, тази стъпка осигурява пълна криптографска проследимост и контрол върху данните в S3.

4.7.4 Прилагане на маркери

Таговете (tags) в Amazon S3 са двойки от ключ и стойност (key-value), които могат да се прикачат към ресурси като S3. [14] Те не влияят пряко върху функционалността на ресурса, но са изключително важни за:

- Идентификация и групиране на ресурси по среди (напр. env=prod), екипи (team=data) или проекти.
- Проследяване на разходи – AWS Cost Explorer може да групира разходите по тагове.
- Управление на достъп – IAM политики могат да използват условия базирани на тагове.
- Автоматизация – много скриптове, инструменти и облачни рамки използват тагове за автоматизирани действия.

S3TaggingStep е отговорна за прилагането, валидирането, премахването и възстановяването на тези тагове.

```
@Override
public StepResult<S3Output> apply() {
    ...
    List<Tag> tagList = tags.entrySet().stream()
        .map(e -> Tag.builder().key(e.getKey()).value(e.getValue()).build())
        .collect(Collectors.toList());

    s3.putTags(bucket, tagList);
    ...
    GetBucketTaggingResponse response = s3.getTags(bucket);
    return S3TaggingResultBuilder.fromResponse(response);
}
```

Фрагмент 65.

Методът започва с извличане на желаните тагове от конфигурацията. Ако няма тагове – връща празен резултат. Преобразува ги в AWS обекти от тип Tag и ги прилага към S3 с putTags. След това се валидара, че таговете са приложени правилно чрез getTags. Резултатът се пакетира чрез S3TaggingResultBuilder.

```
@Override
public StepResult<S3Output> getCurrentState() {
    ...
    GetBucketTaggingResponse response = s3.getTags(bucket);
    return S3TaggingResultBuilder.fromResponse(response);
}
```

Фрагмент 66.

Извиква се getTags за извличане на актуалните тагове от S3. Ако няма тагове, връща се празен резултат. Ако S3 ресурсът не съществува (404 Not Found), методът също връща празен StepResult.

```
@Override
public StepResult<S3Output> destroy(boolean enforcePreventDestroy) {
    ...
    s3.deleteTags(bucket);
    ...
}
```

Фрагмент 67.

Методът изчиства всички тагове чрез `deleteTags`. Това е необходимо при пълно премахване на ресурса или когато политиката за промяна го изисква.

```
@Override
public StepResult<S3Output> revert(StepResult<S3Output> step) {
    ...
    if (revert == null || revert.getTags().isEmpty()) {
        s3.deleteTags(bucket);
    } else {
        List<Tag> tagList = revert.getTags().entrySet().stream()
            .map(e ->
                Tag.builder().key(e.getKey()).value(e.getValue()).build())
            .collect(Collectors.toList());

        s3.putTags(bucket, tagList);
    }
}
```

Фрагмент 68. Възстановяване на предишни тагове

Ако предишната конфигурация не е съдържала тагове – всички тагове се изтриват. В противен случай – се прилагат таговете от предходното състояние, което гарантира пълно възстановяване.

Тази стъпка осигурява декларативен контрол върху метаданните на S3 ресурсите чрез тагове. Това не само подпомага управлението и одита, но също така гарантира съответствие със стандартите за сигурност и разходи в облачна среда. `S3TaggingStep` поддържа пълна симетрия между прилагане, проверка, изтриване и възстановяване на тагове, което е от критично значение за автоматизирана инфраструктура като код.

4.7.5 Политика за достъп

Политиките за достъп (S3 Bucket Policies) представляват JSON-документи, чрез които се дефинира кой има достъп до даден S3 ресурс, какъв тип достъп (четене, писане и т.н.) и при какви условия. Те са мощен инструмент за централизирано управление на достъпа до обекти в S3 и често се използват за:

- Даване на публичен достъп до статично уеб съдържание.
- Ограничаване на достъпа до конкретни IAM потребители, роли или акаунти.
- Дефиниране на условия за достъп (напр. достъп само от определени IP адреси или VPC-и).

- Прилагане на политики за GDPR, логинг и регулации.

S3PolicyStep автоматизира процеса по прилагане, проверка, премахване и възстановяване на тези политики.

```
@Override
public StepResult<S3Output> apply() {
    String policy = config.getPolicy();

    if (policy.isBlank()) {
        return buildPolicyStepResult(policy);
    }

    String bucket = (String) metadata.getOutputs().get(S3Output.NAME);
    s3.putPolicy(bucket, policy);
    ...
    GetBucketPolicyResponse response = s3.getPolicy(bucket);
    return S3PolicyResultBuilder.fromResponse(response);
}
```

Фрагмент 69. Прилагане на политика

Извлича политиката като текст от конфигурацията. Ако няма дефинирана политика, връща празен резултат. Прилага политиката към S3 с putPolicy. След това валидира прилагането чрез getPolicy и връща резултат.

```
@Override
public StepResult<S3Output> getCurrentState() {
    GetBucketPolicyResponse response = s3.getPolicy(bucket);
    return S3PolicyResultBuilder.fromResponse(response);
}
```

Фрагмент 70. Извличане на текущата политика

Методът се използва за получаване на актуалната политика на S3. Ако S3 няма политика или тя не съществува (404 Not Found), връща се празен StepResult.

```
@Override
public StepResult<S3Output> destroy(boolean enforcePreventDestroy) {
    s3.deletePolicy(bucket);
    log.info("Deleted policy from bucket '{}'", bucket);
    return StepResult.<S3Output>builder().stepName(...).build();
}
```

Фрагмент 71. Премахване на политика

Използва се за пълно премахване на политиката от S3. Подходящо при премахване на ресурса или изчистване на неправомерни правила.

```
@Override
public StepResult<S3Output> revert(StepResult<S3Output> previous) {
    String revert = (String) previous.getOutputs().get(S3Output.VALUE_NODE);

    if (revert == null || revert.isEmpty()) {
        s3.deletePolicy(bucket);
    } else {
        s3.putPolicy(bucket, revert);
    }

    return S3PolicyResultBuilder.fromResponse(response);
}
```

Фрагмент 72. Възстановяване на предишната политика

Ако няма предишна политика – новата се изтрива. Ако има – старата политика се прилага отново. Това поведение гарантира, че нежелани промени могат да бъдат напълно възстановени при грешка или откат.

Политиките за достъп са критична част от сигурността и контрола в S3. Чрез S3PolicyStep библиотеката осигурява декларативен, прозрачен и проследим начин за прилагане на политики. Методите в тази стъпка осигуряват пълна жизнена поддръжка на политиките – от прилагане и валидиране до изтриване и възстановяване. Това позволява управление на достъпа до S3 по автоматизиран, сигурен и одитиран начин.

4.7.6 Контрол на собствеността

Контролът на собствеността (Ownership Controls) в Amazon S3 е механизъм, който определя кой е собственикът на обектите, качени в S3, и какви права има този собственик. Това е особено важно в среди, където различни AWS акаунти или IAM идентичности качват файлове в едно и също S3 пространство, например при споделени ресурси или автоматизирани потоци.

Съществуват три основни режима на собственост в S3:

1. BucketOwnerEnforced (Принудително собственик на S3 ресурса)
2. Това е препоръчителният и най-сигурен режим. Всички обекти, качени в S3, автоматично се притежават от собственика на S3 ресурса, независимо кой ги е качил. AWS автоматично премахва нуждата от ACL-и.
3. BucketOwnerPreferred (Предпочитан собственик на пространството)

4. Собствеността по подразбиране се дава на собственика на S3, освен ако качващата страна изрично не зададе ACL (лист за контрол на достъпа).
5. ObjectWriter (Качващият на обекта е собственик)
6. Качващият обекта запазва собствеността, дори ако не е собственик на S3 пространството. Това е старо поведение, което може да доведе до проблеми с достъпа.

```
@Override
public StepResult<S3Output> apply() {
    S3OwnershipType ownership = config.getOwnershipControls();
    ...
    s3.putOwnershipControls(bucket, ownership.toSdkType());
    ...
    return S3OwnershipControlsResultBuilder.fromResponse(response);
}
```

Фрагмент 73. Прилагане на Ownership Control

Извлича предпочитания режим от конфигурацията (BucketOwnerEnforced, ObjectWriter и др.). Прилага този режим към S3 чрез putOwnershipControls. Потвърждава успешно прилагане чрез getOwnershipControls.

```
@Override
public StepResult<S3Output> getCurrentState() {
    ...
    GetBucketOwnershipControlsResponse response =
    s3.getOwnershipControls(bucket);
    ...
    return S3OwnershipControlsResultBuilder.fromResponse(response);
}
```

Фрагмент 74. Извличане на текущ контрол на собствеността

Получава активните настройки за собственост в S3. Ако не са открити настройки (напр. 404 грешка), връща празен StepResult.

```
@Override
public StepResult<S3Output> destroy(boolean enforcePreventDestroy) {
    ...
    s3.putOwnershipControls(bucket, ObjectOwnership.BUCKET_OWNER_ENFORCED);
    ...
    return S3OwnershipControlsResultBuilder.fromResponse(response);
}
```

Фрагмент 75. Нулиране до подразбиране

Рестартира собствеността към сигурното BucketOwnerEnforced, за да се избегнат бъдещи конфликти с ACL-и или достъп.

```
@Override
public StepResult<S3Output> revert(StepResult<S3Output> previous) {
    ...
    S3OwnershipType type = S3OwnershipType.fromValue(typeStr);
    s3.putOwnershipControls(bucket, type.toSdkType());
    ...
    return S3OwnershipControlsResultBuilder.fromResponse(response);
}
```

Фрагмент 76. Възстановяване на предходна конфигурация

Извлича предишния тип от резултатите и го прилага отново. Така се възстановява предишното състояние на S3 при нужда от откат или при неуспешна операция.

Ownership Controls са ключова настройка за управлението на достъпа и собствеността в S3, особено при съвместна работа между акаунти. Използването на BucketOwnerEnforced в комбинация с изключени ACL-и е сигурна и препоръчителна практика. Чрез S3OwnershipControlsStep, библиотеката позволява това поведение да бъде конфигурирано, приложено и възстановено по един декларативен и автоматизиран начин.

4.7.7 Списъци за контрол на достъп

ACLs (Access Control Lists) са стар, но все още поддържан механизъм за управление на достъпа до ресурси в Amazon S3. Те определят кой може да достъпва пространството или обект и какъв тип достъп му е разрешен. Въпреки че AWS насърчава използването на IAM политики, ACLs понякога са нужни в специфични сценарии (напр. междуакаунтни споделяния или интеграции с по-стари системи).

Основни компоненти на ACL

1. Owner (Собственик) – Обикновено това е AWS акаунтът, който е създал ресурса.
2. Grantee (Получател) – Потребител, група или роля, на която се предоставя достъп.
3. Permission (Разрешение) – Тип достъп: READ, WRITE, READ_ACP, WRITE_ACP, FULL_CONTROL

ACL-ите в S3 могат да бъдат два основни вида:

- Canned ACLs (готови шаблони) – Съкратени декларации, като например:
 - private – достъп само за собственика
 - public-read – публичен достъп само за четене
 - authenticated-read – достъп за всички автентикирани AWS потребители
- Custom ACLs (ръчно дефинирани) – Подробна конфигурация чрез AccessControlPolicy, съдържаща собственик и множество Grant елементи с различни права.

Важно е да се отбележи, че ако S3 пространството е конфигурирано с BucketOwnerEnforced, ACL-ите автоматично се деактивират от AWS. Това поведение е проверявано и зачитано от кода чрез условие, преди да се прилагат ACL настройки.

```
@Override
public StepResult<S3Output> apply() {
    ...
    if
    (S3OwnershipControlsResultBuilder.getOwnershipAsString(s3.getOwnershipControls(bucket))
    .equals(S3OwnershipType.BUCKET_OWNER_ENFORCED.getValue())) {
        log.info("ACL configuration skipped for bucket '{}': ownership is set to {}, which disables ACLs",
            bucket, S3OwnershipType.BUCKET_OWNER_ENFORCED.getValue());
        return S3AclResultBuilder.fromResponse(s3.getAcl(bucket), null);
    }
    if (configAccessControlPolicy != null) {
        AccessControlPolicy policy =
        buildAccessControlPolicy(configAccessControlPolicy);
        s3.putAcl(bucket, policy, null);
        log.info("Successfully applied acl policy '{}' to bucket '{}'", policy,
        bucket);
    }
    if (cannedAcl != null) {
        s3.putAcl(bucket, null, cannedAcl.toSdkAcl());
        log.info("Successfully applied canned acl '{}' to bucket '{}'",
        cannedAcl.getValue(), bucket);
    }
    ...
    GetBucketAclResponse aclResponse = s3.getAcl(bucket);
    return S3AclResultBuilder.fromResponse(aclResponse, cannedAcl);
}
```

Фрагмент 77. Прилагане на ACL

Първо се проверява дали ACL конфигурацията не е деактивирана от BucketOwnerEnforced. Ако конфигурацията е разрешена: Прилага се custom ACL,

ако е предоставен AccessControlPolicy. Прилага се canned ACL, ако е зададен тип (S3AclType). След това се извлича текущото състояние чрез getAcl и се връща резултат.

```
@Override
public StepResult<S3Output> getCurrentState() {
    GetBucketAclResponse response = s3.getAcl(bucket);
    return S3AclResultBuilder.fromResponse(response, null);
}
```

Фрагмент 78 Получаване на текущите ACL настройки

Извлича се текущата ACL конфигурация чрез s3.getAcl(bucket). Резултатът се капсулира в StepResult.

```
@Override
public StepResult<S3Output> destroy(boolean enforcePreventDestroy) {
    if
    (S3OwnershipControlsResultBuilder.getOwnershipAsString(s3.getOwnershipControls(bucket))
        .equals(S3OwnershipType.BUCKET_OWNER_ENFORCED.getValue())) {
        return S3AclResultBuilder.fromResponse(s3.getAcl(bucket), null);
    }
    s3.putAcl(bucket, null, S3AclType.PRIVATE.toSdkAcl());
    GetBucketAclResponse response = s3.getAcl(bucket);
    return S3AclResultBuilder.fromResponse(response, S3AclType.PRIVATE);
}
```

Фрагмент 79. Нулиране на ACL

Ако ACL не са активни (поради BucketOwnerEnforced), просто се връща текущото състояние. В противен случай ACL се нулира до private, което означава пълен достъп само за собственика.

```
@Override
public StepResult<S3Output> revert(StepResult<S3Output> previous) throws
CloudResourceStepException {
    if (cannedAcl != null) {
        s3.putAcl(bucket, null, cannedAcl.toSdkAcl());
        GetBucketAclResponse response = s3.getAcl(bucket);
        return S3AclResultBuilder.fromResponse(response, cannedAcl);
    }
    S3ProvisionedAclOwner ownerDto = revert.getOwner();
    List<S3ProvisionedAclGrant> grantsDto = revert.getGrants();
    List<Grant> grants = grantsDto.stream()
        .map(g -> Grant.builder()
            .grantee(buildSdkGrantee(g))
            .permission(Permission.fromValue(g.getPermission()))
            .build())
        .toList();
    Owner owner = Owner.builder()
```

```

        .id(ownerDto.getId())
        .displayName(ownerDto.getDisplayName())
        .build();
AccessControlPolicy policy = AccessControlPolicy.builder()
    .grants(grants)
    .owner(owner)
    .build();
s3.putAcl(bucket, policy, null);
GetBucketAclResponse aclResponse = s3.getAcl(bucket);
return S3AclResultBuilder.fromResponse(aclResponse, null);
}

```

Фрагмент 80. Възстановяване на предишна ACL конфигурация

Ако предишната конфигурация е била canned ACL – тя се прилага отново. Ако е била custom ACL – създава се обект от тип AccessControlPolicy с предишните grants и owner. Прилага се и след това се верифицира чрез getAcl.

ACL механизмът предоставя гъвкав, но остаряващ начин за контрол на достъпа в S3. Подходящ е за сценарии, в които IAM политиките или bucket policies не са достатъчни. Библиотеката поддържа и двата режима на ACL.

4.8 Стъпки за създаване и модифициране на EKS клъстер

В съвременната софтуерна разработка автоматизацията на инфраструктурата е от ключово значение за бързото и надеждно доставяне на приложения. Една от най-популярните технологии за това е Amazon Elastic Kubernetes Service (EKS) – напълно управлявана услуга за стартиране на Kubernetes клъстери в AWS. Тези клъстери служат като оркестратори на контейнеризирани приложения – те управляват състоянието, мащабирането, мрежовата комуникация и здравето на приложенията и услугите. [15]

EKS клъстерът е логически компонент, който обединява в себе си:

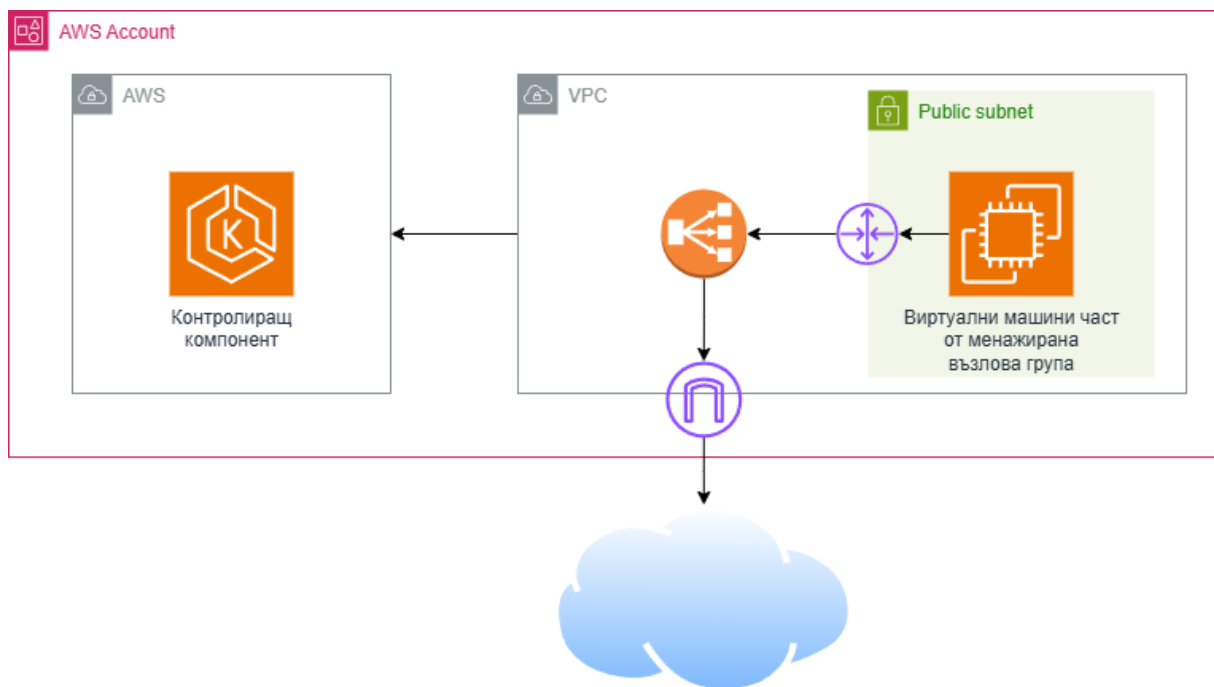
- управляващ control plane (сървърите, които ръководят самия Kubernetes)
- работни виртуални машини (EC2 инстанции), където стартират контейнерите
- мрежови и IAM настройки
- различни Kubernetes приставки, като coredns, kube-proxy, vpc-cni, които поддържат мрежовата и административна функционалност

Създаването на EKS клъстер позволява стартирането на сложни, устойчиви и мащабируеми приложения в облака, с възможност за публичен достъп (например чрез Load Balancer или Ingress) или частна изолация – според случая.

Всеки AWS акаунт автоматично създава по една default VPC (Virtual Private Cloud) мрежа за всеки регион. Това е предварително конфигурирана изолирана мрежова среда с:

- публични и частни подмрежи (subnets)
- рутинг таблици и интернет шлюз (gateway)
- мрежови ACL-и и групи за сигурност (security groups)

В случая на този клъстер ще използваме VPC по подразбиране и нейните публични подмрежи, за да осигурим демонстрационен клъстер, в който стартираните приложения да бъдат публично достъпни. Това не е подходяща конфигурация за продукционни среди, но е достатъчна за цели като демонстрации, интеграционно тестване, обучение и прототипиране.



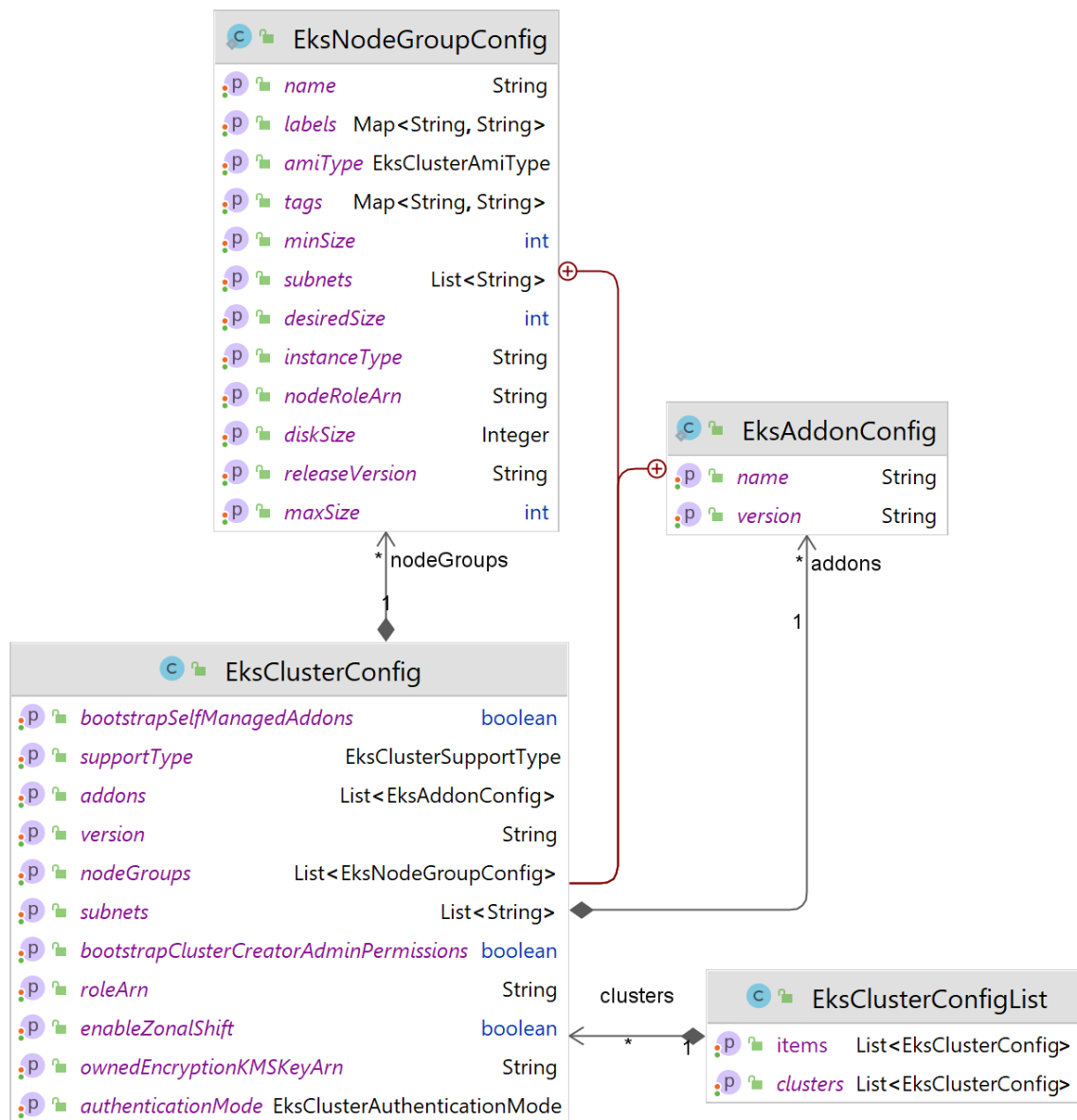
Фиг.21 Архитектура на управляван Kubernetes клъстер в AWS с публична подмрежа

Централният клас в конфигурационната схема е `EksClusterConfig`, който представлява описанието на един конкретен EKS клъстер. Той съдържа основни свойства като версията на Kubernetes, ролята на клъстера, списъка с подмрежи и настройки за автентикация. Освен това позволява да се дефинират свързани компоненти като node group-и и приставки (addons) чрез съответните вложени обекти.

Колекцията от клъстер конфигурации се управлява чрез обвиващия клас `EksClusterConfigList`, който поддържа списък от множество `EksClusterConfig` обекти чрез полето `items`. `EksNodeGroupConfig` описва конфигурацията на всяка `node group` – група от EC2 инстанции, които служат като работни възли в клъстера. Всеки клъстер може да има множество такива групи, което позволява гъвкавост при разпределение на натоварването и ресурсите. `EksAddonConfig` съдържа информация за необходимите разширения (addons), които трябва да бъдат инсталирани в клъстера. Тези разширения са управлявани от AWS и покриват мрежови компоненти, DNS, проксита и други системни услуги.

Всяка част от конфигурацията – дали е самият клъстер, неговите `node group`-и или добавките – се дефинира като обект с конкретни параметри.

Тази архитектура предоставя гъвкав и мащабируем начин за управление на EKS клъстери и улеснява автоматизацията на инфраструктурното изграждане чрез единна и добре структурирана конфигурация.



Фиг. 22 Модел на конфигурацията на EKS клъстери и управлявани групи от машини (възли)

Примерната конфигурация по-долу показва как чрез YAML файл описваме един EKS клъстер и неговата инфраструктура:

```
clusters:
- id: 8982a844-2aed-44c6-a525-d425e48a8fff
  name: dev-tuvarna-eks-app
  region: eu-west-1
  version: "1.32"
  roleArn: "arn:aws:iam::867102643694:role/dev.ekscluster.iamrole"
  tags:
    environment: development
    team: devops
  subnets:
    - subnet-0689621ccf45d992e
    - subnet-0f7fdf8944640fc6d
```

```

bootstrapClusterCreatorAdminPermissions: true
authenticationMode: API_AND_CONFIG_MAP
bootstrapSelfManagedAddons: true
supportType: STANDARD
enableZonalShift: false
addons:
  - name: vpc-cni
    version: v10.19.2-eksbuild.1
  - name: kube-proxy
    version: v1.32.0-eksbuild.2
  - name: coredns
    version: v1.11.4-eksbuild.2
enableReconciliation: false
preventDestroy: false
nodeGroups:
  - name: dev-appcluster-ng
    instanceType: t3.medium
    minSize: 1
    maxSize: 3
    desiredSize: 1
    nodeRoleArn: arn:aws:iam::867102643694:role/dev.nodegroup.iamrole
    subnets:
      - subnet-0689621ccf45d992e
      - subnet-0f7fdf8944640fc6d

```

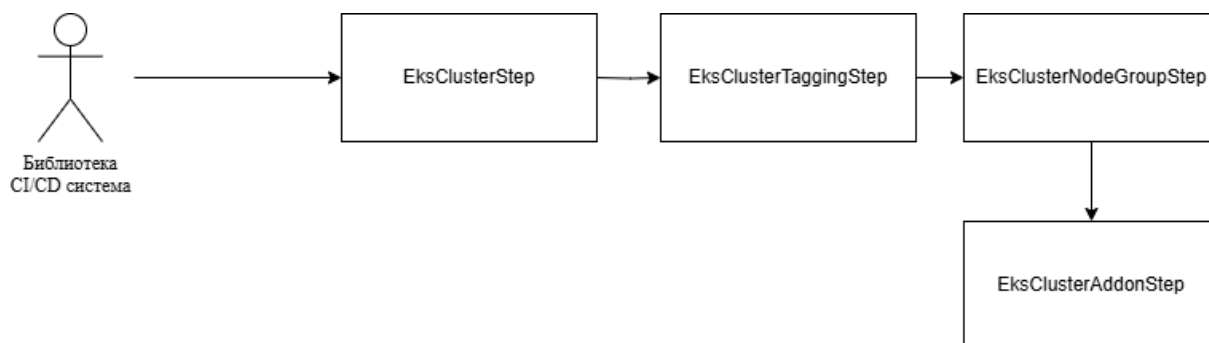
Фрагмент 81. Цялостна декларативна конфигурация на клъстер

Тази декларация позволява автоматизирано създаване на: клъстер с версия 1.32, в регион eu-west-1, с необходимите мрежови и IAM настройки, както и самоуправяема група от възли (nodeGroups), които ще бъдат разположени в публични подмрежи.

EKS клъстерите са фундаментален компонент от съвременния DevOps подход. Те позволяват изграждане на CI/CD работни процеси, автоматизирано мащабиране и самооправяне, изолиране на среди (dev/staging/prod), облачна независимост чрез Kubernetes API.

Създаването и поддържането на такъв клъстер чрез декларативна инфраструктура не само спестява време и грешки, но и гарантира предвидимост и възпроизводимост.

В следващите глави се разглежда подробно всяка стъпка от процеса по създаване, настройка и поддръжка на EKS клъстера.



Фиг. 23 Последователност от стъпки при създаване на EKS клъстер

4.8.1 Основа на клъстър

Създаването на EKS клъстър е фундаменталната първа стъпка при изграждане на среда за стартиране на контейнеризирани приложения в AWS. Тази операция дефинира цялостната рамка, в която впоследствие ще се добавят възли (node групи), мрежови правила, политики и други компоненти. Самият клъстър представлява управляван Kubernetes control plane, който осигурява комуникацията, координацията и контрола върху подовете и услугите в приложението.

```
@Override
public StepResult<EksClusterOutput> apply() {
    eks.create(config);
    eks.waitUntilActive(clusterName, Duration.ofMinutes(10));
    Cluster cluster = eks.describe(clusterName).cluster();
}
```

Фрагмент 82. Метод за създаване на основата на клъстър

Методът `apply` изпълнява реалната заявка за създаване на клъстър с помощта на `eks.create(config)`. В него се използват следните ключови атрибути от конфигурацията.

name	задава уникалното име на клъстъра. Това е референтен идентификатор, чрез който се управлява клъстърът в бъдеще.
version	определя версията на Kubernetes, която ще се използва.
roleArn	ARN идентификатор на IAM роля, която EKS ще използва за управление на AWS ресурси от името на клъстъра
subnets	списък от subnet идентификатори, които описват в кои мрежови подмрежи ще бъдат разположени ресурсите на клъстъра. В нашия случай това са публични subnets от default VPC
authenticationMode	указва дали Kubernetes API ще бъде достъпен само чрез IAM роли (API) или и чрез ConfigMap база от потребители (API AND CONFIG MAP).

bootstrapClusterCreatorAdminPermissions	когато е true, първият потребител, създал клъстър, получава пълни административни права в Kubernetes
supportType	дефинира типа поддръжка от AWS: STANDARD, DEVELOPER, ENTERPRISE
enableZonalShift	активира или деактивира възможността за зоново пренасочване (Zonal shift) на трафик за висока достъпност
ownedKmsKeyId	опционално криптиране с KMS ключ. Когато се зададе, всички Kubernetes secrets в клъстърa ще бъдат криптирани със специфичен ключ

Извличането на текущото състояние на клъстърa се реализира чрез метода `getCurrentState()`, който извиква `eks.describe(clusterName)` и връща същите ключови атрибути, използвани при създаването на ресурса. Това позволява прецизно сравнение между реалното и желаното състояние и е основен механизъм за гарантиране на идемпотентност и консистентност (reconciliation) на инфраструктурата.

```
@Override
public StepResult<EksClusterOutput> getCurrentState() {
    ...
    DescribeClusterResponse response = eks.describe(resource);
    Cluster cluster = response.cluster();
    log.info("Successfully fetched details for EKS cluster '{}'", resource);
    ...
}
```

Фрагмент 83.

При нужда от премахване на ресурса, методът `destroy()` проверява дали не е активирана защита от унищожение чрез флага `preventDestroy` и, ако не е, извиква `eks.delete(clusterName)`, последвано от изчакване на пълното изтриване чрез `waitUntilDeleted`.

```
@Override
public StepResult<EksClusterOutput> destroy(boolean enforcePreventDestroy)
{
    if (enforcePreventDestroy && Boolean.TRUE.equals(preventDestroy)) {
        String msg = "Destruction of EKS cluster '%s' is prevented by
configuration.".formatted(clusterName);
        log.warn(msg);
        throw new CloudResourceStepException(msg);
    }
    eks.delete(clusterName);
    log.info("Deleting EKS cluster '{}'....", clusterName);
}
```

```
eks.waitForDeleted(clusterName, Duration.ofMinutes(10));
}
```

Фрагмент 84.

В случай на неуспешна промяна в конфигурацията, се активира методът `revert()`, който възстановява предходната работеща конфигурация. Това се реализира чрез съставяне на обект `EksClusterConfig` с предишните стойности и актуализиране на клъстъра чрез `eks.updateCluster`. Така се осигурява стабилност и възможност за връщане към предишно валидно състояние при грешки в процеса по модификация.

```
@Override
public StepResult<EksClusterOutput> revert(StepResult<EksClusterOutput>
previous) {
    ...
    EksClusterConfig previousConfig = new EksClusterConfig();
    ...
    if (!this.config.equals(previousConfig)) {
        log.info("Cluster '{}' has changed – reverting configuration...",
clusterName);
        eks.updateCluster(clusterName, previousConfig);
        eks.waitForActive(clusterName, Duration.ofMinutes(10));
    } else {
        log.info("Cluster '{}' is unchanged – skipping update.", clusterName);
        return previous;
    }
}
```

Фрагмент 85.

4.8.2 Група от работни виртуални машини

Node Group (или група от работни виртуални машини) представлява логическа единица в Amazon EKS, която съдържа EC2 инстанции (виртуални машини), използвани за изпълнение на контейнеризирани приложения в рамките на клъстъра. Тези възли са част от управлявана група и автоматично се мащабират, актуализират и наблюдават от AWS. Възловите групи са свързани към конкретен клъстер и разполагат с множество конфигурационни параметри, които определят тяхното поведение и капацитет.

Ключовите свойства при създаване на node group включват:

Таблица 3. Свойства на група от виртуални машини

name	уникално име на node групата
instanceType	видът на EC2 инстанциите, например t3.medium, m5.large, c6g.xlarge и др. Изборът зависи от нуждите на

	приложението – дали то е изчислително интензивно, използва много памет, или има други специфични изисквания
minSize, maxSize, desiredSize	граници за автоматично мащабиране и броят на възли, с които стартира групата
amiType	видът на Amazon Machine Image, която ще се използва за виртуалните машини
subnets	мрежови подсистеми, в които се разполагат възлите
nodeRoleArn	IAM роля, даваща разрешения на EC2 инстанциите в групата
labels, tags	метаинформация, използвана за управление и филтриране на машините

В конкретния случай за node group-a (dev-appcluster-ng) е избрана EC2 инстанция от тип t3.medium. Това е general-purpose тип, което означава, че е балансиран по отношение на изчислителна мощ, памет и мрежова производителност. Подходящ е за разнообразни натоварвания, включително малки и средни контейнеризирани приложения, които не изискват специализирани ресурси.

Основни характеристики на t3.medium:

- vCPU (виртуални процесора): 2
- RAM: 4 GB
- Ценово ефективна: Подходяща за dev/test среди и неинтензивни production приложения.

Групата с автоматично мащабиране винаги ще поддържа поне една EC2 инстанция; може да скалира до максимум 3, ако натоварването го изисква (например при натоварване на CPU) и ще започне с една инстанция, съгласно desiredSize.

В този случай на употреба, групата ще използва Amazon Linux 2, 64-битова версия за x86_64 архитектура (AL2_X86_64).

Основни характеристики:

- Amazon Linux 2 (AL2): Олекотена, оптимизирана ОС за работа в AWS, съвместима с всички основни инструменти и Kubernetes компоненти.
- x86_64 архитектура: Поддържа широк набор от библиотеки и инструменти. Това е стандартната архитектура на Intel/AMD процесори.

- Предпочитана за EKS: AL2 е поддържана и оптимизирана от AWS, включително с предварително инсталиран kubelet и необходимите модули.

Методът `getCurrentState()` извлича текущото състояние на всички групи, асоциирани към клъстера. Това е важно за сравнение между желаната и реалната инфраструктура в процеса на reconciliation. Методът `destroy()` премахва всички групи, дефинирани в метаданните на конфигурацията. Извиква се `eks.deleteNodeGroup` за всяка група и се изчаква те да бъдат напълно премахнати. Методът `revert()` възстановява предходно валидно състояние на групите. Ако някоя група не съществува в предишната конфигурация, тя се изтрива. Ако липсва текущо, но съществува в старата – се създава наново. При различие в конфигурацията, се извършва актуализация. Така се гарантира устойчивост при грешки или прекъсвания в provisioning процеса.

4.8.3 Системни приставки

При създаването на EKS клъстер, последната стъпка от инициализацията е добавянето на така наречените add-ons – системни разширения, които осигуряват базова функционалност за правилната работа на Kubernetes в AWS. Тези приставки се управляват от AWS и могат да бъдат създавани, обновявани или премахвани директно през EKS API, без нужда от ръчно инсталиране чрез `kubectl`.

Най-често използваните и фундаментални приставки са:

- `vpc-cni` – (Amazon VPC Container Network Interface плъгин) отговаря за свързването на контейнерите с мрежата на VPC. Този add-on дава възможност на всеки под да получи IP адрес от VPC и да комуникира директно с други ресурси в AWS. Това е основна част от мрежовия слой на EKS.
- `kube-proxy` – реализира Kubernetes Service абстракцията чрез управление на мрежови правила в машините. Това позволява пренасочване на трафика към правилния pod в клъстера при използване на ClusterIP, NodePort и LoadBalancer услуги.
- `coredns` – отговаря за DNS резолюцията в рамките на клъстера. Когато един pod трябва да се свърже с друг по име (например `my-service.my-namespace.svc.cluster.local`), това се случва чрез CoreDNS, който действа като DNS сървър за Kubernetes.

Тези приставки осигуряват базовата функционалност, необходима за работа на всяко приложение в EKS и са критични за функционирането на Kubernetes.

Методът `apply()` отговаря за създаването или обновяването на приставките. Първо се сравнява текущото състояние на add-ons в EKS клъстър с конфигурацията. Ако дадена приставка не съществува – тя се създава. Ако версията ѝ се различава от желаната – тя се обновява. Накрая се изчаква всички приставки да достигнат състояние „active“.

```
@Override
public StepResult<EksClusterOutput> apply() {
    ...
    EksClusterAddons currentAddons = eks.getAddons(clusterName);
    ...
    for (EksClusterConfig.EksAddonConfig configured : configuredAddons) {
        ...
        if (current == null) {
            eks.createAddon(clusterName, name, version);
            log.info("Created addon '{}' (version '{}') in cluster '{}'", name,
version, clusterName);
        } else if (!Objects.equals(current.getVersion(), version)) {
            eks.updateAddon(clusterName, name, version);
            log.info("Updated addon '{}' to version '{}' in cluster '{}'", name,
version, clusterName);
        } else {
            log.info("Addon '{}' already present with version '{}' – skipping.",
name, version);
        }
    }
    ...
    eks.waitUntilAddonsActive(clusterName, result, Duration.ofMinutes(5));
}
```

Фрагмент 86.

Методът `getCurrentState()` извлича текущото състояние на инсталираните add-ons от AWS, което позволява сравнение с конфигурацията и засичане на разлики в версии или липсващи приставки.

```
@Override
public StepResult<EksClusterOutput> getCurrentState() {
    ...
    EksClusterAddons currentAddons = eks.getAddons(clusterName);
    ...
}
```

Фрагмент 87.

Методът `destroy()` премахва всички приставки от клъстър, които са записани в текущото състояние, след което изчаква тяхното пълно изтриване.

```

@Override
public StepResult<EksClusterOutput> destroy(boolean enforcePreventDestroy)
{
    ...
    EksClusterAddons currentAddons = eks.getAddons(clusterName);
    for (EksClusterAddon addon : currentAddons.getAddons()) {
        eks.removeAddon(clusterName, addon.getName());
    }
    ...
    eks.waitUntilAddonsDeleted(clusterName, currentAddons,
        Duration.ofMinutes(5));
    ...
}

```

Фрагмент 88.

Методът `revert()` служи за възстановяване на предходно състояние при неуспешна операция. Първо премахва всички приставки, които не съществуват в „предишното състояние“, след което възстановява липсващите или обновява тези с различна версия. Това гарантира, че системните компоненти на клъстърта остават в желаня вид, дори при грешки по време на `deployment`.

```

@Override
public StepResult<EksClusterOutput> revert(StepResult<EksClusterOutput>
previous) {
    ...
    EksClusterAddons currentAddons = eks.getAddons(clusterName);
    ...
    EksClusterAddons deleted = new EksClusterAddons();
    for (String addon : currentNames) {
        if (!previousNames.contains(addon)) {
            eks.removeAddon(clusterName, addon);
            ...
        }
    }
    ...
    eks.waitUntilAddonsDeleted(clusterName, deleted, Duration.ofMinutes(5));
    // Add or update addons that are missing or have different versions
    for (EksClusterAddon addon : previousSet) {
        ...
        if (match.isEmpty()) {
            eks.createAddon(clusterName, addon.getName(), addon.getVersion());
            ...
        } else if (!addon.equals(match.get())) {
            eks.updateAddon(clusterName, addon.getName(), addon.getVersion());
            ...
        }
    }
    eks.waitUntilAddonsActive(clusterName, updated, Duration.ofMinutes(5));
    ...
}

```

Фрагмент 89.

Използването на системни разширения в EKS е пример за интегрирана автоматизация, при която управлението на базови Kubernetes компоненти е прехвърлено към AWS, което улеснява поддръжката и подобрява сигурността и съвместимостта с останалата част от AWS екосистемата.

Глава 5. Тестове и резултати

5.1 Автентикация

Първият етап от тестовете е фокусиран върху автентикацията към AWS, тъй като успешната идентификация на потребителя е основа за всяка следваща операция по създаване или модифициране на облачни ресурси. Тестването ще бъде извършено върху конфигурацията, поддържаща три различни механизма за удостоверяване: чрез Vault, чрез статични AWS ключове, и чрез потребителски профил от AWS CLI.

Тестовете за автентикация се изпълняват спрямо следната конфигурация, дефинирана в YAML файл, която описва различни механизми за удостоверяване към AWS:

```
providers:
- vault:
  scheme: http
  host: localhost
  port: 8200
  secretPath: /v1/application/data/cloudprovisioner/867102643694
  token: ${VAULT_TOKEN}
- staticCredentials:
  accessKeyId: ${AWS_ACCESS_KEY_ID}
  secretAccessKey: ${AWS_SECRET_ACCESS_KEY}
- profileCredentials:
  profileName: ${AWS_PROFILE}
```

Фрагмент 90.

Тази конфигурация указва, че приложението трябва да се опита да автентикира потребителя в реда, в който са изброени доставчиците (Vault → Статични крeденшъли → Профил от AWS CLI). Всеки от тези подходи ще бъде тестван индивидуално, като се проверява:

- дали съответният provider е достъпен и правилно конфигуриран
- дали се извличат валидни AWS ключове
- дали може успешно да се извърши базова заявка към AWS (например sts:GetCallerIdentity)

5.1.1 Vault

В този сценарий се очаква приложението да се свърже с локално работещ Vault сървър (localhost:8200) и да извлече AWS идентификационни данни от зададения secretPath. Необходим е валиден VAULT_TOKEN, зададен като променлива на средата. Очакваният резултат от теста е успешна автентикация и извличане на accessKeyId и secretAccessKey от хранилището на Vault. При грешна конфигурация или недостъпен Vault, трябва да се хвърли AuthenticationException изключение.

Таблица 4. Тест с успешна автентикация към Vault.

Вход	Очакван резултат	Изход
<pre>providers: - vault: scheme: http host: localhost port: 8200 secretPath: /v1/application/data/cloudprovisioner/867102643694 token: \${VAULT_TOKEN}</pre>	Успешна автентикация към Vault.	<pre>Authenticating using Vault (host=localhost, port=8200, secretPath=/v1/application /data/cloudprovisioner/867 102643694) AWS credentials successfully fetched from Vault. --- data: AWS_ACCESS_KEY_ID : '****' AWS_SECRET_ACCESS _KEY: '****' Authenticated AWS identity: Account = 867102643694, ARN = arn:aws:iam::8671026436 94:user/yavorchamov, UserId = AIDA4TY2YKXXCMM OR3K4F</pre>

Таблица 5. Тест с провалена автентикация към Vault при сбъркан secretPath.

Вход	Очакван резултат	Изход
<pre> providers: - vault: scheme: http host: localhost port: 8200 secretPath: /v1/application/data/cloudprovisioner/error token: \${VAULT_TOKEN} </pre>	Провалена автентикация към Vault при сбъркан secretPath	Authenticating using Vault (host=localhost, port=8200, secretPath=/v1/application/data/cloudprovisioner/error) Authentication attempt failed using available configuration: Failed to retrieve secrets from Vault at URI: http://localhost:8200/v1/application/data/cloudprovisioner/86710264369. Check connectivity, Vault availability, or authentication token.

5.1.2 Статични AWS ключове с променливи на средата

Този механизъм използва променливи на средата AWS_ACCESS_KEY_ID и AWS_SECRET_ACCESS_KEY, които се подават директно чрез конфигурацията. Това е най-простият и директен начин за автентикация. Очакванията при тестване включват успешно извличане на ключовете от средата и последваща автентикация чрез AWS SDK. При липсващи или невалидни стойности, се очаква изключение от типа AuthenticationException.

Таблица 6. Тест с успешна автентикация с променливи на средата включващи AWS статични ключове.

Вход	Очакван резултат	Изход
<pre> providers: - staticCredentials: accessKeyId: \${AWS_ACCESS_KEY_ID} secretAccessKey: \${AWS_SECRET_ACCESS_KEY} </pre>	Успешна автентикация с променливи на средата включващи AWS статични ключове.	Authenticating using static credentials Authenticated AWS identity: Account = 867102643694, ARN = arn:aws:iam::867102643694:user/yavorchamov, UserId = AIDA4TY2YKXXCMMOR3K4F

Таблица 7. Тест с провалена автентикация с променливи на средата включващи AWS статични ключове с грешни стойности.

Вход	Очакван резултат	Изход
<pre>providers: - staticCredentials: accessKeyId: \${AWS_ACCESS_KEY_ID} secretAccessKey: \${AWS_SECRET_ACCESS_KEY} }</pre>	Провалена автентикация с променливи на средата включващи AWS статични ключове с грешни стойности	Authenticating using static credentials Failed to verify AWS credentials using STS: The security token included in the request is invalid. (Service: Sts, Status Code: 403, Request ID: af14f1f6-4e52-4fbf-9118-cb194d0bdba7) (SDK Attempt Count: 1)

5.1.3 Запазен профил от AWS CLI

Тук се използва конфигурационен профил, дефиниран в локалния файл `~/.aws/credentials`. Променливата `${AWS_PROFILE}` указва кой профил да се използва. Тестовите следва да проверят, че профилът е наличен, че съдържа валидни ключове, и че е възможно с тях да се осъществи автентикация. При липсващ профил или грешни данни, се очаква `SdkClientException`.

Таблица 8. Тест с успешна автентикация с конфигурационен профил

Вход	Очакван резултат	Изход
<pre>providers: - profileCredentials: profileName: \${AWS_PROFILE}</pre>	Успешна автентикация с конфигурационен профил	Authenticating using AWS profile ,867102643694' Authenticated AWS identity: Account = 867102643694, ARN = arn:aws:iam::867102643694:user/yavorchamov, UserId = AIDA4TY2YKXXCMMOR3K4F
<pre>providers: - profileCredentials: profileName: \${AWS_PROFILE}</pre>	Провалена автентикация с конфигурационен профил при липсващ такъв	Profile file contained no credentials for profile ,867102643694'

5.2 Услугата Amazon S3

5.2.1 Създаване на нов ресурс

Подглавата засяга тестовете, свързани с услугата Amazon S3, и по-конкретно — към валидирането на създаването и конфигурацията на S3 пространство, описано в предоставения YAML файл.

```
buckets:
- id: 11cf0cf4-76d3-4615-aecc-21497b01ecc9
  name: tuvarna-dev-eu-central-1-867102643694
  region: eu-central-1
  preventDestroy: false
  tags:
    environment: dev
    team: storage
  versioning: Enabled
  encryption:
    type: aes256
    bucketKeyEnabled: true
  ownershipControls: BucketOwnerEnforced
  acl:
    canned: private
  policy: |
    {
      "Version": "2012-10-17",
      "Statement": [
        {
          "Sid": "AllowFullAccessToSpecificUser",
          "Effect": "Allow",
          "Principal": {
            "AWS": "arn:aws:iam::867102643694:root"
          },
          "Action": "s3:*",
          "Resource": [
            "arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694",
            "arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694/*"
          ]
        }
      ]
    }
  }
```

Фрагмент 91. Тестова конфигурация

Избраният сценарий има за цел да провери дали системата правилно прилага желаните свойства и дали състоянието на създаденото пространство в AWS съответства на конфигурацията.

Конкретно, S3 с име tuvarna-dev-eu-central-1-867102643694, разположен в регион eu-central-1, е маркиран с тагове environment: dev и team: storage. В тестовете се

очаква тези метаданни да се приложат успешно, както и да бъдат правилно извлечени и отчетени при проверка на текущото състояние.

Също така, S3 има включено версионизиране, което предполага, че всяка модификация на обект в S3 създава нова версия. Това поведение ще бъде валидирано чрез добавяне и презаписване на файлове, последвано от проверка за наличието на множество версии.

Зададена е сървърна страна на криптиране (Server-Side Encryption) от тип AES256, което указва, че AWS ще криптира всички обекти, съхранявани в кофата, автоматично. Също така, `bucketKeyEnabled: true` предполага използване на по-ефективен механизъм за KMS ключове при голям обем данни. Очакването е криптиращата конфигурация да бъде успешно приложена и върната обратно.

Допълнително, S3 използва контроли на собственост със стойност `BucketOwnerEnforced`, което изключва използването на ACL-и и гарантира, че обектите в него принадлежат на собственика на кофата. Това ще бъде тествано чрез проверка за отсъствието на ACL разрешения и потвърждение на собствеността.

За контрол на достъпа е зададен ACL от тип `private` (по подразбиране), което указва, че само собственикът на S3 има достъп до обектите в него. Очаква се този `canned ACL` да бъде приложен и потвърден чрез извличане на разрешенията.

Накрая, конфигурацията включва персонализирана IAM политика, която предоставя пълен достъп (`s3:*`) на конкретен `root` потребител от същия AWS акаунт. В рамките на тестовете ще се валидира дали тази политика е успешно приложена към S3 и дали съответства на зададения JSON.

Целта на тестовете е не само да потвърдят успешното създаване на кофата, но и да валидират всяка една от зададените конфигурационни стъпки чрез сравнение между желаното и реалното състояние, като се гарантира идемпотентност и пълно съответствие с инфраструктурната декларация.

Таблица 9. Тест с успешно създаване на нов S3 ресурс

Изход
Bucket 'tuvarna-dev-eu-central-1-867102643694' is not yet provisioned. Will be created from scratch
Changes in step 'bg.tuvarna.sit.cloud.core.aws.s3.step.S3TaggingStep': ↪ value: 'null' → '{environment=dev, team=storage}'
Changes in step 'bg.tuvarna.sit.cloud.core.aws.s3.step.S3VersioningStep': ↪ status: 'null' → 'Enabled'
Changes in step 'bg.tuvarna.sit.cloud.core.aws.s3.step.S3BucketStep': ↪ name: 'null' → 'tuvarna-dev-eu-central-1-867102643694'

Changes in step 'bg.tuvarna.sit.cloud.core.aws.s3.step.S3PolicyStep': ↪ value: 'null' →
 '{"Version":"2012-10-17","Statement":[{"Sid":"AllowFullAccessToSpecificUser","Effect":"Allow","Principal":{"AWS":"arn:aws:iam::867102643694:root"},"Action":"s3:*","Resource":["arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694","arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694/*"]}]}'

Changes in step 'bg.tuvarna.sit.cloud.core.aws.s3.step.S3AclStep': ↪ value: 'null' →
 '{owner=null, grants=null, cannedAcl=private}'

Changes in step 'bg.tuvarna.sit.cloud.core.aws.s3.step.S3OwnershipControlsStep': ↪
 type: 'null' → 'BucketOwnerEnforced'

Changes in step 'bg.tuvarna.sit.cloud.core.aws.s3.step.S3EncryptionStep':
 ↪ type: 'null' → 'AES256'
 ↪ bucketKeyEnabled: 'null' → 'true'

Successfully created bucket 'tuvarna-dev-eu-central-1-867102643694'
 S3 bucket 'tuvarna-dev-eu-central-1-867102643694' is now accessible.
 Successfully applied versioning status 'Enabled' to bucket 'tuvarna-dev-eu-central-1-867102643694'
 Successfully verified versioning status 'Enabled' for bucket 'tuvarna-dev-eu-central-1-867102643694'
 Successfully applied server-side encryption 'AES256' to bucket 'tuvarna-dev-eu-central-1-867102643694'
 Successfully verified server-side encryption 'AES256' for bucket 'tuvarna-dev-eu-central-1-867102643694'
 Successfully applied tags '[Tag(Key=environment, Value=dev), Tag(Key=team, Value=storage)]' to bucket 'tuvarna-dev-eu-central-1-867102643694'
 Successfully applied policy '{"Version":"2012-10-17","Statement":[{"Sid":"AllowFullAccessToSpecificUser","Effect":"Allow","Principal":{"AWS":"arn:aws:iam::867102643694:root"},"Action":"s3:*","Resource":["arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694","arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694/*"]}]}' to bucket 'tuvarna-dev-eu-central-1-867102643694'
 Successfully applied ownership controls 'BucketOwnerEnforced' to bucket 'tuvarna-dev-eu-central-1-867102643694'
 ACL configuration skipped for bucket 'tuvarna-dev-eu-central-1-867102643694':
 ownership is set to BucketOwnerEnforced, which disables ACLs

5.2.2 Модификация на вече съществуващ ресурс

В следващия тестов сценарий ще преминем към модификация на вече съществуващ S3 ресурс, като целта е да се валидира дали системата правилно открива разлика между текущото и желаното състояние и прилага необходимите промени върху S3.

Първата промяна е добавянето на нов етикет (tag) с ключ organisation и стойност tuvarna. Очакването е, че след повторното изпълнение на конфигурацията, този tag

ще бъде добавен към вече съществуващия набор от тагове, без да се наруши целостта на останалите.

Следващата промяна е изключване на versioning функционалността, чрез задаване на versioning: Disabled. Това означава, че S3 вече няма да поддържа версии на обекти. При изпълнение на теста ще бъде извикан съответният API за промяна на настройката, а валидирането ще включва извличане на текущото състояние и потвърждение, че versioning е изключен.

Също така се променя поведението на контрола на собствеността, като вместо BucketOwnerEnforced, се задава стойността BucketOwnerPreferred. Това позволява използване на ACL списъци в определени случаи. След изпълнение на теста се очаква кофата да отразява тази настройка чрез getOwnershipControls, което ще потвърди успешната модификация.

ACL политиката остава private, но тъй като BucketOwnerPreferred допуска използване на ACL-и, се очаква системата да приложи този canned ACL и той да бъде видим в резултатите от getAcl.

Накрая, в IAM политиката се въвежда промяна на разрешените действия, като вместо пълен достъп (s3:*), новата политика дефинира специфични действия: "s3:GetObject", "s3:GetBucketLocation" и "s3:ListBucket". Това ограничава потребителя само до четящи операции. Системата следва да актуализира вече съществуващата политика чрез putBucketPolicy, а тестът ще провери дали новата политика замества предишната и е успешно приложена.

Този тест е ключов за валидиране на модификации върху живи ресурси, като демонстрира способността на системата да открива разлики и да извършва точно и безопасно обновление на конфигурацията в съответствие с принципите на идемпотентност и автоматично управление на инфраструктура.

```
buckets:
- id: 11cf0cf4-76d3-4615-aecc-21497b01ecc9
  name: tuvarna-dev-eu-central-1-867102643694
  region: eu-central-1
  preventDestroy: false
  tags:
    environment: dev
    team: storage
    organisation: tuvarna
  versioning: Enabled
  encryption:
    type: aes256
    bucketKeyEnabled: true
  ownershipControls: BucketOwnerPreferred
  acl:
    canned: private
  policy: |
    {
      "Version": "2012-10-17",
```

```

"Statement": [
  {
    "Sid": "AllowFullAccessToSpecificUser",
    "Effect": "Allow",
    "Principal": {
      "AWS": "arn:aws:iam::867102643694:root"
    },
    "Action": [
      "s3:GetObject",
      "s3:GetBucketLocation",
      "s3:ListBucket"
    ],
    "Resource": [
      "arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694",
      "arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694/*"
    ]
  }
]
}

```

Фрагмент 92. Тестова конфигурация

Таблица 10. Тест с модификация на съществуващ S3 ресурс

Изход
Differences detected in steps: [bg.tuvarna.sit.cloud.core.aws.s3.step.S3TaggingStep, bg.tuvarna.sit.cloud.core.aws.s3.step.S3PolicyStep, bg.tuvarna.sit.cloud.core.aws.s3.step.S3OwnershipControlsStep, bg.tuvarna.sit.cloud.core.aws.s3.step.S3AclStep] Changes in step 'bg.tuvarna.sit.cloud.core.aws.s3.step.S3TaggingStep': ↪ value: '{environment=dev, team=storage}' → '{environment=dev, team=storage, organisation=tuvarna}' Changes in step 'bg.tuvarna.sit.cloud.core.aws.s3.step.S3PolicyStep': ↪ value: '{"Version":"2012-10-17","Statement":[{"Sid":"AllowFullAccessToSpecificUser","Effect":"Allow","Principal":{"AWS":"arn:aws:iam::867102643694:root"},"Action":"s3:*","Resource":["arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694","arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694/*"]}]}' → '{"Version":"2012-10-17","Statement":[{"Sid":"AllowFullAccessToSpecificUser","Effect":"Allow","Principal":{"AWS":"arn:aws:iam::867102643694:root"},"Action":["s3:GetObject","s3:GetBucketLocation","s3:ListBucket"],"Resource":["arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694","arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694/*"]}]}' Changes in step 'bg.tuvarna.sit.cloud.core.aws.s3.step.S3AclStep': ↪ value: '{owner={id=4e845a7b7e2838ec224e7b2e222c3f7e950cd51374c8b19b6ba55adf595b773b, displayName=null}, grants=[{grantee={type=CanonicalUser, identifier=4e845a7b7e2838ec224e7b2e222c3f7e950cd51374c8b19b6ba55adf595b773b, uri=null, email=null}, permission=FULL_CONTROL}], cannedAcl=null}' → '{owner=null, grants=null, cannedAcl=private}' Changes in step 'bg.tuvarna.sit.cloud.core.aws.s3.step.S3OwnershipControlsStep':

```
↪ type: 'BucketOwnerEnforced' → 'BucketOwnerPreferred'
Successfully applied tags '[Tag(Key=environment, Value=dev), Tag(Key=team, Value=storage), Tag(Key=organisation, Value=tuvarna)]' to bucket 'tuvarna-dev-eu-central-1-867102643694'
Successfully applied policy '{"Version":"2012-10-17","Statement":[{"Sid":"AllowFullAccessToSpecificUser","Effect":"Allow","Principal":{"AWS":"arn:aws:iam::867102643694:root"},"Action":["s3:GetObject","s3:GetBucketLocation","s3:ListBucket"],"Resource":["arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694","arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694/*"]}]}' to bucket 'tuvarna-dev-eu-central-1-867102643694'
Successfully applied ownership controls 'BucketOwnerPreferred' to bucket 'tuvarna-dev-eu-central-1-867102643694'
Successfully applied canned acl 'private' to bucket 'tuvarna-dev-eu-central-1-867102643694'
```

5.2.3 Валидиране на прилагането на ръчно дефинирана ACL политика

В този тестов сценарий ще валидиране на прилагането на ръчно дефинирана ACL политика (Access Control List) към съществуващ S3, като заместим предварително зададения canned ACL (private) с детайлно описана политика на достъп. Целта е да се провери дали системата правилно интерпретира и прилага сложни ACL конфигурации, които предоставят конкретни права на посочени потребители.

Конфигурацията включва секция `accessControlPolicy`, която определя собственика на S3 чрез неговия `CanonicalUser ID` и `displayName`, както и списък с грантове (grants), които определят правата за други потребители. В конкретния случай се задава грант с право `WRITE_ACP` (право за промяна на ACL политиката на S3) за потребител с посочен `CanonicalUser ID`.

Тестът ще провери дали системата правилно замества canned ACL с новата `accessControlPolicy`, прилагайки я чрез `putAcl()`, и дали резултатът от `getAcl()` отразява новите стойности – включително собственика и грантовете.

Това е важен тест за покриване на случаи с повишена нужда от контрол и гъвкавост, когато достъпът до S3 не може да бъде описан с някой от вградените canned ACL-и, а изисква персонализирано конфигуриране. Успешното прилагане на такава ACL политика потвърждава, че системата поддържа пълен контрол върху управлението на достъпа до S3 ресурсите.

```
buckets:
- id: 11cf0cf4-76d3-4615-aecc-21497b01ecc9
  name: tuvarna-dev-eu-central-1-867102643694
  region: eu-central-1
```

```

preventDestroy: false
tags:
  environment: dev
  team: storage
  organisation: tuvarna
versioning: Enabled
encryption:
  type: aes256
  bucketKeyEnabled: true
ownershipControls: BucketOwnerPreferred
acl:
  accessControlPolicy:
    owner:
      id:
        „4e845a7b7e2838ec224e7b2e222c3f7e950cd51374c8b19b6ba55adf595b773b“
      displayName: „webfile“
    grants:
      - permission: WRITE_ACP
        grantee:
          type: CanonicalUser
          id:
            „4e845a7b7e2838ec224e7b2e222c3f7e950cd51374c8b19b6ba55adf595b773b“
  policy: |
    {
      „Version“: „2012-10-17“,
      „Statement“: [
        {
          „Sid“: „AllowFullAccessToSpecificUser“,
          „Effect“: „Allow“,
          „Principal“: {
            „AWS“: „arn:aws:iam::867102643694:root“
          },
          „Action“: [
            „s3:GetObject“,
            „s3:GetBucketLocation“,
            „s3:ListBucket“
          ],
          „Resource“: [
            „arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694“,
            „arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694/*“
          ]
        }
      ]
    }

```

Фрагмент 93. Тестова конфигурация

Таблица 11. Тест с успешно прилагане на ръчно дефинирана ACL политика

Изход	
Changes in step ,bg.tuvarna.sit.cloud.core.aws.s3.step.S3AclStep‘:	
↪	value:
, {owner={id=4e845a7b7e2838ec224e7b2e222c3f7e950cd51374c8b19b6ba55adf595b773b, displayName=null}, grants=[{grantee={type=CanonicalUser, identifier=4e845a7b7e2838ec224e7b2e222c3f7e950cd51374c8b19b6ba55adf595b773b, uri=null, email=null}, permission=FULL_CONTROL}], cannedAcl=null}‘ →	
, {owner={id=4e845a7b7e2838ec224e7b2e222c3f7e950cd51374c8b19b6ba55adf595b773b, displayName=null}, grants=[{grantee={type=CanonicalUser, identifier=4e845a7b7e2838ec224e7b2e222c3f7e950cd51374c8b19b6ba55adf595b773b, uri=null, email=null}, permission=FULL_CONTROL}], cannedAcl=null}	

```
5b773b,      displayName=webfile},      grant“=[ {g‘antee={type=CanonicalUser,
identifier=4e845a7b7e2838ec224e7b2e222c3f7e950cd51374c8b19b6ba55adf595b7
73b, uri=null, email=null}, permission=WRITE_ACP}], cannedAcl=null}‘
Successfully      applied      acl      policy
,AccessControlPolicy(Grants=[Grant(Grantee=Grantee(ID=4e845a7b7e2838ec224e
7b2e222c3f7e950cd51374c8b19b6ba55adf595b773b,      Type=CanonicalUser),
Permission=WRITE_ACP)],      Owner=Owner(DisplayName=webfile,
ID=4e845a7b7e2838ec224e7b2e222c3f7e950cd51374c8b19b6ba55adf595b773b))‘
to bucket ,tuvarna-dev-eu-central-1-867102643694‘
Successfully verified acl configuration for bucket ,tuvarna-dev-eu-central-1-
867102643694‘
```

5.2.4 Прилагане на ръчно зададен KMS ключ за криптиране чрез LocalStack

В този тестов сценарий ще валидираме прилагането на ръчно зададен KMS ключ за криптиране на S, като използваме тип `aws:kms` и предоставим конкретен `kmsKeyId`. Основната цел е да се потвърди, че системата правилно интерпретира и прилага KMS-базирана криптирация чрез конфигурацията, без да разчита на автоматично управлявани ключове от AWS (`sse-s3` или `aes256`).

Тъй като в реалния AWS акаунт не разполагаме с валиден и съществуващ KMS ключ с посочения ARN, този тест ще бъде изпълнен в контролирана среда чрез LocalStack. LocalStack е локална емуляция на AWS, която предоставя съвместим API и позволява интеграционно тестване без реално свързване към AWS инфраструктурата.

Тестът ще потвърди, че системата успешно извлича `kmsKeyId` от YAML конфигурацията, подава го правилно за прилагане и полученото състояние от `getBucketEncryption()` съдържа коректния `aws:kms` тип и референция към ключа.

Използването на LocalStack в този контекст е важна част от интеграционното покритие, тъй като позволява да се валидира логиката и API повикванията без нужда от реални облачни ресурси. Така се гарантира, че имплементацията поддържа KMS-базирана криптирация по зададен ключ, дори когато такъв не е предварително създаден в истинската среда.

```
buckets:
- id: 11cf0cf4-76d3-4615-aecc-21497b01ecc9
  name: tuvarna-dev-eu-central-1-867102643694
  region: eu-central-1
  preventDestroy: false
  tags:
    environment: dev
    team: storage
```

```

    organisation: tuvarna
    versioning: Enabled
    encryption:
      type: aws:kms
      kmsKeyId: arn:aws:kms:us-east-1:123456789013:key/your-key-id
    ownershipControls: BucketOwnerPreferred
    acl:
      accessControlPolicy:
        owner:
          id:
            "4e845a7b7e2838ec224e7b2e222c3f7e950cd51374c8b19b6ba55adf595b773b"
          displayName: "webfile"
        grants:
          - permission: WRITE_ACP
            grantee:
              type: CanonicalUser
              id:
                "4e845a7b7e2838ec224e7b2e222c3f7e950cd51374c8b19b6ba55adf595b773b"
      policy: |
        {
          "Version": "2012-10-17",
          "Statement": [
            {
              "Sid": "AllowFullAccessToSpecificUser",
              "Effect": "Allow",
              "Principal": {
                "AWS": "arn:aws:iam::867102643694:root"
              },
              "Action": [
                "s3:GetObject",
                "s3:GetBucketLocation",
                "s3:ListBucket"
              ],
              "Resource": [
                "arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694",
                "arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694/*"
              ]
            }
          ]
        }
  }
}

```

Фрагмент 94. Тестова конфигурация.

Таблица 12. Тест с успешно прилагане на KMS чрез LocalStack

Изход
Changes in step 'bg.tuvarna.sit.cloud.core.aws.s3.step.S3EncryptionStep': ↳ type: 'AES256' → 'AWS:KMS' ↳ kmsKeyId: 'null' → 'arn:aws:kms:us-east-1:123456789013:key/your-key-id' Successfully applied server-side encryption 'aws:kms' to bucket 'tuvarna-eu-west-1-000000000000' Successfully verified server-side encryption 'aws:kms' for bucket 'tuvarna-eu-west-1-000000000000'

5.2.5 Проверка на функционалността за възстановяване

В този тестов сценарий се валидира функционалността за възстановяване (revert) на ресурс до предходно състояние при частично неуспешно прилагане на промените. Тестването се извършва чрез умишлено въвеждане на грешка във втората последователна стъпка, с цел да се провери дали системата правилно връща всички предходни коректно приложени изменения.

В конкретния случай първата промяна (стъпка №4) е добавяне на нов таг към S3 – faculty: FITA. Тази промяна е валидна и ще бъде успешно приложена върху ресурса. След нея обаче следва нова стъпка (№5), която се опитва да приложи нова IAM policy конфигурация. Подаденият JSON документ обаче съдържа синтактична грешка – липсваща запетая след масива Action, което води до невалиден формат на политиката за достъп. В резултат, тази стъпка ще се провали по време на изпълнение.

Поради възникналата грешка системата трябва да изпълни механизма за връщане назад (revert), който разчита на запазено предходно състояние (state) на ресурсите. В рамките на възстановяването ще се премахне новият таг faculty: FITA, тъй като той не е бил част от оригиналната конфигурация, и ще се възстанови последно валидното състояние на S3. Освен това, документът за политика на достъп няма да бъде променен, тъй като стъпката за неговото обновяване е прикратена при фазата на прилагане.

Този тест е изключително важен, тъй като демонстрира устойчивостта и безопасността на системата при частични неуспехи. Той гарантира, че всяка серия от промени се изпълнява или изцяло успешно, или се връща до напълно предходното валидно състояние, като по този начин се избягват недетерминирани или невалидни състояния в облачната инфраструктура.

```
buckets:
- id: 11cf0cf4-76d3-4615-aecc-21497b01ecc9
  name: tuvarna-dev-eu-central-1-867102643694
  region: eu-central-1
  preventDestroy: false
  tags:
    environment: dev
    team: storage
    organisation: tuvarna
    faculty: FITA
  versioning: Enabled
  encryption:
    type: aes256
    bucketKeyEnabled: true
  ownershipControls: BucketOwnerPreferred
  acl:
    accessControlPolicy:
      owner:
```

```

      id:
"4e845a7b7e2838ec224e7b2e222c3f7e950cd51374c8b19b6ba55adf595b773b"
      displayName: "webfile"
      grants:
        - permission: WRITE_ACP
          grantee:
            type: CanonicalUser
            id:
"4e845a7b7e2838ec224e7b2e222c3f7e950cd51374c8b19b6ba55adf595b773b"
      policy: |
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowFullAccessToSpecificUser",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::867102643694:root"
      },
      "Action": [
        "s3:GetObject",
        "s3:GetBucketLocation",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694",
        "arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694/*"
      ]
    }
  ]
}

```

Фрагмент 95. Тестова конфигурация

Таблица 13. Тест с успешно възстановяване при грешка

Изход
Changes in step 'bg.tuvarna.sit.cloud.core.aws.s3.step.S3TaggingStep': ↪ value: '{environment=dev, organisation=tuvarna, team=storage}' → '{environment=dev, team=storage, organisation=tuvarna, faculty=FITA}' Changes in step 'bg.tuvarna.sit.cloud.core.aws.s3.step.S3PolicyStep': ↪ value: '{"Version":"2012-10-17","Statement":[{"Sid":"AllowFullAccessToSpecificUser","Effect":"Allow","Principal":{"AWS":"arn:aws:iam::867102643694:root"},"Action":["s3:GetObject","s3:GetBucketLocation","s3:ListBucket"],"Resource":["arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694","arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694/*"]}]}' → '{"Version":"2012-10-17","Statement":[{"Sid":"AllowFullAccessToSpecificUser","Effect":"Allow","Principal":{"AWS":"arn:aws:iam::867102643694:root"},"Action":["s3:GetObject","s3:GetBucketLocation","s3:ListBucket"],"Resource":["arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694","arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694/*"]}]}'

```
Successfully applied tags '[Tag(Key=environment, Value=dev), Tag(Key=team, Value=storage), Tag(Key=organisation, Value=tuvarna), Tag(Key=faculty, Value=FITA)]' to bucket 'tuvarna-dev-eu-central-1-867102643694'
Initiating rollback to the initial state to maintain transactional consistency...
Changes in step 'bg.tuvarna.sit.cloud.core.aws.s3.step.S3TaggingStep':
  ↳ value: '{environment=dev, organisation=tuvarna, team=storage, faculty=FITA}'
  ↳ '{environment=dev, organisation=tuvarna, team=storage}'
S3 bucket reverter for 'tuvarna-dev-eu-central-1-867102643694' finished in 278 ms
Provisioning failed for tuvarna-dev-eu-central-1-867102643694 '867102643694', but
all changes were rolled back to the previously persisted state. No partial modifications
remain
S3 provisioning failed
details:
- Failed to attach policy to bucket 'tuvarna-dev-eu-central-1-867102643694'.
- "This policy contains invalid Json (Service: S3, Status Code: 400, Request ID:
FA3VXKV9P38QBBVS, Extended Request ID:
XUFaN4sX3senjT1YFdcPsatjGqHfa9sixTvprhDdfvccNeSc7GuSAAn/b+PnyWeD/
Wcp2aHObQU=) (SDK Attempt Count: 1)"
```

5.2.6 Предотвратяване на изтриване на главен ресурс

В този тестов сценарий се проверява функционалността за предотвратяване на изтриване на ресурс чрез свойството `preventDestroy`. Това поведение е от съществено значение за защита на критични ресурси от случайно или неоторизирано премахване, особено в продукционна среда.

Допълнително, този тестов сценарий ни позволява да валидираме и поведението на останалите подстъпки, които съпровождат основния S3 ресурс — като `versioning`, `encryption`, `tagging`, `policy`, `ownership controls` и други. Тъй като главният ресурс (S3) и неговите подстъпки не присъстват в конфигурацията, всички тези стъпки също ще бъдат интерпретирани като "нежелани" и съответно ще бъде стартирана процедура по тяхното изтриване.

Въпреки че тези подстъпки първоначално ще бъдат успешно премахнати, процесът ще бъде прекъснат веднага след като системата достигне до основния ресурс и засече, че `preventDestroy` е `true`. Това ще предизвика `rollback` (възстановяване), при който всички вече премахнати подстъпки ще бъдат върнати в предишното си състояние. По този начин, тестът не само валидира защитата срещу изтриване на S3, но ефективно тества и способността на системата да възстанови коректно всички зависими стъпки, запазвайки консистентността и идемпотентността на конфигурацията.

Симулацията се извършва чрез предоставяне на празен списък от кофи в YAML конфигурацията:

```
buckets:
```

Фрагмент 96.

Това означава, че няма нито един S3, описан като желан за бъдещото състояние, а съществуващите такива ще бъдат интерпретирани като ресурси, подлежащи на унищожаване. В този конкретен случай, в state файла вече е записан ресурс, чиято конфигурация съдържа флага:

```
preventDestroy: true
```

Фрагмент 97.

Този флаг изрично указва, че S3 не трябва да бъде изтрит при никакви обстоятелства, освен ако не бъде ръчно премахнат или флагът бъде изключен.

При изпълнение на системата, логиката за унищожаване ще се опита да премахне S3, тъй като той вече не присъства в списъка с желани ресурси. Обаче, при проверка на preventDestroy, ще бъде засечено, че ресурсът е защитен. В резултат на това ще бъде генерирана грешка, която ще прекрати процеса на унищожаване и ще върне съответното съобщение, описващо причината за отказа.

Този тест валидира, че системата спазва строго конфигурационните политики за безопасност, като предотвратява непреднамерено изтриване на важни ресурси и предоставя механизъм за защита на инфраструктурата в дългосрочен план.

```
buckets:
# - id: 11cf0cf4-76d3-4615-aecc-21497b01ecc9
#   name: tuvarna-dev-eu-central-1-867102643694
#   region: eu-central-1
#   preventDestroy: true
#   tags:
#     environment: dev
#     team: storage
#     organisation: tuvarna
#   versioning: Enabled
#   encryption:
#     type: aes256
#     bucketKeyEnabled: true
#   ownershipControls: BucketOwnerPreferred
#   acl:
#     accessControlPolicy:
#       owner:
#         id:
# "4e845a7b7e2838ec224e7b2e222c3f7e950cd51374c8b19b6ba55adf595b773b"
#       displayName: "webfile"
#       grants:
#         - permission: WRITE_ACP
#           grantee:
#             type: CanonicalUser
#             id:
# "4e845a7b7e2838ec224e7b2e222c3f7e950cd51374c8b19b6ba55adf595b773b"
#   policy: |
#     {
#       "Version": "2012-10-17",
#       "Statement": [
#         {
#           "Sid": "AllowFullAccessToSpecificUser",
```

```

#           "Effect": "Allow",
#           "Principal": {
#             "AWS": "arn:aws:iam::867102643694:root"
#           },
#           "Action": [
#             "s3:GetObject",
#             "s3:GetBucketLocation",
#             "s3:ListBucket"
#           ],
#           "Resource": [
#             "arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694",
#             "arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694/*"
#           ]
#         }

```

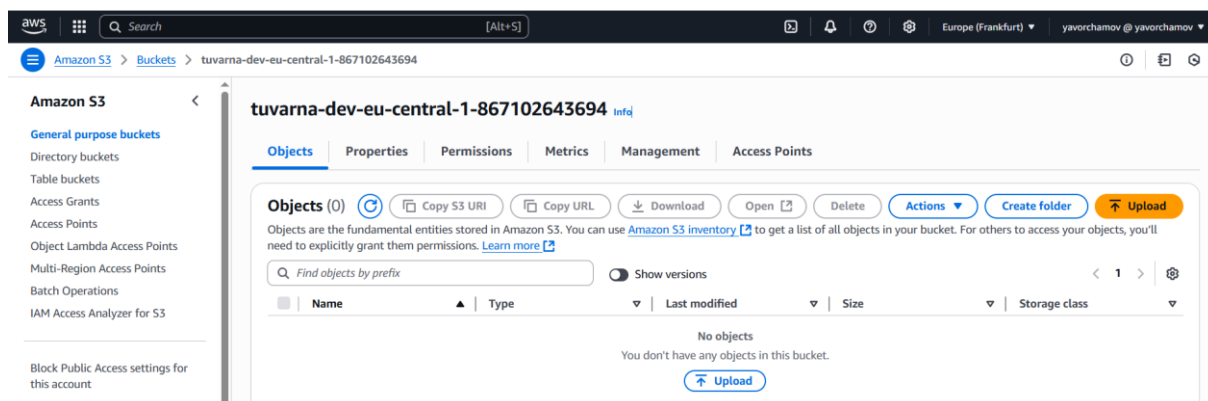
Фрагмент 98. Тестова конфигурация

Таблица 14. Тест с успешно предотвратяване на изтриване на ресурс

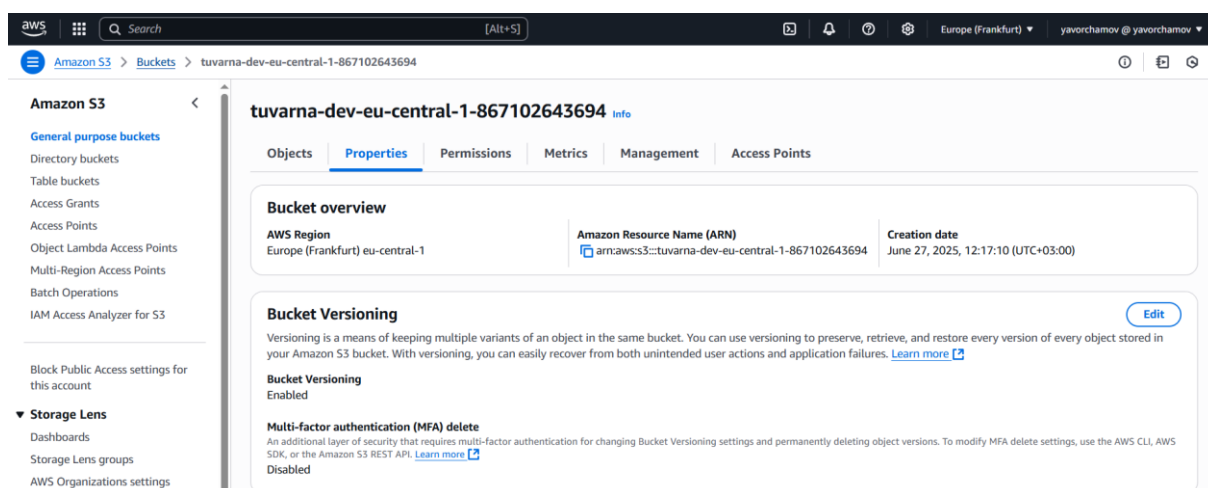
Изход
Step 'bg.tuvarna.sit.cloud.core.aws.s3.step.S3TaggingStep' has been cleared and will be deleted. Step 'bg.tuvarna.sit.cloud.core.aws.s3.step.S3BucketStep' has been cleared and will be deleted. Step 'bg.tuvarna.sit.cloud.core.aws.s3.step.S3PolicyStep' has been cleared and will be deleted. Changes in step 'bg.tuvarna.sit.cloud.core.aws.s3.step.S3OwnershipControlsStep': ↳ type: 'BucketOwnerPreferred' → 'BucketOwnerEnforced' Changes in step 'bg.tuvarna.sit.cloud.core.aws.s3.step.S3EncryptionStep': ↳ bucketKeyEnabled: 'true' → 'false' Deleted policy from bucket 'tuvarna-dev-eu-central-1-867102643694' Deleted all tags from bucket 'tuvarna-dev-eu-central-1-867102643694' Destruction of bucket 'tuvarna-dev-eu-central-1-867102643694' is prevented by configuration. Initiating rollback to the initial state to maintain transactional consistency... Changes in step 'bg.tuvarna.sit.cloud.core.aws.s3.step.S3TaggingStep': ↳ value: 'null' → '{environment=dev, organisation=tuvarna, team=storage}' Changes in step 'bg.tuvarna.sit.cloud.core.aws.s3.step.S3PolicyStep': ↳ value: 'null' → '{"Version":"2012-10-17","Statement":[{"Sid":"AllowFullAccessToSpecificUser","Effect":"Allow","Principal":{"AWS":"arn:aws:iam::867102643694:root"},"Action":["s3:GetObject","s3:GetBucketLocation","s3:ListBucket"],"Resource":["arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694","arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694/*"]}]}' S3 bucket reverter for 'tuvarna-dev-eu-central-1-867102643694' finished in 913 ms Provisioning failed for tuvarna-dev-eu-central-1-867102643694 '867102643694', but all changes were rolled back to the previously persisted state. No partial modifications remain. S3 provisioning failed code: S3 1100

details:

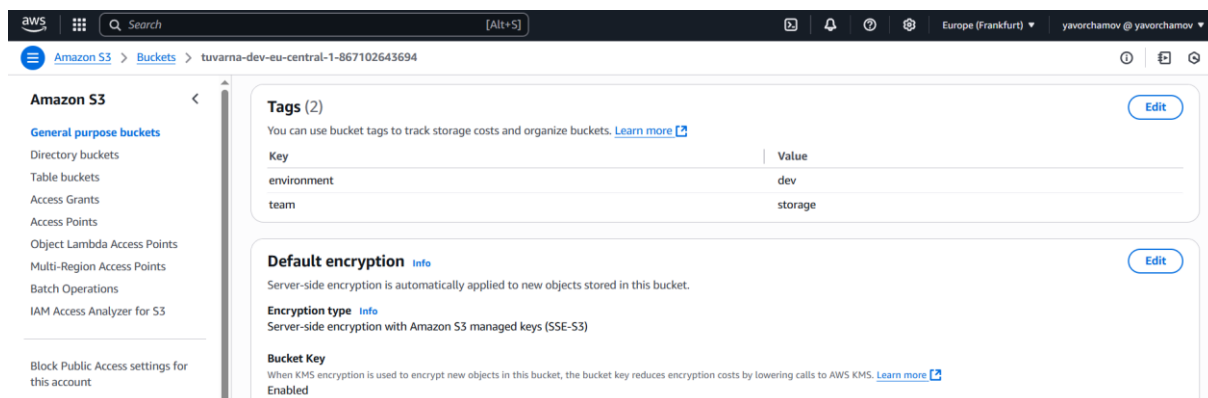
- Destruction of bucket 'tuvarna-dev-eu-central-1-867102643694' is prevented by configuration.



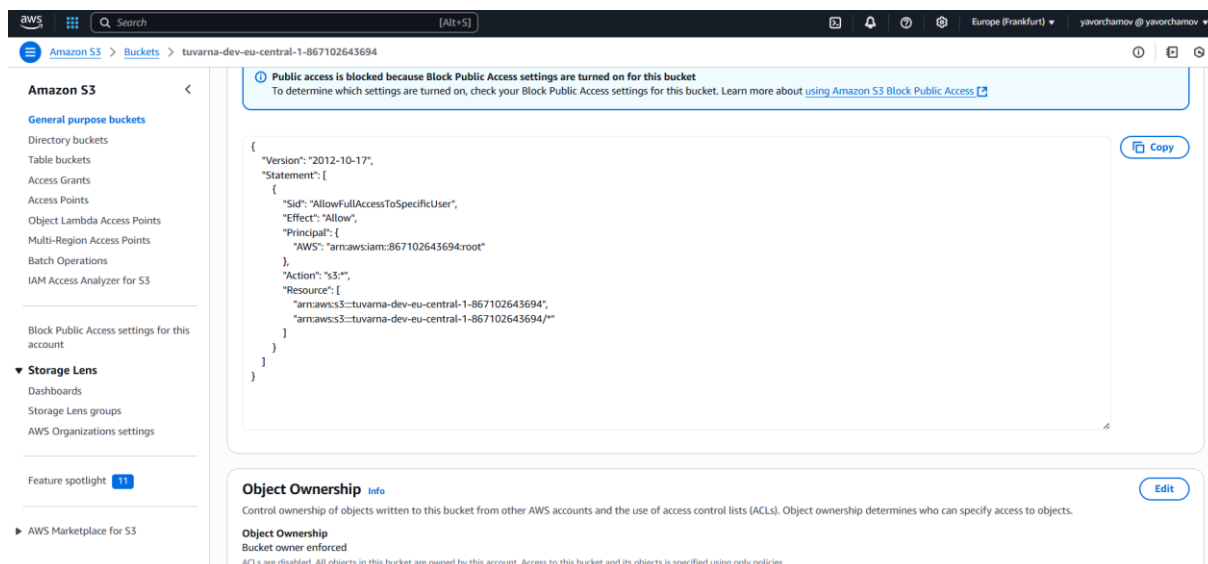
Фиг. 24 Преглед на празен S3 в AWS Management Console



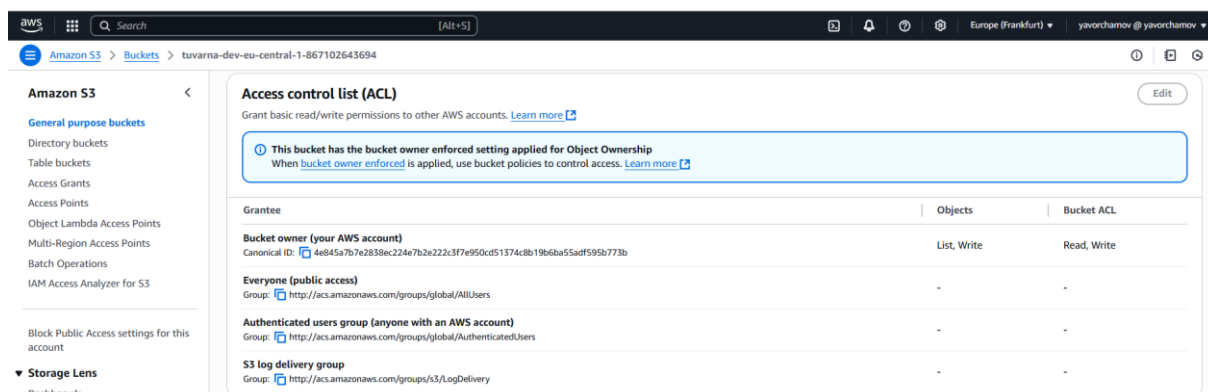
Фиг. 25 Свойства на S3 в AWS: Версионизиране и ARN



Фиг. 26 Маркери и криптиране по подразбиране за AWS S3



Фиг. 27 Политика за достъп и собственост на обекти в AWS S3



Фиг. 28 Контрол на достъпа чрез ACL в AWS S3

5.2.7 Реална операция по качване на ресурс

В този тест валидираме създадения S3 ресурс чрез реална операция по качване и последващо изброяване на съдържанието. За целта първо се удостоверяваме, че работим с правилния AWS профил чрез `export AWS_PROFILE=867102643694`. След това създаваме примерен файл `demo.txt` с помощта на командата `touch`, който ще бъде използван като тестов ресурс. Използваме командата `aws s3 cp`, за да качим файла в предварително създадения S3 `s3://tuvarna-dev-eu-central-1-867102643694`. За да потвърдим, че операцията е успешна и S3 е достъпен, изпълняваме `aws s3 ls`, което ни връща списък с файловете в S3. Този тест потвърждава, че кофата е конфигурирана правилно, че достъпът до нея е наличен и че качването и четенето на обекти функционират очаквано.

Таблица 15. Тест с успешно качване на файл в създадения S3

Вход	Изход
<code>export AWS_PROFILE=867102643694</code> <code>touch demo.txt</code> <code>aws s3 cp demo.txt s3://tuvarna-dev-eu-central-1-867102643694</code> <code>aws s3 ls s3://tuvarna-dev-eu-central-1-867102643694/</code>	<code>upload: ./demo.txt to s3://tuvarna-dev-eu-central-1-867102643694/demo.txt</code> 2025-06-28 17:46:13 0 demo.txt

5.2.8 Окончателно изтриване на ресурсите

В последния тест от поредицата се проверява правилната работа на механизма за окончателно изтриване на ресурсите. За целта предоставяме празен списък под ключа `buckets` в YAML конфигурацията, което означава, че не се очаква нито един S3 ресурс да бъде поддържан след изпълнение на заявката. В резултат на това всички създадени до момента S3 трябва да бъдат идентифицирани като излишни и подлежат на изтриване. Ако някоя от тях не е маркирана с `preventDestroy: true`, тя ще бъде премахната напълно. Чрез този тест се потвърждава, че процесът на унищожаване работи коректно – премахват се не само S3, но и всички свързани с тях компоненти като политики, тагове, ACL настройки и други. Това е ключова част от поддържането на идемпотентност и автоматизиран жизнен цикъл на инфраструктурата.

В допълнение към сценария за окончателно изтриване, е важно да бъде разгледан и случай, при който S3 не е празен, което възпрепятства нейното унищожаване. В реална AWS среда, когато се направи опит за изтриване на S3, която съдържа

обекти (напр. файлове), операцията се проваля, освен ако предварително не бъдат изтрети всички вътрешни ресурси.

```
buckets:
# - id: 11cf0cf4-76d3-4615-aecc-21497b01ecc9
#   name: tuvarna-dev-eu-central-1-867102643694
#   region: eu-central-1
#   preventDestroy: false
#   tags:
#     environment: dev
#     team: storage
#     organisation: tuvarna
#   versioning: Enabled
#   encryption:
#     type: aes256
#     bucketKeyEnabled: true
#   ownershipControls: BucketOwnerPreferred
#   acl:
#     accessControlPolicy:
#       owner:
#         id:
#           „4e845a7b7e2838ec224e7b2e22c3f7e950cd51374c8b19b6ba55adf595b773b“
#         displayName: „webfile“
#       grants:
#         - permission: WRITE_ACP
#           grantee:
#             type: CanonicalUser
#             id:
#               „4e845a7b7e2838ec224e7b2e22c3f7e950cd51374c8b19b6ba55adf595b773b“
#     policy: |
#       {
#         „Version“: „2012-10-17“,
#         „Statement“: [
#           {
#             „Sid“: „AllowFullAccessToSpecificUser“,
#             „Effect“: „Allow“,
#             „Principal“: {
#               „AWS“: „arn:aws:iam::867102643694:root“
#             },
#             „Action“: [
#               „s3:GetObject“,
#               „s3:GetBucketLocation“,
#               „s3:ListBucket“
#             ],
#             „Resource“: [
#               „arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694“,
#               „arn:aws:s3:::tuvarna-dev-eu-central-1-867102643694/*“
#             ]
#           }
#         ]
#       }
#     }
```

Фрагмент 99. Тестова конфигурация

Таблица 16. Тест с предупреждение за непразен S3 преди изтриване и последващо изтриване

Изход
S3 provisioning failed code: S3_1100 details: - S3 service error while deleting bucket ,tuvarena-dev-eu-central-1-867102643694‘ - „The bucket you tried to delete is not empty. You must delete all versions in the bucket. (Service: S3, Status Code: 409, Request ID: Q8T970G7H7NCQDSQ, Extended Request ID: 0K15qomei5wrEOYbJn1mSSjb77K6nGZU2t9IS9QPXM5a2ybTeKYocpmeg61ZbSMJ3gbL6cvt3c0=) (SDK Attempt Count: 1)“
Deleted policy from bucket ,tuvarena-dev-eu-central-1-867102643694‘ Deleted all tags from bucket ,tuvarena-dev-eu-central-1-867102643694‘ Deleted bucket ,tuvarena-dev-eu-central-1-867102643694‘ S3 bucket destroyer for ,tuvarena-dev-eu-central-1-867102643694‘ finished in 981 ms Main resource ,S3BucketStep‘ marked for deletion Deleted state file for destroyed bucket: state-tuvarena-dev-eu-central-1-867102643694#11cf0cf4-76d3-4615-aecc-21497b01ecc9.json

5.3 Услуга EKS

5.3.1 Цялостен процес по създаване на клъстер

В този тест се валидира цялостния процес по създаване на EKS клъстер, включително всички негови основни компоненти – самия клъстер, група от работни възли (node group) и необходимите приставки (addons). Конфигурацията дефинира клъстер с име dev-tuvarena-eks, който ще бъде създаден в региона eu-west-1 и разположен в две подсмрежи от стандартната (default) VPC мрежа. Клъстерът използва IAM роля за управление (roleArn) и съдържа етикети за среда и екип. Конфигурацията също така включва активиране на административни права за създателя и избрана стратегия за автентикация чрез API и ConfigMap.

По време на теста ще бъде създадена една група работни възли (node group) с име dev-appcluster-ng, която ще съдържа виртуални машини от тип t3.medium, предназначени да покрият основните нужди на разработката и демонстрацията. Конфигурирани са минимален брой от 1 възел и максимум от 3, като в началото се стартира с 1 желан (desired) екземпляр. Възлите ще използват отделна IAM роля за достъп до AWS ресурсите и ще бъдат разпределени в същите две подсмрежи, както и самият клъстер.

Допълнително, към клъстъра ще бъдат инсталирани три основни разширения (EKS addons): vpc-cni за управление на мрежовата свързаност на контейнерите, kube-proxy за маршрутизиране на заявките в Kubernetes мрежата и coredns за обслужване на вътрешния DNS трафик между контейнерите и услугите. С тях се гарантира минималната необходима функционалност, за да може клъстърът да работи правилно и да стартира приложения.

Този тест покрива цялата верига по създаване на Kubernetes инфраструктура чрез EKS и ще потвърди коректната работа на последователните стъпки, описани в предходните глави: създаване на клъстър, добавяне на група възли, инсталиране на разширения, както и обработка на тагове и допълнителни конфигурации.

```
clusters:
- id: 8982a844-2aed-44c6-a525-d425e48a8fff
  name: dev-tuvarna-eks
  region: eu-west-1
  version: "1.32"
  roleArn: "arn:aws:iam::867102643694:role/dev.ekscluster.iamrole"
  tags:
    environment: development
    team: devops
  subnets:
    - subnet-0689621ccf45d992e
    - subnet-0f7fdf8944640fc6d
  bootstrapClusterCreatorAdminPermissions: true
  authenticationMode: API_AND_CONFIG_MAP
  bootstrapSelfManagedAddons: true
  supportType: STANDARD
  enableZonalShift: false
  addons:
    - name: vpc-cni
      version: v10.19.2-eksbuild.1
    - name: kube-proxy
      version: v1.32.0-eksbuild.2
    - name: coredns
      version: v1.11.4-eksbuild.2
  enableReconciliation: false
  preventDestroy: false
  nodeGroups:
    - name: dev-appcluster-ng
      instanceType: t3.medium
      minSize: 1
      maxSize: 3
      desiredSize: 1
      nodeRoleArn: arn:aws:iam::867102643694:role/dev.nodegroup.iamrole
      subnets:
        - subnet-0689621ccf45d992e
        - subnet-0f7fdf8944640fc6d
```

Фрагмент 100. Тестова конфигурация

Таблица 17. Тест с успешно създаване на функциониращ клъстер

Изход
Successfully created EKS cluster 'dev-tuvarna-eks' Waiting for EKS cluster 'dev-tuvarna-eks' to reach status ACTIVE... Cluster 'dev-tuvarna-eks' status: CREATING Cluster 'dev-tuvarna-eks' status: ACTIVE EKS cluster 'dev-tuvarna-eks' is now ACTIVE. Successfully verified existence of EKS cluster 'dev-tuvarna-eks' Successfully tagged EKS cluster 'arn:aws:eks:eu-west-1:867102643694:cluster/dev-tuvarna-eks' Successfully applied tags '{environment=development, team=devops}' to EKS cluster 'arn:aws:eks:eu-west-1:867102643694:cluster/dev-tuvarna-eks' Successfully created node group 'dev-appcluster-ng' in cluster 'dev-tuvarna-eks' Waiting for node groups on cluster 'dev-tuvarna-eks' to become ACTIVE... Node group 'dev-appcluster-ng' status: CREATING All node groups in cluster 'dev-tuvarna-eks' are ACTIVE Created addon 'vpc-cni' (version 'v1.19.2-eksbuild.1') in cluster 'dev-tuvarna-eks' Created addon 'kube-proxy' (version 'v1.32.0-eksbuild.2') in cluster 'dev-tuvarna-eks' Created addon 'coredns' (version 'v1.11.4-eksbuild.2') in cluster 'dev-tuvarna-eks' Addon 'kube-proxy' status: ACTIVE Addon 'coredns' status: ACTIVE Addon 'vpc-cni' status: ACTIVE All addons in cluster 'dev-tuvarna-eks' are ACTIVE. EKS provisioner for cluster 'dev-tuvarna-eks' finished in 559863 ms

Глава 6. Заключение

Разработената дипломна работа постига основните цели, формулирани в Глава 2 – „Анализ на проблема“, чрез създаване на библиотека за декларативно управление на облачни ресурси. Решението позволява на разработчиците да дефинират желаното състояние на инфраструктурата чрез конфигурации, които се интерпретират и прилагат автоматично чрез Java библиотека, използваща програмния приложен интерфейс на облачния доставчик. Постига се ключовата цел за идемпотентност, изравняване на инфраструктурни състояния и възможност за връщане назад при грешка.

За осигуряване на сигурен и гъвкав достъп до облачните ресурси е реализиран механизъм за автентикация, който абстрахира конкретния източник на удостоверяване. Системата поддържа три основни подхода: използване на статични ключове, дефинирани в конфигурационен файл, използване на профили, конфигурирани в средата на изпълнение, и извличане на идентификационни данни от Vault. Последният подход позволява централно управление на чувствителна информация.

Реализирани са абстракции за управление на ресурси като S3 и EKS клъстър, включително техните допълнителни компоненти (маркери, политики за достъп, разширения, работни групи от виртуални машини и др.). Всяка операция е изградена на база отделна стъпка, която поддържа пълния цикъл от създаване, проверка на текущото състояние, обновяване, изтриване и възстановяване. Това покрива изискванията, дефинирани в заданието към дипломния проект – за гъвкавост, модулност и надеждност.

Проведените тестове потвърждават, че системата работи устойчиво при различни сценарии: създаване на ресурси, прилагане на промени, симулация на грешки и опити за неправомерно изтриване. Проверени са различни методи на автентикация, като Vault, статични ключове за автентикация и AWS профили, осигурявайки гъвкавост при внедряване в различни среди. Използването на LocalStack позволи безопасно и евтино провеждане на интеграционни тестове.

В заключение, системата изпълнява поставените задачи успешно и демонстрира реална приложимост и практики за управление на инфраструктурата чрез код. Получените резултати показват, че подходът е разширяем, стабилен и приложим в различни сценарии на разработка и внедряване на облачни приложения.

ИЗТОЧНИЦИ

- [1], HashiCorp, What is Terraform? <https://developer.hashicorp.com/terraform/intro>, 2025
- [2], HashiCorp (Dadgar, Armon), Introduction to HashiCorp Terraform, <https://youtu.be/h970ZBgKINg?si=HkSnIOjEYzpdAlJ>, 2018.
- [3], Gogineni, Krishna и Shaik, Srinivasa, Approach to migrate Spring Cloud microservices applications to Amazon EKS, <https://aws.amazon.com/blogs/mt/approach-to-migrate-spring-cloud-microservices-applications-to-amazon-eks/>, 2023.
- [4], Amazon Web Services, AWS SDK for Java, <https://aws.amazon.com/sdk-for-java/>, 2025.
- [5], Google Guice, Motivation, <https://github.com/google/guice/wiki/Motivation>, 2020.
- [6], Java Community Process, JSR 330: Dependency Injection for Java, <https://jcp.org/en/jsr/detail?id=330>, 2009.
- [7], FasterXML, What is Jackson?, <https://github.com/FasterXML/jackson/blob/master/README.md#what-is-jackson>, 2025.
- [8], HashiCorp, What is Vault?, <https://developer.hashicorp.com/vault/docs/about-vault/what-is-vault>, 2025.
- [9], LocalStack, Getting Started, <https://docs.localstack.cloud/getting-started/>, 2025.
- [10], QOS.ch Sarl (Switzerland), Simple Logging Facade for Java (SLF4J), <https://www.slf4j.org/>, 2025.
- [11], Amazon Web Services (AWS), What is Amazon S3?, <https://docs.aws.amazon.com/s3/index.html>, 2025
- [12], Amazon Web Services (AWS), Retaining multiple versions of objects with S3 Versioning, <https://docs.aws.amazon.com/s3/index.html>, 2025
- [13], Amazon Web Services (AWS), Protecting data with encryption, <https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingEncryption.html>, 2025
- [14], Amazon Web Services (AWS), Managing object tags, <https://docs.aws.amazon.com/AmazonS3/latest/userguide/tagging-managing.html>, 2025
- [15], Amazon Web Services (AWS), Amazon Elastic Kubernetes Service, <https://docs.aws.amazon.com/whitepapers/latest/overview-deployment-options/amazon-elastic-kubernetes-service.html>, 2025

Приложение

```
@Slf4j
public class Main {

    public static void main(String[] services) {

        if (services.length < 1) {
            String message = ""
                Missing required arguments. Usage:
                - java -jar cloudprovisioner.jar [services...]
                - ./gradlew provisionCloudResources --args='[services...]'
                Example: s3 eks"";
            log.error(message);
            throw new InitializationException(message);
        }

        Injector initializer = Guice.createInjector(new
        InitializerInjectionModule());

        BaseAuthenticationManager<AwsBasicCredentials> authenticationManager =
        initializer.getInstance(AwsBasicCredentialsAuthenticationManager.class);

        AwsBasicCredentials credentials =
        authenticationManager.getCredentials();

        try (StsClient stsClient = StsClient.builder()
            .credentialsProvider(StaticCredentialsProvider.create(credentials))
            .endpointOverride(
                EnvVar.ENDPOINT_URL.getValueOrDefault(null) != null ?
                URI.create(EnvVar.ENDPOINT_URL.getValue()) : null)
            .region(Region.AWS_GLOBAL)
            .build()) {
            GetCallerIdentityResponse identity = stsClient.getCallerIdentity();
            log.info("Authenticated AWS identity: Account = {}, ARN = {}, UserId
            = {}",
                identity.account(), identity.arn(), identity.userId());
        } catch (StsException e) {
            log.error("Failed to verify AWS credentials using STS: {}",
            e.getMessage(), e);
            throw new InitializationException("Unable to validate credentials
            with STS");
        }

        CloudBundleDispatcher dispatcher =
        initializer.getInstance(CloudBundleDispatcher.class);

        Arrays.stream(services).forEach(service -> dispatcher.dispatch(service,
        credentials));
    }
}
```

```

@SuppressWarnings({"unused"})
public class InitializerInjectionModule extends AbstractModule {

    @Override
    protected void configure() {}

    @Provides
    @Named(NamedInjections.YAML_MAPPER)
    @Singleton
    ObjectMapper provideYamlMapper() {

        YAMLFactory factory = new YAMLFactory();
        factory.disable(YAMLGenerator.Feature.SPLIT_LINES);
        factory.enable(YAMLGenerator.Feature.MINIMIZE_QUOTES);
        factory.enable(YAMLGenerator.Feature.LITERAL_BLOCK_STYLE);

        return new ObjectMapper(factory);
    }

    @Provides
    @Named(NamedInjections.JSON_MAPPER)
    @Singleton
    ObjectMapper provideJsonMapper() {
        return new ObjectMapper();
    }

    @Provides
    @Named(NamedInjections.DEFAULT_PRETTY_PRINTER)
    @Singleton
    ObjectWriter provideJsonMapperPrettyPrinter() {
        return new ObjectMapper().writerWithDefaultPrettyPrinter();
    }

    @Provides
    @Named(NamedInjections.JSON_MAPPER_SINGLE_VALUE_ARRAY)
    @Singleton
    ObjectMapper provideYamlMapperPrettyPrinter() {
        return new ObjectMapper()
            .enable(DeserializationFeature.ACCEPT_SINGLE_VALUE_AS_ARRAY);
    }

    @Provides
    @Singleton
    S3StateComparator s3StateComparator() { return new S3StateComparator(); }

    @Provides
    @Singleton
    EksClusterStateComparator eksClusterStateComparator() { return new
EksClusterStateComparator(); }

    @Provides
    @Named(NamedInjections.BUNDLES)
    @Singleton
    Set<CloudBundleRunner<?>> provideBundles(S3CloudBundleRunner s3,
EksClusterCloudBundleRunner eks) {
        return Set.of(s3, eks);
    }
}

```

```

@Slf4j
@Singleton
public class AwsBasicCredentialsAuthenticationManager extends
BaseAuthenticationManager<AwsBasicCredentials> {

    private final VaultAwsCredentialsProvider vaultAwsCredentialsProvider;
    private final Slf4jLoggingUtil loggingUtil;

    @Inject
    public AwsBasicCredentialsAuthenticationManager(ConfigurationUtil config,
VaultAwsCredentialsProvider vaultAwsCredentialsProvider,
                                                    Slf4jLoggingUtil
loggingUtil) {
        super(config);
        this.vaultAwsCredentialsProvider = vaultAwsCredentialsProvider;
        this.loggingUtil = loggingUtil;
    }

    @Override
    protected String getAuthenticationConfigurationPath() {

        return
"src/main/resources/cloud/%s/authentication.yml".formatted(EnvVar.AWS_PROFI
LE.getValue());
    }

    @Override
    public AwsBasicCredentials getCredentials() {

        String authenticationConfigurationPath =
getAuthenticationConfigurationPath();

        log.info("Loading authentication configuration from: {}",
authenticationConfigurationPath);
        AuthenticationConfig authConfig;
        try {
            authConfig = config.load(authenticationConfigurationPath,
AuthenticationConfig.class);
        } catch (ConfigurationLoadException e) {
            loggingUtil.logError(log, ErrorCode.AUTH_CONFIG_LOAD_ERROR, e);
            throw e;
        }

        try {
            return authenticate(authConfig);
        } catch (AuthenticationException e) {
            loggingUtil.logError(log, ErrorCode.AUTH_ERROR, e);
            throw e;
        }
    }

    public AwsBasicCredentials authenticate(AuthenticationConfig config) {

        for (AuthenticationConfig.ProviderConfig provider :
config.getProviders()) {

            try {

                // Vault authentication
                if (provider.getVault() != null) {

```

```

        AuthenticationConfig.VaultConfig vault = provider.getVault();

        if (vault.getToken() != null) {
            String resolvedToken =
                resolveEnvPlaceholders(vault.getToken());
            vault.setToken(resolvedToken);

            log.info("Authenticating using Vault (host={}, port={},
secretPath={})",
                vault.getHost(), vault.getPort(), vault.getSecretPath());

            return
                vaultAwsCredentialsProvider.config(vault).fetchCredentials();
        }

        // Static credentials authentication
        if (provider.getStaticCredentials() != null) {
            AuthenticationConfig.StaticConfig staticCfg =
                provider.getStaticCredentials();
            String accessKeyId =
                resolveEnvPlaceholders(staticCfg.getAccessKeyId());
            String secretAccessKey =
                resolveEnvPlaceholders(staticCfg.getSecretAccessKey());
            if (!accessKeyId.isBlank() && !secretAccessKey.isBlank()) {
                log.info("Authenticating using static credentials");
                return AwsBasicCredentials.create(accessKeyId,
                    secretAccessKey);
            }
        }

        // Profile authentication
        if (provider.getProfileCredentials() != null) {
            AuthenticationConfig.ProfileConfig profileCfg =
                provider.getProfileCredentials();
            String profileName =
                resolveEnvPlaceholders(profileCfg.getProfileName());
            if (!profileName.isBlank()) {
                try (ProfileCredentialsProvider profileCredentials =
                    ProfileCredentialsProvider.create(profileName)) {
                    log.info("Authenticating using AWS profile '{}'",
                        profileName);
                    AwsCredentials credentials =
                        profileCredentials.resolveCredentials();
                    return AwsBasicCredentials.create(credentials.accessKeyId(),
                        credentials.secretAccessKey());
                }
            }
        }

        } catch (AuthenticationException e) {
            log.warn("Authentication attempt failed using available
configuration: {}", e.getMessage());
        }

        throw new AuthenticationException("No authentication method
succeeded");
    }
}

```

```

@Singleton
public class CloudBundleDispatcher {

    private final Set<CloudBundleRunner<?>> bundles;

    @Inject
    public CloudBundleDispatcher(@Named(NamedInjections.BUNDLES)
Set<CloudBundleRunner<?>> bundles) {
        this.bundles = bundles;
    }

    public void dispatch(String resourceType, AwsBasicCredentials
credentials) {

        bundles.stream()
            .filter(p -> p.getType().toString().equalsIgnoreCase(resourceType))
            .findFirst()
            .orElseThrow(() -> new IllegalArgumentException("No provisioner
found for type: " + resourceType))
            .run(credentials);
    }
}

public abstract class CloudBundleRunner<K extends Enum<K>> {

    protected final Logger log;
    protected final ObjectMapper yaml;
    protected final BaseStoredStateLoader<K> storedState;
    protected final StateComparator<K> stateComparator;
    protected final ConfigurationUtil configUtil;
    protected final Slf4jLoggingUtil loggingUtil;

    private final StepResultStateWriter<K> stateWriter;

    protected CloudBundleRunner(org.slf4j.Logger log, ObjectMapper yaml,
BaseStoredStateLoader<K> storedState,
                                StateComparator<K> stateComparator,
ConfigurationUtil configUtil,
                                StepResultStateWriter<K> stateWriter,
Slf4jLoggingUtil loggingUtil) {
        this.log = log;
        this.yaml = yaml;
        this.storedState = storedState;
        this.stateComparator = stateComparator;
        this.configUtil = configUtil;
        this.stateWriter = stateWriter;
        this.loggingUtil = loggingUtil;
    }

    protected abstract CloudResourceType getType();

    protected abstract String getServiceConfigurationPath();

    protected abstract void run(AwsBasicCredentials credentials);
}

@Singleton
@Slf4j

```

```

public class S3CloudBundleRunner extends CloudBundleRunner<S3Output> {

    @Inject
    public S3CloudBundleRunner(@Named(NamedInjections.YAML_MAPPER)
ObjectMapper yaml, S3StoredStateLoader storedState,
                                S3StateComparator stateComparator,
StepResultStateWriter<S3Output> stateWriter,
                                ConfigurationUtil config, Slf4jLoggingUtil
loggingUtil) {

        super(log, yaml, storedState, stateComparator, config, stateWriter,
loggingUtil);
    }

    @Override
    public CloudResourceType getType() {

        return CloudResourceType.S3;
    }

    @Override
    protected String getServiceConfigurationPath() {

        return
"src/main/resources/cloud/%s/s3.yml".formatted(EnvVar.AWS_PROFILE.getValue(
));
    }

    @Override
    public void run(AwsBasicCredentials credentials) {

        String path = getServiceConfigurationPath();
        log.info("Loading S3 config from: {}", path);
        S3BucketConfigList configList;
        try {
            configList = loadConfig(path, S3BucketConfigList.class);
        } catch (ConfigurationLoadException e) {
            loggingUtil.logError(log, ErrorCode.S3_CONFIG_LOAD_ERROR, e);
            return;
        }

        Set<String> ids =
configList.getBuckets().stream().map(S3BucketConfig::getId).collect(Collectors.toSet());

        String profile = EnvVar.AWS_PROFILE.getValue();
        File stateDir = new File(".cloudprovisioner/" + profile + "/" +
getType().toString().toLowerCase());

        for (String stateKey : findObsoleteStateFiles(stateDir, ids)) {

            String[] parts = stateKey.split("#");
            S3BucketConfig delete = new S3BucketConfig();
            delete.setName(parts[0]);
            delete.setId(parts[1].replace(".json", ""));
            delete.setToDelete(true);
            delete.setPreventDestroy(null);
            configList.getBuckets().add(delete);

            log.info("Bucket with configKey '{}' is removed from config and is
marked for deletion", stateKey);

```

```

    }

    executeProvisioningTasks(configList.getBuckets(),
configList.getFixedThreadPool().getMaxSize(),
    config -> () -> provisionS3(config, credentials, profile,
stateDir), ErrorCode.S3_PROVISION_ERROR);
    }

    private CloudProvisionerSuccessfulResponse<S3Output>
provisionS3(S3BucketConfig config,

    AwsBasicCredentials credentials, String profile,
File
stateDir)
    throws CloudProvisioningTerminationException {

        return provisionGeneric(
            ProvisionRequest.<S3Output, S3BucketConfig>builder()
                .config(config)
                .profile(profile)
                .stateDir(stateDir)
                .metadataStepName(S3PersistentMetadataStep.class.getName())
                .stepProvider(inj -> inj.getInstance(Key.get(new
TypeLiteral<>() {
                    }, Names.named(NamedInjections.S3_STEPS))))
                .destroyerProvider(inj ->
inj.getInstance(S3BucketDestroyer.class))
                .provisionerProvider(inj ->
inj.getInstance(S3BucketProvisioner.class))
                .reverterProvider(inj ->
inj.getInstance(S3BucketReverter.class))
                .liveStateGenerator(inj ->
generateLiveCloudState(inj.getInstance(S3LiveStateGenerator.class)))
                .desiredStateGenerator(inj ->
generateDesiredState(inj.getInstance(S3DesiredStateGenerator.class)))
                .injectorFactory((region, metadata) -> Guice.createInjector(new
S3InjectionModule(
                    new S3ProvisioningContext(credentials,
EnvVar.S3_ENDPOINT_URL.getValue(), Region.of(region)), config,
                    metadata)))
                .metadataSupplier(cfg -> StepResult.<S3Output>builder()
                    .stepName(S3PersistentMetadataStep.class.getName())
                    .put(S3Output.NAME, cfg.getName())
                    .put(S3Output.REGION, cfg.getRegion().getValue())
                    .put(S3Output.ARN, cfg.buildArn())
                    .put(S3Output.PREVENT_DESTROY, cfg.preventDestroy() != null
? cfg.preventDestroy() : Boolean.TRUE)
                    .build())
                    .nameExtractor(step -> (String)
step.getOutputs().get(S3Output.NAME))
                    .regionExtractor(step -> (String)
step.getOutputs().get(S3Output.REGION))
                    .outputClass(S3Output.class)
                    .build()
                );
    }
}

```

```

protected <C extends BaseCloudResourceConfiguration>
CloudProvisionerSuccessfulResponse<K> provisionGeneric(
    ProvisionRequest<K, C> request
) throws CloudProvisioningTerminationException {

    log.info("Loading the last persisted infrastructure state from the local
library...");

    List<StepResult<K>> loadedState = new ArrayList<>();
    try {
        File[] matchingFiles =
Optional.ofNullable(request.stateDir.listFiles((dir, name) ->
    name.startsWith("state-") && name.endsWith("#" +
request.config.getId() + ".json"))).orElse(new File[0]);

        if (matchingFiles.length > 1) {
            throw new ConfigurationLoadException("Found multiple state files for
id: " + request.config.getId());
        } else if (matchingFiles.length == 0) {
            throw new FileNotFoundException("No state file found for id: " +
request.config.getId());
        }

        File stateFile = matchingFiles[0];
        loadedState = storedState.load(stateFile, request.outputClass);
    } catch (FileNotFoundException e) {
        log.info("{} '{}' is not yet provisioned. Will be created from
scratch", getType().getValue(), request.config.getName());
    } catch (IOException e) {
        loggingUtil.logError(log, ErrorCode.S3_STORED_STATE_ERROR, e);
        String message = "Failed to load persisted state for %s '%s'. File
might be unreadable or corrupted."
            .formatted(getType().getValue().toLowerCase(),
request.config.getName());
        throw new CloudProvisioningTerminationException(message, e);
    }

    StepResult<K> metadata = loadedState.stream()
        .filter(result ->
result.getStepName().equals(request.metadataStepName))
        .findFirst()
        .orElseGet(() -> request.metadataSupplier.apply(request.config));

    String name = request.nameExtractor.apply(metadata);
    String region = request.regionExtractor.apply(metadata);

    Injector injector = request.injectorFactory.apply(region, metadata);

    List<CloudProvisionStep<K>> allSteps =
request.stepProvider.apply(injector);
    if (loadedState.isEmpty()) {
        for (CloudProvisionStep<K> step : allSteps) {
            loadedState.add(step.getCurrentState());
        }
    }

    List<StepResult<K>> currentState =
request.liveStateGenerator.apply(injector);
    log.info("Comparing live state with the last persisted state to detect
configuration drift...");
    List<StepResult<K>> changed = stateComparator.diff(loadedState,

```

```

currentState);
    logProvisioningChanges(changed, currentState);

    CloudResourceDestroyer<K> destroyer =
request.destroyerProvider.apply(injector);
    CloudResourceProvisioner<K> provisioner =
request.provisionerProvider.apply(injector);
    CloudProvisionerSuccessfulResponse<K> applied;

    try {
        if (!request.config.isEnableReconciliation()) {
            log.info("Reconciliation is disabled for {} '{}'. Skipping
provisioning changes", getType().getValue().toLowerCase(), name);
            changed = new ArrayList<>();
        }

        applied = applyChanges(request.profile, name, request.config.getId(),
allSteps, destroyer, provisioner, changed, currentState, loadedState);
    } catch (CloudProvisioningTerminationException e) {

        handleRollback(injector, destroyer, request.reverterProvider, name,
request.profile, request.config.getId(), allSteps, loadedState,
request.liveStateGenerator);
        throw e;
    }

    List<StepResult<K>> desiredState =
request.desiredStateGenerator.apply(injector);
    log.info("Comparing desired configuration with the current live state to
identify necessary changes...");
    changed = stateComparator.diff(desiredState, loadedState);
    logProvisioningChanges(changed, currentState);

    try {
        applied = applyChanges(request.profile, name, request.config.getId(),
allSteps, destroyer, provisioner, changed, applied.getResults(),
loadedState);
    } catch (CloudProvisioningTerminationException e) {
        handleRollback(injector, destroyer, request.reverterProvider, name,
request.profile, request.config.getId(), allSteps, loadedState,
request.liveStateGenerator);
        throw e;
    }

    return applied;
}

protected List<StepResult<K>> generateLiveCloudState(LiveStateGenerator<K>
liveState) throws CloudProvisioningTerminationException {

    log.info("Retrieving the current live state of resources from the cloud
provider...");

    return liveState.generate();
}

protected List<StepResult<K>> generateDesiredState(DesiredStateGenerator<K>
desiredState)
    throws CloudProvisioningTerminationException {

```

```

        log.info("Computing the desired infrastructure state based on the current
configuration...");

        return desiredState.generate();
    }

    private CloudProvisionerSuccessfulResponse<K> applyChanges(
        String profile,
        String resourceName,
        String id,
        List<CloudProvisionStep<K>> allSteps,
        CloudResourceDestroyer<K> destroyer,
        CloudResourceProvisioner<K> provisioner,
        List<StepResult<K>> changes,
        List<StepResult<K>> referenceState,
        List<StepResult<K>> originState
    ) throws CloudProvisioningTerminationException {

        // Validate if changes are allowed
        verifyModificationRestrictions(changes, referenceState);

        // Determine which steps to apply
        List<CloudProvisionStep<K>> stepsToProvision =
            resolveStepsToProvision(allSteps, changes);

        // Determine which steps to execute
        List<CloudProvisionStep<K>> stepsToDelete =
            resolveStepsToDelete(allSteps, changes);

        CloudProvisionerSuccessfulResponse<K> destroyed = destroy(destroyer,
            stepsToDelete, true);

        if (isMainResourceDeleted(destroyed, profile, resourceName, id)) {
            return destroyed;
        }

        CloudProvisionerSuccessfulResponse<K> response = provision(provisioner,
            stepsToProvision);
        mergeAndPersistState(profile, resourceName, id, response, originState);

        return response;
    }

    private void handleRollback(
        Injector injector,
        CloudResourceDestroyer<K> destroyer,
        Function<Injector, CloudResourceReverter<K>> reverterProvider,
        String name,
        String profile,
        String id,
        List<CloudProvisionStep<K>> allSteps,
        List<StepResult<K>> loadedState,
        Function<Injector, List<StepResult<K>>> liveStateGenerator
    ) {

        try {
            log.info("Initiating rollback to the initial state to maintain
transactional consistency...");

```

```

        List<StepResult<K>> currentState = liveStateGenerator.apply(injector);
        List<StepResult<K>> changed = stateComparator.diff(loaderState,
currentState);
        logProvisioningChanges(changed, currentState);

        CloudResourceReverter<K> reverter = reverterProvider.apply(injector);
        revertChanges(profile, name, id, allSteps, destroyer, reverter,
changed, loaderState);
    } catch (CloudProvisioningTerminationException rollbackError) {
        loggingUtil.logError(log, ErrorCode.S3_PROVISION_ERROR, rollbackError);
    }
    log.warn("Provisioning failed for {} '{}', but all changes were rolled
back to the previously "
        + "persisted state. No partial modifications remain.",
getType().getValue().toLowerCase(), name);
}

```

```

@SuppressWarnings("UnusedReturnValue")
private CloudProvisionerSuccessfulResponse<K> revertChanges(
    String profile,
    String resourceName,
    String id,
    List<CloudProvisionStep<K>> allSteps,
    CloudResourceDestroyer<K> destroyer,
    CloudResourceReverter<K> reverter,
    List<StepResult<K>> changes,
    List<StepResult<K>> originState
) throws CloudProvisioningTerminationException {

    // Determine which steps to apply
    List<CloudProvisionStep<K>> stepsToProvision =
resolveStepsToProvision(allSteps, changes);

    // Determine which steps to execute
    List<CloudProvisionStep<K>> stepsToDelete =
resolveStepsToDelete(allSteps, changes);

    CloudProvisionerSuccessfulResponse<K> destroyed = destroy(destroyer,
stepsToDelete, false);

    if (isMainResourceDeleted(destroyed, profile, resourceName, id)) {
        return destroyed;
    }

    CloudProvisionerSuccessfulResponse<K> response = revert(reverter,
stepsToProvision, originState);
    mergeAndPersistState(profile, resourceName, id, response, originState);

    return response;
}

private void logDifferences(List<StepResult<K>> loaded, List<StepResult<K>>
changed) {

    Map<String, StepResult<K>> loadedMap = loaded.stream()
        .collect(Collectors.toMap(StepResult::getStepName, r -> r));

    Map<String, StepResult<K>> changedMap = changed.stream()
        .collect(Collectors.toMap(StepResult::getStepName, r -> r));

```

```

for (String stepName : changedMap.keySet()) {
    StepResult<K> newResult = changedMap.get(stepName);
    StepResult<K> oldResult = loadedMap.get(stepName);

    if (oldResult == null) {
        log.info("NEW Step '{}' is new:\n{}", stepName,
formatOutputs(newResult.getOutputs()));
        continue;
    }

    Map<K, Object> oldOutputs = oldResult.getOutputs();
    Map<K, Object> newOutputs = newResult.getOutputs();

    if (!oldOutputs.isEmpty() && newOutputs.isEmpty()) {
        log.info("🗑 Step '{}' has been cleared and will be deleted.",
stepName);
        continue;
    }

    List<String> diffs = new ArrayList<>();
    for (Map.Entry<K, Object> entry : newOutputs.entrySet()) {
        Object oldVal = oldOutputs.get(entry.getKey());
        Object newVal = entry.getValue();

        if (!Objects.equals(oldVal, newVal)) {
            diffs.add("↔ " + entry.getKey() + ": '" + oldVal + "' → '" +
newVal + "'");
        }
    }

    if (!diffs.isEmpty()) {
        log.info("🔄 Changes in step '{}':\n{}", stepName, String.join("\n",
diffs));
    }
}

private CloudProvisionerSuccessfulResponse<K>
destroy(CloudResourceDestroyer<K> destroyer,

List<CloudProvisionStep<K>> resources,                                boolean
enforcePreventDestroy)
    throws CloudProvisioningTerminationException {

    if (resources.isEmpty()) {
        log.info("Nothing to destroy. Continuing with provisioning phase.");
    }

    return destroyer.destroy(resources, enforcePreventDestroy);
}

private void mergeAndPersistState(
    String profile,
    String resourceName,
    String id,
    CloudProvisionerSuccessfulResponse<K> response,

```

```

        List<StepResult<K>> originState
    ) {
        List<StepResult<K>> mergedList = mergeStates(originState,
response.getResults());
        response.setResults(mergedList);

        File targetFile = new File(".cloudprovisioner/" + profile + "/" +
getType().toString().toLowerCase()
        + "/state-" + resourceName + "#" + id + ".json");
        File parentDir = targetFile.getParentFile();
        if (!parentDir.exists() && !parentDir.mkdirs()) {
            log.warn("Failed to create directories for persisting state: {}",
parentDir.getAbsolutePath());
        }

        try {
            stateWriter.write(targetFile, mergedList);
        } catch (StepResultStateWriteException e) {
            log.warn("Failed to persist step results for {} '{}'. "
+ "Provisioning changes were applied, but state was not saved.
Reason: {}",
            getType().getValue().toLowerCase(), resourceName, e.getMessage());
        }
    }

private CloudProvisionerSuccessfulResponse<K>
revert(CloudResourceReverter<K> reverter,

List<CloudProvisionStep<K>> resources,
List<StepResult<K>>
previous)
    throws CloudProvisioningTerminationException {

    if (resources.isEmpty()) {
        log.info("No changes detected. Skipping revert operation "
+ "as all resources are already in their previous state.");
    }

    return reverter.revert(resources, previous);
}

@Slf4j
public class CloudStepDeleteExecutor<K extends Enum<K>> {

    public List<StepResult<K>> execute(List<CloudProvisionStep<K>> steps,
boolean enforcePreventDestroy)
        throws InterruptedException, ExecutionException {

        List<StepResult<K>> results = new ArrayList<>();

        Map<Boolean, List<CloudProvisionStep<K>>> haveAsyncAnnotation =
StepExecutionUtils.classifySteps(steps);
        List<CloudProvisionStep<K>> async = haveAsyncAnnotation.get(true);
        List<CloudProvisionStep<K>> sync = haveAsyncAnnotation.get(false);

        results.addAll(StepExecutionUtils.executeAsync(async, step -> () ->
step.destroy(enforcePreventDestroy)));
        results.addAll(StepExecutionUtils.executeSyncDescending(sync, step ->
step.destroy(enforcePreventDestroy)));

        return results;
    }
}

```

```

    }
}

@Slf4j
public class CloudStepRevertExecutor<K extends Enum<K>> {

    private final BaseCloudResourceConfiguration.RetryConfiguration retry;

    public
    CloudStepRevertExecutor(BaseCloudResourceConfiguration.RetryConfiguration
    retry) {
        this.retry = retry;
    }

    public List<StepResult<K>> execute(List<CloudProvisionStep<K>> steps,
    List<StepResult<K>> previousResults)
        throws InterruptedException, ExecutionException {

        List<StepResult<K>> results = new ArrayList<>();

        Map<Boolean, List<CloudProvisionStep<K>>> haveAsyncAnnotation =
        StepExecutionUtils.classifySteps(steps);
        List<CloudProvisionStep<K>> async = haveAsyncAnnotation.get(true);
        List<CloudProvisionStep<K>> sync = haveAsyncAnnotation.get(false);

        results.addAll(StepExecutionUtils.executeAsync(async, step -> () ->
        executeWithRetry(step, previousResults)));
        results.addAll(StepExecutionUtils.executeSyncDescending(sync, step ->
        executeWithRetry(step, previousResults)));

        return results;
    }

    private StepResult<K> executeWithRetry(CloudProvisionStep<K> step,
    List<StepResult<K>> previousResults) {

        // Find previous result for the step, if any
        StepResult<K> previous = previousResults.stream()
            .filter(r -> r.getStepName().equals(step.getClass().getName()))
            .findFirst()
            .orElse(null);

        int attempts = 0;
        while (true) {
            try {
                return step.revert(previous);
            } catch (RetryableCloudResourceStepException e) {
                attempts++;
                CloudExceptionHandler.handleRetryableException(e, step, attempts,
                retry.getMax(), retry.getBackoffMs());
            } catch (CloudResourceStepException e) {
                // Propagate the non-retryable exception
                throw e;
            } catch (Exception ex) {
                throw CloudExceptionHandler.handleUnexpectedException(ex, step,
                attempts);
            }
        }
    }
}

```

```

}
@Slf4j
public class CloudStepStrategyExecutor<K extends Enum<K>> {

    private final BaseCloudResourceConfiguration.RetryConfiguration retry;

    public
    CloudStepStrategyExecutor(BaseCloudResourceConfiguration.RetryConfiguration
    retry) {
        this.retry = retry;
    }

    public List<StepResult<K>> execute(List<CloudProvisionStep<K>> steps,
    StepExecutionStrategy<K> strategy)
        throws InterruptedException, ExecutionException {

        List<StepResult<K>> results = new ArrayList<>();

        Map<Boolean, List<CloudProvisionStep<K>>> haveAsyncAnnotation =
        StepExecutionUtils.classifySteps(steps);
        List<CloudProvisionStep<K>> async = haveAsyncAnnotation.get(true);
        List<CloudProvisionStep<K>> sync = haveAsyncAnnotation.get(false);

        // TODO [Documentation] Document usage of strategy pattern
        results.addAll(StepExecutionUtils.executeSync(sync, step ->
        executeWithRetry(step, strategy)));
        results.addAll(StepExecutionUtils.executeAsync(async, step -> () ->
        executeWithRetry(step, strategy)));

        return results;
    }

    private StepResult<K> executeWithRetry(CloudProvisionStep<K> step,
    StepExecutionStrategy<K> strategy) {

        int attempts = 0;
        while (true) {
            try {
                return strategy.execute(step);
            } catch (RetryableCloudResourceStepException e) {
                attempts++;
                CloudExceptionHandler.handleRetryableException(e, step, attempts,
                retry.getMax(), retry.getBackoffMs());
            } catch (CloudResourceStepException e) {
                // Propagate the non-retryable exception
                throw e;
            } catch (Exception ex) {
                throw CloudExceptionHandler.handleUnexpectedException(ex, step,
                attempts);
            }
        }
    }
}

public class StepExecutionUtils {

    public static <K extends Enum<K>> Map<Boolean,
    List<CloudProvisionStep<K>>> classifySteps(
        List<CloudProvisionStep<K>> steps) {

```

```

        return steps.stream().collect(Collectors.partitioningBy(
            step ->
            step.getClass().isAnnotationPresent(ProvisionAsync.class)));
    }

    public static <K extends Enum<K>> List<StepResult<K>> executeAsync(
        List<CloudProvisionStep<K>> steps,
        Function<CloudProvisionStep<K>, Callable<StepResult<K>>> toCallable
    ) throws InterruptedException, ExecutionException {

        List<Callable<StepResult<K>>> tasks =
        steps.stream().map(toCallable).toList();

        try (ExecutorService executor = Executors.newFixedThreadPool(4)) {
            List<Future<StepResult<K>>> futures = executor.invokeAll(tasks);
            List<StepResult<K>> results = new ArrayList<>();
            for (Future<StepResult<K>> future : futures) {
                results.add(future.get());
            }
            return results;
        }
    }

    public static <K extends Enum<K>> List<StepResult<K>> executeSync(
        List<CloudProvisionStep<K>> steps,
        Function<CloudProvisionStep<K>, StepResult<K>> executor
    ) {

        return steps.stream()
            .sorted(Comparator.comparingInt(s ->
            s.getClass().getAnnotation(ProvisionOrder.class).value()))
            .map(executor)
            .toList();
    }

    public static <K extends Enum<K>> List<StepResult<K>>
    executeSyncDescending(
        List<CloudProvisionStep<K>> steps,
        Function<CloudProvisionStep<K>, StepResult<K>> executor
    ) {

        return steps.stream()
            .sorted((s1, s2) -> {
                int order1 =
                s1.getClass().getAnnotation(ProvisionOrder.class).value();
                int order2 =
                s2.getClass().getAnnotation(ProvisionOrder.class).value();
                return Integer.compare(order2, order1);
            })
            .map(executor)
            .toList();
    }
}

```